COMPUTER VISION

计算机视觉

课程讲义

成像几何与图像处理
局部特征与多尺度表示
图像拼接与三维几何

2026 年秋季

目录

第 1 章　图像与视觉任务 **1**

1.1　成像与逆问题 1

1.2　图像坐标与变换 2

第 2 章　图像形成 **3**

2.1　针孔模型与相机矩阵 3

2.2　消失点与消失线 5

2.3　透镜、景深与视场 6

2.4　镜头畸变 7

2.5　颜色、采样与图像信号 8

第 3 章　图像处理 **10**

3.1　点运算与直方图 10

3.2　线性滤波 11

3.3　非线性滤波与模板匹配 14

3.4　采样、混叠与高斯金字塔 16

3.5　图像插值 18

3.6　拉普拉斯金字塔与多频带融合 20

3.7　几何变换与图像形变 22

第 4 章　边缘、角点与局部特征 **25**

4.1　梯度与 Canny 边缘检测 25

4.2　Harris 角点检测 28

4.3　尺度空间与 LoG 30

4.4　DoG 检测 32

4.5　SIFT 描述子与匹配 34

4.6　HOG 描述子 36

第 5 章 图像拼接 **38**
5.1 图像间的变换 . . . 38
5.2 DLT 单应估计 . . . 40
5.3 RANSAC . . . 41
5.4 画布与图像融合 . . . 43
5.5 泊松融合 . . . 43

第 6 章 相机标定与三角测量 **46**
6.1 相机矩阵估计 . . . 46
6.2 内参与外参 . . . 47
6.3 三角测量 . . . 49
6.4 消失点标定 . . . 51

第 7 章 对极几何与双视图重建 **54**
7.1 极平面、极线与极点 . . . 54
7.2 本质矩阵 . . . 55
7.3 基础矩阵 . . . 58
7.4 三种常见的相机位置 . . . 59
7.5 八点法 . . . 61
7.6 误差与错误匹配 . . . 63
7.7 相机位姿 . . . 65

第 8 章 双视图立体匹配 **69**
8.1 视差与深度 . . . 69
8.2 立体校正 . . . 71
8.3 局部立体匹配 . . . 74
8.4 遮挡与匹配约束 . . . 76
8.5 扫描线动态规划 . . . 77
8.6 整幅图像的视差优化 . . . 81
8.7 双目深度的计算示例 . . . 82
8.8 学习匹配代价与深度 . . . 84
8.9 主动立体视觉 . . . 86

第 1 章　图像与视觉任务

相机把场景记录成图像，计算机视觉则利用图像判断物体的位置、形状和类别。先了解成像过程，有助于说明这些判断需要哪些数据和假设。

1.1　成像与逆问题

同一个物体，换个角度或换盏灯拍摄，照片就会变化。反过来，两张看起来相似的照片，也可能来自不同的物体。为了研究图像与场景之间的关系，把几何、材料、光照和相机参数合记为 z，成像过程写成

$$I = \mathcal{F}(z) + \varepsilon. \tag{1.1}$$

$\mathcal{F}$ 根据场景参数生成图像，I 是相机的观测，ε 表示噪声和模型误差。给定场景求图像，是成像的正向问题；根据图像估计场景参数，则是逆问题。

下图给出了同一明暗图案的几种来源。它可以画在平面上，也可以由光照或物体表面的起伏产生。这些场景都能解释图中的像素，因此我们还需要其他观测或已有知识来选择解释。

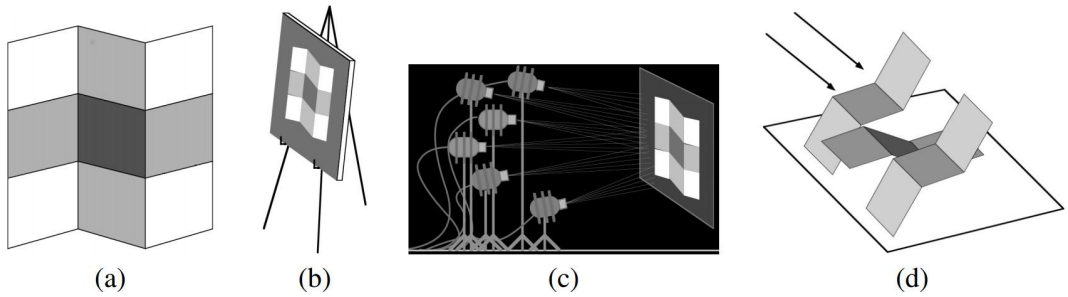


图 1.1　相似的明暗图案可以来自表面的颜色、光照变化或物体形状。

物体大小与距离也有类似的关系：小物体放在近处、大物体放在远处，可以在照片中占据相同的大小。下一章会用针孔模型算出这种关系。要进一步测量物体，就可以加入已知尺寸作为参照，或从另一个位置再拍一张图。

求解时，常把图像中的观测与已有知识一起写进目标函数：

$$\hat{z} = \arg\min_{z} \underbrace{D\big(\mathcal{F}(z), I\big)}_{\text{观测误差}} + \lambda \underbrace{\mathcal{R}(z)}_{\text{先验约束}}. \tag{1.2}$$

D 衡量预测图像与实际图像的差别，$\mathcal{R}$ 根据已有知识对结果作约束。$\arg\min$ 表示取使整个表达式最小的 z，$\lambda \geq 0$ 决定第二项的作用有多大。

例如，图像去噪既要保留观测到的内容，也希望同一平滑区域内的像素值接近。前一项让结果靠近原图，后一项减少相邻像素之间的剧烈波动。增大 λ 会加强平滑：残留的噪声通常变少，细节也可能随之变模糊。

数学直觉 ｜ 额外观测与先验

从另一位置拍摄，能为深度计算增加一条视线；已知光源方向，能帮助判断表面朝向；假设图像在局部平滑，能帮助估计受噪声干扰的像素。训练数据也可以提供这样的知识，例如某类物体常见的形状和纹理。这些信息分别适用于不同任务，使用时需要说明相应的假设。

1.2 图像坐标与变换

后面会同时用到三维点的位置、二维像素的位置和像素值，先约定它们的记号。灰度图像写成 $I:\Omega\subset\mathbb{Z}^2\to\mathbb{R}$，即每个像素位置对应一个灰度值。彩色图像写成 $I:\Omega\to\mathbb{R}^3$，每个位置对应三个通道的值。常见的存储方式有 0 到 255 的整数，也有归一化后的浮点数。

记号	含义与约定
$\boldsymbol{X}_w,\boldsymbol{X}_c\in\mathbb{R}^3$	世界坐标、相机坐标；相机前方取 $Z_c>0$。
$\boldsymbol{x}=(u,v)^{\mathsf{T}}$	图像坐标：u 向右，v 向下；数组访问为 `I[v, u]`。
$\tilde{\boldsymbol{x}}=(u,v,1)^{\mathsf{T}}$	二维齐次坐标；$\sim$ 表示相差一个非零比例因子。
K, R, $\boldsymbol{t}$	相机内参、世界到相机的旋转、世界到相机的平移。

齐次坐标

二维点 $(2,3)$ 可以写成三维向量 $(2,3,1)^{\mathsf{T}}$，也可以写成 $(4,6,2)^{\mathsf{T}}$。将前两个分量除以最后一个分量，就都得到 $(2,3)$。一般地，$(a,b,c)^{\mathsf{T}}$ 在 $c\neq0$ 时表示二维点 $(a/c,b/c)$。这样，整体相差一个非零倍数的向量就表示同一个点，我们用 $\sim$ 记下这种关系。

添加一个分量后，还能将平移写进矩阵乘法。以三维点为例，

$$\begin{bmatrix}R & \boldsymbol{t}\\ \boldsymbol{0}^{\mathsf{T}} & 1\end{bmatrix}\begin{bmatrix}\boldsymbol{X}\\ 1\end{bmatrix}=\begin{bmatrix}R\boldsymbol{X}+\boldsymbol{t}\\ 1\end{bmatrix}.$$

右侧的前三个分量就是旋转、平移后的坐标。下一章会继续用这种写法，把坐标变换和相机投影连起来。

等变性与不变性

假设把一张图像向右平移十个像素。检测器找到的角点位置也应向右移动十个像素，这种位置随图像一起变化的性质称为*等变性*。若要判断移动前后是不是同一个角点，则希望它的局部描述保持相似，这称为近似*不变性*。

设 T 将旧坐标变到新坐标。新图像在 $\boldsymbol{x}$ 处的像素来自原图的 $T^{-1}\boldsymbol{x}$，因此记 $(T\cdot I)(\boldsymbol{x})=I(T^{-1}\boldsymbol{x})$。再用 $\mathcal{D}(I)$ 表示检测到的关键点集合，$\phi(I,\boldsymbol{x})$ 表示位置 $\boldsymbol{x}$ 处的描述子，前面的两种性质就写成

$$\mathcal{D}(T\cdot I)\approx T\cdot\mathcal{D}(I),\qquad \phi(T\cdot I,T\boldsymbol{x})\approx\phi(I,\boldsymbol{x}).$$

第四章的 SIFT 会先根据图像内容确定关键点的位置、尺度和方向，再在相应的局部坐标系中计算描述子，以便比较不同照片中的同一处物体。

推导时，我们常把亮度写成连续函数，这样就可以计算导数。实际图像由离散像素组成，程序会用邻域差分、插值等运算实现这些计算。

第 2 章　图像形成

这一章先计算空间点在图像中的位置，再介绍透镜怎样对焦、传感器怎样记录光线。相机的几何参数决定像点的位置，光学和传感过程则影响照片的清晰度与颜色。

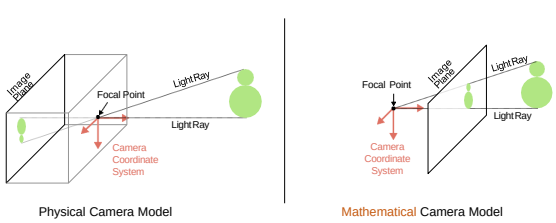
2.1　针孔模型与相机矩阵

给定一个空间点，我们希望算出它在照片中的像素位置。先把点变换到相机坐标系，再投影到像平面，最后换成像素单位。这里从针孔模型开始：光线沿直线通过一个小孔，落在像平面上。

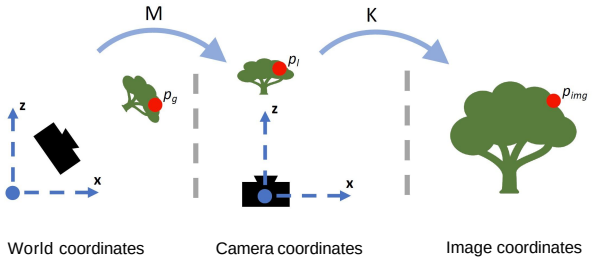
设光心为原点，Z_c 轴朝前。把数学像平面放在光心前方距离 f 处，空间点 (X_c, Y_c, Z_c)、光心与像点共线。由相似三角形，$x/f = X_c/Z_c$、$y/f = Y_c/Z_c$，因此

$$x = f\frac{X_c}{Z_c}, \qquad y = f\frac{Y_c}{Z_c}, \qquad Z_c > 0. \tag{2.1}$$

这些量使用相同的长度单位，(x, y) 表示像平面上的位置。固定 X_c, Y_c 而增大 Z_c，像点向中心靠近，正是近大远小的来源。



真实倒像与数学正像



世界、相机与图像坐标

图 2.1　左：将数学像平面置于光心前方，方便写投影公式。右：空间点先变到相机坐标，再经投影和内参得到像素坐标。

外参

空间点通常记录在世界坐标中。使用投影公式前，先将它换到相机坐标系。外参描述这两个坐标系的关系：$R \in \mathbb{R}^{3\times 3}$ 负责旋转，$\boldsymbol{t} \in \mathbb{R}^3$ 负责平移，

$$\boldsymbol{X}_c = R\boldsymbol{X}_w + \boldsymbol{t}, \qquad R^{\mathsf{T}}R = I_3, \quad \det R = 1. \tag{2.2}$$

I_3 为三阶单位矩阵，旋转矩阵满足 $R^{-1} = R^{\mathsf{T}}$。相机光心在自己的坐标系中为零，所以它的世界坐标 $\boldsymbol{C}$ 满足 $R\boldsymbol{C} + \boldsymbol{t} = 0$。两侧左乘 R^{T}，便得到

$$\boldsymbol{C} = -R^{\mathsf{T}}\boldsymbol{t}, \qquad \boldsymbol{X}_c = R(\boldsymbol{X}_w - \boldsymbol{C}).$$

这里 $\boldsymbol{C}$ 描述相机在世界中的位置，$\boldsymbol{t}$ 则用于将世界坐标变到相机坐标。若给定的是相机到世界的变换 $\boldsymbol{X}_w = R_{cw}\boldsymbol{X}_c + \boldsymbol{C}$，对它求逆便得到 $R = R_{cw}^{\mathsf{T}}$、$\boldsymbol{t} = -R_{cw}^{\mathsf{T}}\boldsymbol{C}$。

内参

接下来需要知道，像平面上的长度怎样换成像素，以及光轴落在图像中的位置。先除以深度得到归一化坐标 $x_n = X_c/Z_c$、$y_n = Y_c/Z_c$，再缩放和平移：$u = f_x x_n + s y_n + c_x$，$v = f_y y_n + c_y$。写成齐次形式就是

$$K = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}, \qquad \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} x_n \\ y_n \\ 1 \end{bmatrix}. \tag{2.3}$$

$f_x = f/p_x$、$f_y = f/p_y$ 是像素单位的焦距，p_x, p_y 为像素间距。(c_x, c_y) 是光轴与像平面的交点，称为主点，通常靠近图像中心。偏斜参数 s 通常取 0。

把前面的步骤连起来。令 $\tilde{\boldsymbol{X}}_w = (X_w, Y_w, Z_w, 1)^{\mathsf{T}}$，有

$$K(R\boldsymbol{X}_w + \boldsymbol{t}) = K \begin{bmatrix} X_c \\ Y_c \\ Z_c \end{bmatrix} = \begin{bmatrix} f_x X_c + s Y_c + c_x Z_c \\ f_y Y_c + c_y Z_c \\ Z_c \end{bmatrix} = Z_c \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}.$$

这说明未做最后除法时，矩阵输出是像素齐次坐标的 Z_c 倍，于是

$$\boxed{\lambda \tilde{\boldsymbol{x}} = K[R \mid \boldsymbol{t}]\, \tilde{\boldsymbol{X}}_w = P \tilde{\boldsymbol{X}}_w.} \tag{2.4}$$

这里 $\lambda = Z_c$，P 为 3×4 矩阵。**将输出的前两个分量除以第三个分量，就得到像素坐标**。例如 $s = 0$、$f_x = f_y = 800$、主点为 $(320, 240)$ 时，相机坐标 $(0.2, 0.1, 2)$ 投到 $(400, 280)$；把三维坐标整体乘以二，像素位置不变。

> **数学直觉 ｜ 反投影射线**
>
> 从 $Z_c \tilde{\boldsymbol{x}} = K \boldsymbol{X}_c$ 左乘 K^{-1}，再用 $\boldsymbol{X}_w = R^{\mathsf{T}}(\boldsymbol{X}_c - \boldsymbol{t})$ 返回世界坐标，就得到
>
> $$\boldsymbol{X}_c = Z_c K^{-1} \tilde{\boldsymbol{x}}, \qquad \boldsymbol{X}_w = \boldsymbol{C} + Z_c R^{\mathsf{T}} K^{-1} \tilde{\boldsymbol{x}}.$$
>
> 给定像素后，改变 Z_c 就得到这条射线上的不同空间点。例如，焦距 $f_x = 800$ 像素、$u - c_x = 80$ 像素时，$X_c/Z_c = 0.1$。取 $Z_c = 2$ 米得到 $X_c = 0.2$ 米，取 $Z_c = 5$ 米得到 $X_c = 0.5$ 米，二者投影到同一个像素。

弱透视与正交投影

对远处而深度起伏较小的物体，逐点除以 Z_c 有时可以简化。写 $Z_c = \bar{Z} + \Delta Z$，当 $|\Delta Z| \ll \bar{Z}$ 时，$1/(\bar{Z} + \Delta Z) \approx 1/\bar{Z}$，所以

$$x \approx \frac{f}{\bar{Z}} X_c, \qquad y \approx \frac{f}{\bar{Z}} Y_c.$$

弱透视用统一的 $f/\bar{Z}$ 代替逐点尺度；正交投影则直接保留 (X_c, Y_c)。近似是否合适，取决于物体的相对深度变化。

算法实现 ｜ 投影三维点

输入 `Xw` 按行存放 N 个点，所以代码用 `R.T` 实现式 (2.2) 的列向量变换。

```
import numpy as np

Xc = Xw @ R.T + t
valid = Xc[:, 2] > 1e-8
q = Xc[valid] @ K.T
uv = q[:, :2] / q[:, 2:3]
```

代码保留深度大于阈值的点。计算完像素位置后，再按图像宽高检查边界；若要判断可见性，还需根据场景几何处理遮挡。

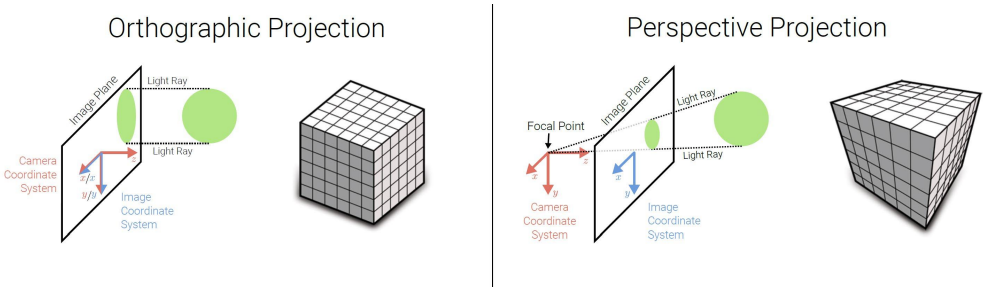


图 2.2 正交投影忽略远近带来的尺度变化；透视投影中，远处的网格与边长更小。弱透视是在深度变化较小时，用统一尺度近似右侧模型。

2.2 消失点与消失线

照片里的铁轨为什么在远方汇合，而笔直的铁轨仍然是直线？取一条不过光心的空间直线，它与光心确定一个平面。这个平面和像平面相交于一条直线，原直线上的各点都投影到这条交线上。因此，透视投影保留共线关系。长度和夹角则会随拍摄位置与朝向改变。

现在把一组平行线写成 $\boldsymbol{X}(\tau) = \boldsymbol{X}_0 + \tau \boldsymbol{d}$，其中 $\boldsymbol{d}$ 是共同方向。沿直线走向远处，即 $\tau \to \infty$。利用齐次坐标可整体缩放的性质，

$$\begin{bmatrix} \boldsymbol{X}_0 + \tau \boldsymbol{d} \\ 1 \end{bmatrix} \sim \begin{bmatrix} \boldsymbol{X}_0/\tau + \boldsymbol{d} \\ 1/\tau \end{bmatrix} \longrightarrow \begin{bmatrix} \boldsymbol{d} \\ 0 \end{bmatrix}.$$

最后一个分量变为零，得到方向 $\boldsymbol{d}$ 对应的无穷远点。将它代入相机矩阵，

$$\tilde{\boldsymbol{v}} \sim K[R \mid \boldsymbol{t}] \begin{bmatrix} \boldsymbol{d} \\ 0 \end{bmatrix} = KR\boldsymbol{d}. \tag{2.5}$$

末尾的零使平移项消去，剩下 $KR\boldsymbol{d}$。因此，**同一方向的平行线共享一个消失点**，它的位置由方向、相机朝向和内参决定。若 $(R\boldsymbol{d})_3 = 0$，消失点位于无穷远，对应的像直线互相平行。

同一个平面内可以有许多方向，这些方向的消失点又有什么关系？设平面在世界坐标中的法向量为 $\boldsymbol{n}$，则面内方向满足 $\boldsymbol{n}^{\mathsf{T}}\boldsymbol{d} = 0$。由 $\tilde{\boldsymbol{v}} \sim KR\boldsymbol{d}$ 得 $\boldsymbol{d} \sim R^{\mathsf{T}}K^{-1}\tilde{\boldsymbol{v}}$，代回可得

$$\boldsymbol{n}^{\mathsf{T}}R^{\mathsf{T}}K^{-1}\tilde{\boldsymbol{v}} = 0.$$

这是像平面上一条直线的方程，故

$$\boldsymbol{\ell}_\infty \sim K^{-\mathsf{T}}R\boldsymbol{n}, \qquad \boldsymbol{\ell}_\infty^{\mathsf{T}}\tilde{\boldsymbol{v}} = 0. \tag{2.6}$$

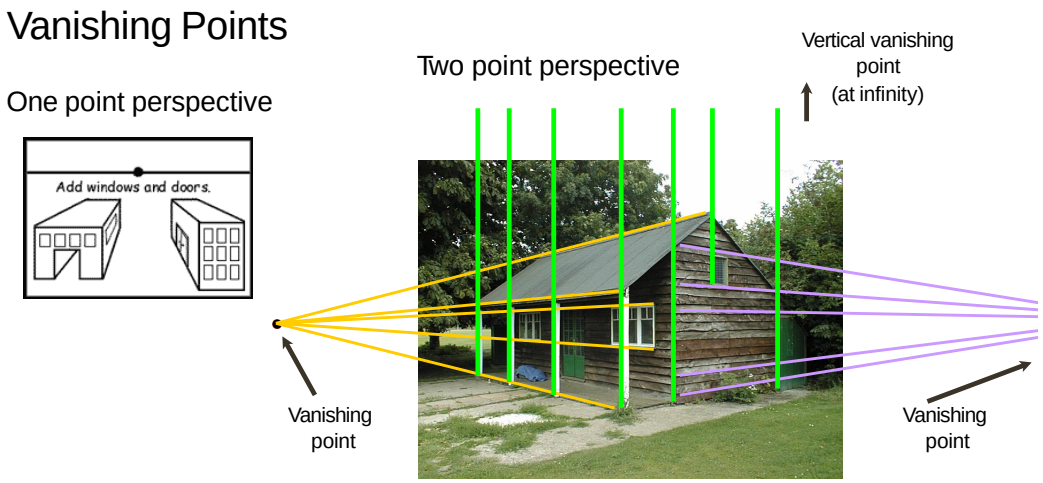


图 2.3 不同方向对应不同消失点。右图中的竖直方向在像平面上保持平行，对应无穷远消失点。

这里 $\ell = (a, b, c)^{\mathsf{T}}$ 表示直线 $au + bv + c = 0$。所有面内方向的消失点都在这条消失线上；地平线就是一个常见例子。

> **数学直觉 ｜ 大小与距离**
> 若一段长度为 H 的线段平行于像平面，其投影长度为 $h = fH/Z$。将 H 和 Z 同时加倍，h 保持不变。所以测量实际大小时，需要已知尺寸、相机高度等参照，或结合其他视角和已知基线进行计算。

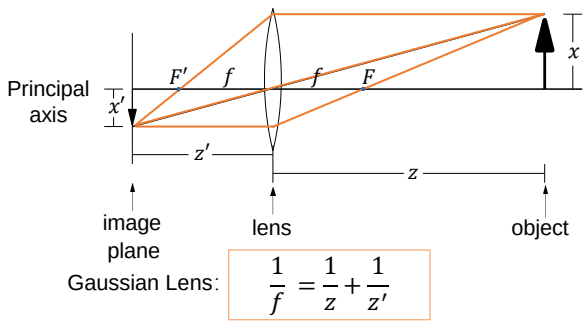
2.3 透镜、景深与视场

真实相机通常用透镜收集光线。透镜将来自物体同一点的光线汇聚，汇聚位置随物体距离改变。把传感器放到对应位置，就能得到清晰的像；偏离这个位置时，点会在传感器上形成模糊光斑。相比单纯扩大针孔，透镜可以通过折射汇聚光线，让相机在较大进光量下清晰成像。

用 z_o 表示物距、z_i 表示像距、f 表示焦距，距离均从薄透镜处测量。对实物、实像取正距离，薄透镜方程为

$$\frac{1}{f} = \frac{1}{z_o} + \frac{1}{z_i}, \qquad z_i = \frac{f z_o}{z_o - f}. \tag{2.7}$$

横向放大率的绝对值为 $|m| = z_i / z_o$，实像相对于物体倒立。当 $z_o \to \infty$ 时 $z_i \to f$；对焦到更近的物体时，理想像距会增大。



透镜把光线汇聚到像平面

收小光圈，清晰深度范围扩大

图 2.4 透镜光路与景深效果。左图 z, z' 对应 z_o, z_i；右图由左到右光圈缩小，背景更清晰。

图中两组相似三角形给出 $h'/h = z_i/z_o$ 与 $h'/f = (h+h')/z_o$，合并即得薄透镜方程。远距离对焦时 $z_i \approx f$；近距离对焦时，按式 (2.7)计算相应的像距。

景深

设传感器固定在 $z_i^\star$，孔径直径为 D。一个点本应在 z_i 汇聚，光线到达传感器时仍占据一块圆形区域，称为弥散圆。光锥在透镜处直径为 D，到汇聚点时缩为零，故离汇聚点 $|z_i - z_i^\star|$ 处的截面满足 $c/D = |z_i - z_i^\star|/z_i$，即

$$c = D\frac{|z_i - z_i^\star|}{z_i}. \tag{2.8}$$

给定可接受阈值 $c_{\max}$，满足 $c \le c_{\max}$ 的物距区间就是景深。可接受的阈值会随照片的显示尺寸和观看要求改变，因此同一镜头设置下，景深还与这些条件有关。

光圈数定义为 $N = f/D$。固定焦距、对焦距离和清晰度阈值时，缩小孔径通常增加景深，同时减少进入相机的光。延长曝光可以收集更多光子，但运动的物体可能留下拖影；提高增益则会把记录到的信号和噪声一起放大。孔径很小时，衍射还会使细节变模糊。选择光圈时，需要根据景深、光照和物体运动情况作取舍。

视场

景深描述哪些距离清晰，视场则描述能看见多宽的范围。从光心看传感器两端，半个视场角 $\alpha/2$ 满足 $\tan(\alpha/2) = (d/2)/z_i$，其中 d 是该方向的传感器尺寸。因此

$$\mathrm{FoV} = 2\arctan\frac{d}{2z_i} \approx 2\arctan\frac{d}{2f}. \tag{2.9}$$

右侧近似适用于远距离对焦。同一传感器上，焦距越长，视场越窄。在无偏斜、主点居中的像素坐标模型中，水平视场约为 $2\arctan(W/(2f_x))$，W 为图像宽度。

> **备注 | 透视变化与镜头畸变**
> 在理想中心投影下，相机位置固定时，更换焦距主要改变视场和图像尺度。为了让人脸在画面中一样大，广角拍摄通常需要更靠近人物，这个拍摄距离变化会改变鼻子与耳朵的相对透视大小。这种变化来自拍摄位置，下一节的径向畸变则来自镜头对光线的偏折。

2.4 镜头畸变

广角照片中的直线有时会向外鼓或向内弯。这种镜头畸变可以看成像点相对于理想投影位置的偏移。径向模型用距光轴的距离描述这种偏移：先在归一化像平面上计算半径 r 和缩放因子 $a(r)$，

$$r^2 = x_n^2 + y_n^2, \qquad a(r) = 1 + k_1 r^2 + k_2 r^4 + k_3 r^6, \tag{2.10}$$

若只考虑径向畸变，便有 $(x_d, y_d) = a(r)(x_n, y_n)$：离中心越远，高次项影响越大。镜头与传感器不完全对准时，还可加入切向偏移参数 p_1, p_2：

$$x_d = x_n a(r) + 2p_1 x_n y_n + p_2(r^2 + 2x_n^2), \tag{2.11}$$

$$y_d = y_n a(r) + p_1(r^2 + 2y_n^2) + 2p_2 x_n y_n. \tag{2.12}$$

这里的 r 在归一化像平面中计算，得到 (x_d, y_d) 后，再用 K 换成像素坐标。

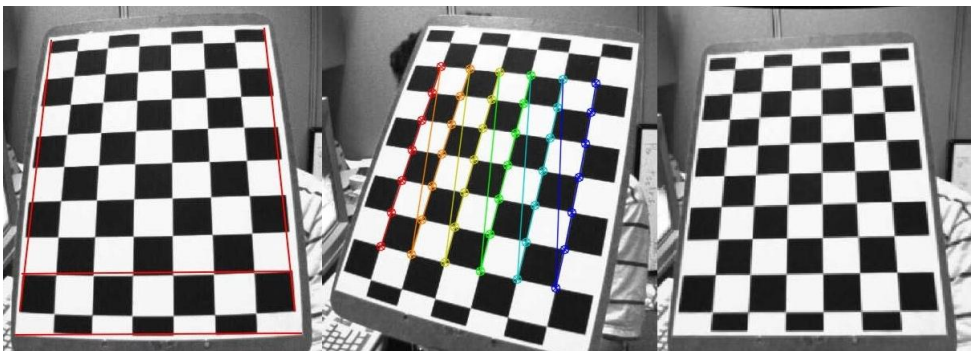


图 2.5 用标定板检查畸变校正的效果：原来弯曲的格线经过校正后恢复为直线。

当小半径范围内 k_1 起主导作用时，在上述“理想到畸变”的映射约定下，$k_1 < 0$ 对应桶形趋势，$k_1 > 0$ 对应枕形趋势。大视场和复杂镜头可能需要更高阶或不同的模型。

校正时遍历输出的理想像素，用畸变模型寻找输入位置，再插值取样。这与后面的逆向采样思想相同。

算法实现 | 畸变校正

```
import cv2 as cv
import numpy as np

dist = np.array([k1, k2, p1, p2, k3], dtype=float)
corrected = cv.undistort(image, K, dist, None, K)
```

K 和畸变参数需事先标定。若图像已缩放或裁剪，内参也要相应更新：缩放改变焦距的像素数值与主点坐标，裁剪改变主点偏移。

色差使不同颜色聚焦位置不同，常表现为彩边；暗角使边缘变暗，可近似写为 $I_{\text{obs}}(\boldsymbol{x}) = v(\boldsymbol{x}) I_{\text{ideal}}(\boldsymbol{x})$。这两种现象分别影响颜色和亮度。

2.5 颜色、采样与图像信号

同一件衣服在不同灯光下会呈现不同的颜色。要解释相机记录的 RGB 数值，需要一起考虑物体反射的光、光源的光谱和传感器的响应。一个通道会按自身的敏感度，对不同波长 λ 的光加权累积：

$$c_j(\boldsymbol{x}) = \int s_j(\lambda)\, \ell(\boldsymbol{x}, \lambda)\, \mathrm{d}\lambda, \qquad j \in \{R, G, B\}, \tag{2.13}$$

其中 ℓ 是到达传感器的光谱信号，s_j 是通道的光谱敏感度。在简化的反射模型中，ℓ 同时受光源光谱、材料反射率和表面几何影响。

RGB 用三个通道概括光谱。**不同光谱也可能产生相同的三通道响应**，称为同色异谱；固定三个物理原色的非负混合也只能覆盖有限色域。

RGB 常用于存储和显示。HSV 将色相、饱和度和明暗分开，便于按颜色筛选区域；接近灰色时饱和度很低，色相容易随微小误差变化。Lab 用来近似描述人眼感受到的颜色差异，计算时需要给定参考白点。

传感器与颜色采样

一个简化的成像链路是

$$\text{光学成像} \longrightarrow \text{曝光与传感} \longrightarrow \text{去马赛克、白平衡等处理} \longrightarrow \text{非线性编码与量化}.$$

传感器在有限面积和曝光时间内积累光信号。Bayer 阵列每个 2×2 单元含两个绿、一个红、一个蓝采样点；每个位置只直接测得一个通道，缺失通道通过邻域插值补全。

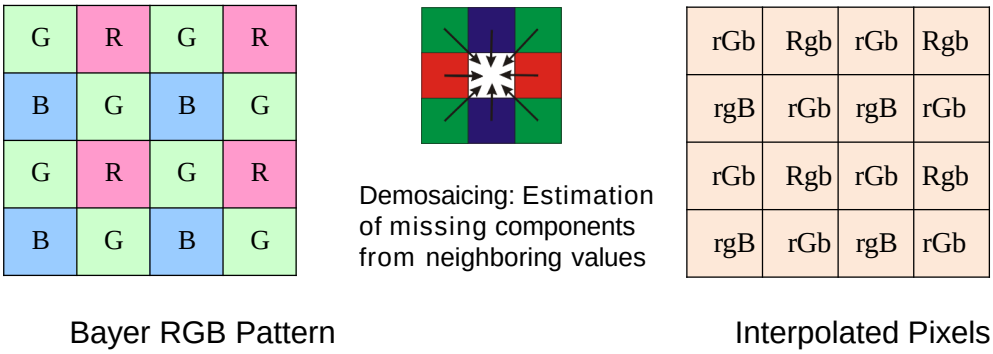


图 2.6 Bayer 阵列每个位置只测得一种颜色。去马赛克利用邻居补全另外两个通道，因此彩色像素既含直接观测，也含插值估计。

若光电子计数服从泊松分布 $n \sim \mathrm{Poisson}(\mu)$，则 $\mathbb{E}[n] = \mathrm{Var}(n) = \mu$，于是标准差与均值之比为 $\sqrt{\mu}/\mu = 1/\sqrt{\mu}$。收集到的光子越多，相对噪声越小。提高数字增益则会将已有的信号和噪声一起放大。

Gamma 编码

图像保存时通常会对光强作非线性编码。进行混色等光强运算前，需要先将编码值换回线性光强。令 L 为归一化的线性光强，V 为编码值，一种简化模型是

$$V = L^{1/\gamma}, \qquad L = V^{\gamma}, \qquad L, V \in [0, 1]. \tag{2.14}$$

通常取 $\gamma > 1$，例如 $\gamma = 2.2$ 时，较暗的 L 值会被映到更大的编码值。编码在暗部安排了更密集的量化层级。实际 sRGB 使用分段传递函数，上式是便于理解的近似模型。

> **数学直觉 ｜ 线性光强下的混色**
>
> 把编码值 0 和 1 直接平均得到 $V = 0.5$，解码后是 $L = 0.5^{2.2} \approx 0.218$。若要混合一半黑、一半白的线性光强，正确顺序是先解码、在线性域平均得到 $L = 0.5$，再编码为 $V = 0.5^{1/2.2} \approx 0.73$。两种次序产生不同的亮度，因此处理图像时要先明确，数组保存的是线性光强还是编码值。

第 3 章　图像处理

图像处理包括调整亮度、去除噪声、缩放和变形等操作。本章先介绍单个像素的运算，再用邻域加权实现滤波，随后讨论采样、插值和金字塔。

3.1　点运算与直方图

一张照片偏暗时，可以通过改变像素值来调整亮度。如果每个输出值都由同一位置的输入值决定，就称为点运算，写成 $J(\boldsymbol{x}) = g(I(\boldsymbol{x}))$。仿射变换 $g(r) = ar + b$ 中，b 改变整体亮度，a 改变像素间的差值：

$$J(\boldsymbol{x}_1) - J(\boldsymbol{x}_2) = a\big[I(\boldsymbol{x}_1) - I(\boldsymbol{x}_2)\big].$$

上式描述实数运算的结果。保存为八位图像时，超过 255 的值会饱和，小于 0 的值会截断；被归到同一个端点的像素就会失去原来的亮度差别。

若大多数像素集中在某个灰度区间，可以先统计分布，再设计非线性映射。灰度直方图记录每种灰度出现的次数。令 n_r 是灰度 r 的像素数，总像素数为 N，则 $p(r) = n_r / N$，累积分布函数为

$$F(r) = \sum_{t=0}^{r} p(t). \tag{3.1}$$

$F(r)$ 的含义是“灰度不超过 r 的像素占多少比例”。均衡化把这个累计比例当作新灰度：例如 $F(60) = 0.5$ 时，八位图的灰度 60 被映到约 128。一般地，输出有 Q 个灰度级时，

$$J(\boldsymbol{x}) = \operatorname{round}\big[(Q-1)F\big(I(\boldsymbol{x})\big)\big], \tag{3.2}$$

实际实现还常把最小非零累积值映射到零，以充分利用输出范围；整幅图只有一个灰度时需单独处理。

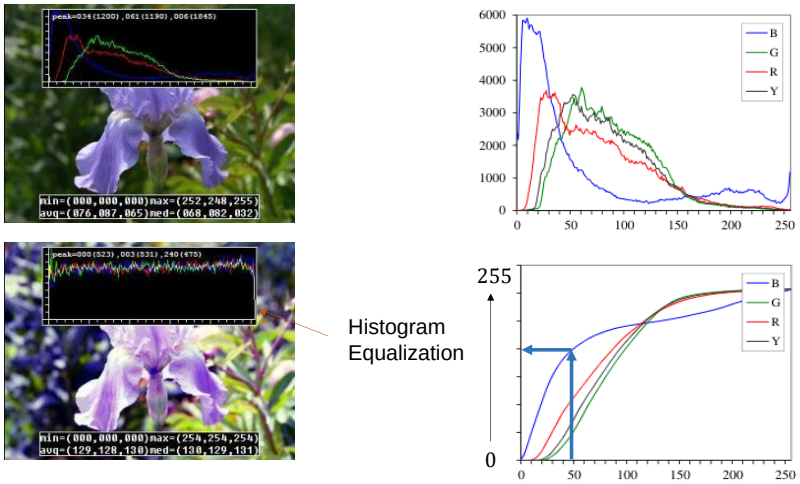


图 3.1　左上为原图，左下为均衡化结果；右侧的累积映射把密集的灰度区间展开，暗部细节和噪声都变得更明显。

数学直觉 ｜ 灰度均衡化

在像素很多的灰度区间，F 增长快，映射会把相近的原灰度拉开；像素很少的区间则被压缩。连续模型下，这相当于把灰度换成百分位数，使分布均匀。离散图像中，相同灰度的像素会映到同一个输出值，因此均衡化后的直方图通常仍有起伏。

算法实现 ｜ 灰度直方图均衡化

```
gray = cv.imread("image.png", cv.IMREAD_GRAYSCALE)
equalized = cv.equalizeHist(gray)
```

输入是单通道 `uint8` 图像。对彩色图像分别均衡化 RGB 通道会改变颜色关系；通常更适合在亮度通道处理后再转换回来。

均衡化对整幅图使用同一个灰度映射，所以暗部纹理和暗部噪声都会一起增强。若各区域的光照差别很大，可以分别调整局部对比度，并限制增强幅度。

3.2 线性滤波

图像中孤立的亮点可能是噪声，也可能是细节。观察周围的像素，有助于处理这类变化。线性滤波在每个位置取一个邻域，将其中的像素乘以各自的权重，再求和。保存这些权重的小数组称为*核*。核沿图像移动，就在各个位置重复同样的计算。

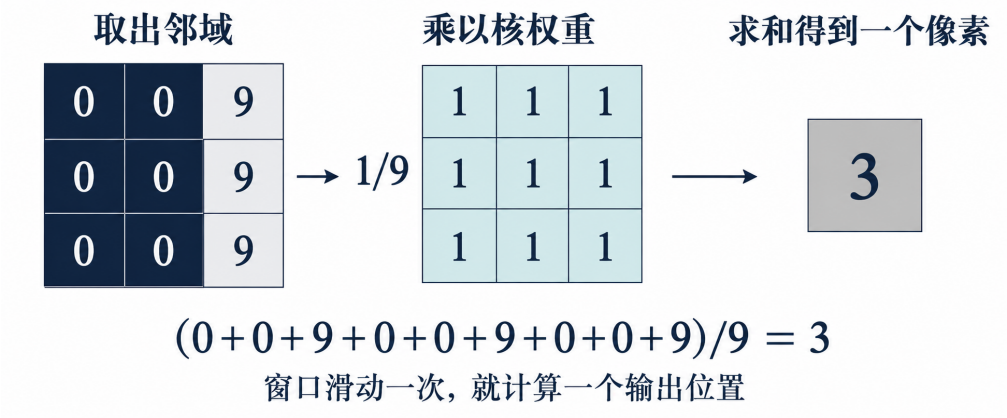


图 3.2 一次 3×3 均值运算：九个数相加得到 27，除以 9 后，中心输出为 3。此例的核中心对称，卷积与互相关结果相同。

以 (x, y) 为当前输出位置，(u, v) 为相对核中心的整数偏移，核外权重取零。按下面两种读取邻居的约定，可分别定义互相关与卷积：

$$(I \star h)(x, y) = \sum_{u,v} h(u, v)\, I(x + u, y + v), \tag{3.3}$$

$$(I * h)(x, y) = \sum_{u,v} h(u, v)\, I(x - u, y - v). \tag{3.4}$$

卷积等价于把核关于原点翻转后再做互相关。对中心对称核，两者一致；对导数核，两者可能相差一个符号。下面的公式和代码会注明采用哪一种运算。

每个输出都是输入的固定线性组合，所以固定核滤波是线性运算。例如输入一行五个像素，用核 $(1,2,1)/4$ 只计算三个完整窗口：

$$\begin{bmatrix} J_1 \\ J_2 \\ J_3 \end{bmatrix} = \underbrace{\frac{1}{4}\begin{bmatrix} 1 & 2 & 1 & 0 & 0 \\ 0 & 1 & 2 & 1 & 0 \\ 0 & 0 & 1 & 2 & 1 \end{bmatrix}}_{H} \begin{bmatrix} I_0 \\ I_1 \\ I_2 \\ I_3 \\ I_4 \end{bmatrix}.$$

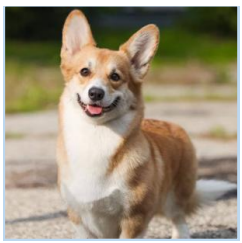
将输入记为列向量 $\boldsymbol{i}$，矩阵 H 的每一行就是把同一组权重放到一个新位置。于是 $H(a\boldsymbol{i}+b\boldsymbol{j}) = aH\boldsymbol{i} + bH\boldsymbol{j}$。在不受边界影响时，输入平移多少，输出也平移多少，即**对平移等变**。

均值核与高斯核

设计平滑核时，通常希望均匀背景仍保持原亮度。把常量图像 $I = c$ 代入求和式，输出就是 $c\sum h$；要让它仍等于 c，应取 $\sum h = 1$。相反，均匀区域没有边缘，所以导数核通常满足 $\sum h = 0$。图 3.3 把常见核的样子与处理效果放在一起；其中 $1,2,1$ 型核是离散高斯近似。均值核给邻居相同权重。若希望越近的像素贡献越大，可用随距离平滑衰减的高斯核：

$$G_\sigma(x,y) = \frac{1}{2\pi\sigma^2}\exp\left(-\frac{x^2+y^2}{2\sigma^2}\right). \tag{3.5}$$

连续高斯核前的系数使积分为 1。离散实现时截取有限窗口，再将系数归一化到和为 1，以保持常量背景的亮度。σ 以像素距离为单位，数值越大，参与平滑的邻域越宽。



原图

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

均值核：邻域等权平均

$\frac{1}{16}$

1	2	1
2	4	2
1	2	1

高斯近似核：中心权重更大

0	0	0
0	2	0
0	0	0

$-\frac{1}{9}$

1	1	1
1	1	1
1	1	1

锐化核：增强与邻域的差异

图 3.3 常用核及效果示意。正权重的平均使图像平滑；锐化核含负权重，突出局部差异。平滑强度还取决于核的范围。

可分离滤波

高斯核可写成两个一维核的乘积：

$$G_\sigma(x,y) = g_\sigma(x)g_\sigma(y), \qquad g_\sigma(x) = \frac{1}{\sqrt{2\pi}\sigma}e^{-x^2/(2\sigma^2)}.$$

离散情形可以直接看见这种分解：

$$\frac{1}{16}\begin{bmatrix}1 & 2 & 1\\ 2 & 4 & 2\\ 1 & 2 & 1\end{bmatrix} = \left(\frac{1}{4}\begin{bmatrix}1\\ 2\\ 1\end{bmatrix}\right)\left(\frac{1}{4}\begin{bmatrix}1 & 2 & 1\end{bmatrix}\right).$$

外积中的每个系数等于“横向权重 $\times$ 纵向权重”。把二维求和写成嵌套求和，内层先做横向加权，外层再做纵向加权，便得到同样的结果。$K \times K$ 核每像素约需 K^2 次乘加。当核可以分解为两个一维核的外积时，先横向、再纵向滤波，计算量就降为约 $2K$ 次。

算法实现 | 滤波与数值类型

```
I = gray.astype(np.float32) / 255.0
smooth = cv.GaussianBlur(I, (0, 0), sigmaX=1.2)

# filter2D applies correlation; flip h for convolution.
conv = cv.filter2D(I, -1, h[::-1, ::-1],
                   borderType=cv.BORDER_REFLECT_101)
```

中间计算使用浮点数；最后显示或保存时再裁剪与量化。核 h 在此假定为奇数边长、中心锚定。`filter2D` 的互相关约定见其 接口说明。

数学直觉 | 高斯滤波的频率响应

可以把图像看成不同疏密、不同方向的波纹的叠加：缓慢变化是低频，细密变化是高频。记 $\hat{I}(\boldsymbol{\omega})$ 为频率 $\boldsymbol{\omega}$ 的成分，卷积在频域中对应乘法：

$$\widehat{I * G_\sigma}(\boldsymbol{\omega}) = \hat{I}(\boldsymbol{\omega}) e^{-\sigma^2 \|\boldsymbol{\omega}\|^2/2}.$$

频率越高，乘上的因子越小；增大 σ 会让衰减更强。噪声和细纹理都可能含有高频成分，因此二者在平滑后都会减弱。

连续做两次高斯平滑时，两个频率因子相乘：$e^{-\sigma_1^2\|\boldsymbol{\omega}\|^2/2} e^{-\sigma_2^2\|\boldsymbol{\omega}\|^2/2} = e^{-(\sigma_1^2+\sigma_2^2)\|\boldsymbol{\omega}\|^2/2}$。指数中的方差相加，故

$$G_{\sigma_1} * G_{\sigma_2} = G_{\sqrt{\sigma_1^2+\sigma_2^2}}. \tag{3.6}$$

若已有模糊尺度为 σ_a 的图像，想得到尺度 $\sigma_b > \sigma_a$，只需再施加标准差 $\sigma_{\text{inc}} = \sqrt{\sigma_b^2 - \sigma_a^2}$ 的高斯核。这样可以利用已有的模糊图像，逐步生成更大尺度的结果。

锐化

平滑结果还可以用于锐化。差值 $I - G_\sigma * I$ 在平坦区域接近零，在边缘处较大。把这份差值按 $\alpha > 0$ 加回原图，就能增强局部变化。令 δ 为中心取 1、其余取 0 的单位核，$\delta * I = I$，则

$$I_{\text{sharp}} = I + \alpha(I - G_\sigma * I) = \big[(1+\alpha)\delta - \alpha G_\sigma\big] * I. \tag{3.7}$$

实现时可写 `sharp = I + 0.8 * (I - smooth)`。核的和仍为 1，所以均匀区域保持原亮度，边缘和细纹理则得到增强。增益较大时，边缘附近容易出现光晕，噪声也会更明显。

边界处理

核移到图像边缘时，部分位置会超出图像范围。程序需要为这些位置指定值，常见方法有补零、复制边缘、反射和周期延拓。补零后，平滑核会把边界附近的像素与零平均，造成边缘变暗；反射延拓则让边界外的值与附近内容相接，通常能减轻这种突变。复现滤波结果时，边界模式、核中心位置和数据类型都应一致。

3.3 非线性滤波与模板匹配

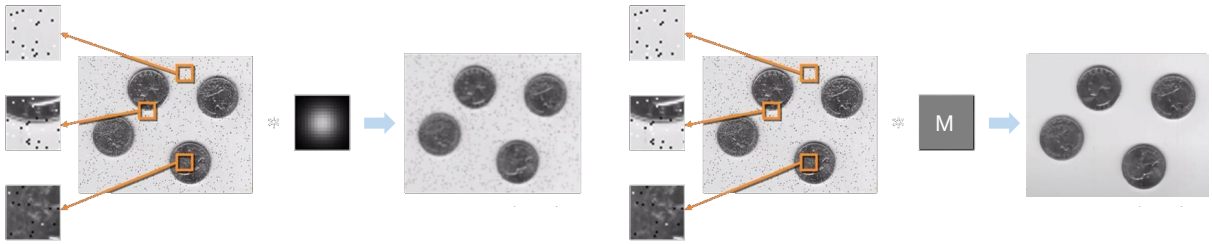
高斯滤波会对邻近像素求平均。遇到孤立的极亮噪点时，它会将亮度扩散到周围；在物体边缘处，它也会混合两侧的颜色。中值滤波适合处理前一种情况，双边滤波则有助于保留边缘。本节介绍这两种方法，随后用相关运算寻找与模板相似的图像区域。

中值滤波

先把窗口里的像素值记为 $a_1, \dots, a_n$，希望用一个数 z 代表它们。若最小化平方误差，对 z 求导会得到 $2\sum_i (z - a_i) = 0$，解就是均值；若最小化绝对误差，最优点两侧的数据数量应当平衡，解就是中位数：

$$\bar{a} = \arg\min_z \sum_i (a_i - z)^2, \qquad \operatorname{median}(a_i) \in \arg\min_z \sum_i |a_i - z|.$$

例如 $(10, 10, 10, 10, 255)$ 的均值是 59，中位数仍是 10。这说明中位数受少数极端值的影响较小，适合去除椒盐噪声。它也会去掉一些很细的真实结构，因此窗口大小需要根据图像细节选择。实现可用 `cv.medianBlur(gray, 3)`；窗口边长取奇数。



高斯滤波　　中值滤波

图 3.4　同样的椒盐噪声，高斯平均会把异常值扩散到邻居，中值滤波则用排序抑制离群值。小噪点被去掉，较大的圆形结构得以保留。

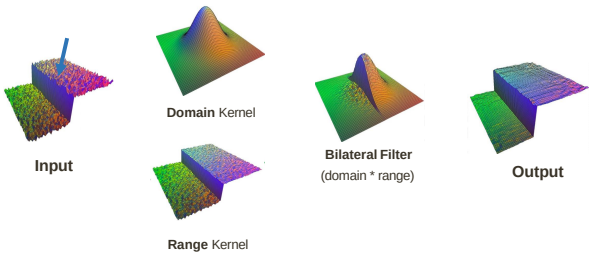
双边滤波

双边滤波同时考虑位置和颜色：邻居离中心越近、像素值越相似，权重就越大。设中心为 $\boldsymbol{p}$、邻居为 $\boldsymbol{q}$，将空间距离与像素差异各变成一个权重，再相乘：

$$w_{\boldsymbol{pq}}(I) = \exp\left(-\frac{\|\boldsymbol{p} - \boldsymbol{q}\|^2}{2\sigma_s^2} - \frac{\|I(\boldsymbol{p}) - I(\boldsymbol{q})\|^2}{2\sigma_r^2} \right), \tag{3.8}$$

$$B[I](\boldsymbol{p}) = \frac{\sum_{\boldsymbol{q} \in \mathcal{N}(\boldsymbol{p})} w_{\boldsymbol{pq}}(I)\, I(\boldsymbol{q})}{\sum_{\boldsymbol{q} \in \mathcal{N}(\boldsymbol{p})} w_{\boldsymbol{pq}}(I)}. \tag{3.9}$$

指数中的第一项随空间距离增大而衰减，第二项随颜色差异增大而衰减。分母把权重和归一到 1，使均匀区域保持原值。σ_s 使用像素距离单位，σ_r 使用灰度或颜色值的单位。当 σ_r 很大时，值域权重接近常量，趋于同一窗口内的高斯滤波；σ_r 过小时，噪声本身也会阻止像素相互平均。



边缘两侧的权重较小

原图、高斯滤波、双边滤波

图 3.5 左图解释权重如何避开跨边缘的平均；右图观察实际结果。双边滤波保留较清楚的五官边缘，同时平滑局部纹理。

双边滤波的权重随输入图像改变，因此是非线性运算。当物体边缘的颜色差异明显大于噪声幅度时，可以选择合适的 σ_r，让区域内部的像素相互平均，同时降低跨边缘的权重。

联合双边滤波

目标图像噪声较大时，可以借助另一幅已对齐的清晰图像 J 来判断边界。联合双边滤波用 J 计算权重，再用这些权重平均目标图像 I 的数值：

$$B[I;J](\boldsymbol{p}) = \frac{\sum_{\boldsymbol{q}} w_{\boldsymbol{pq}}(J) I(\boldsymbol{q})}{\sum_{\boldsymbol{q}} w_{\boldsymbol{pq}}(J)}.$$

例如，闪光照片提供清楚的边界，非闪光照片提供自然的环境色调。固定 J 后，权重不随 I 改变，所以上式对 I 是线性的。

若 I_{low} 是低分辨率深度图，$\boldsymbol{q}$ 表示它的采样点，$\boldsymbol{p}$ 表示待估计的高分辨率位置。先用 $\pi(\boldsymbol{q})$ 把低分辨率采样点换到高分辨率坐标，再计算

$$\tilde{I}(\boldsymbol{p}) = \frac{\sum_{\boldsymbol{q}} w_{\boldsymbol{p},\pi(\boldsymbol{q})}(J) I_{\text{low}}(\boldsymbol{q})}{\sum_{\boldsymbol{q}} w_{\boldsymbol{p},\pi(\boldsymbol{q})}(J)}.$$

权重中的距离统一在高分辨率坐标中计算，颜色差异则从引导图 J 中取得。这种做法适合颜色边界与深度边界相近的场景。若表面只有纹理变化而深度连续，引导图中的纹理也可能被带入深度结果。

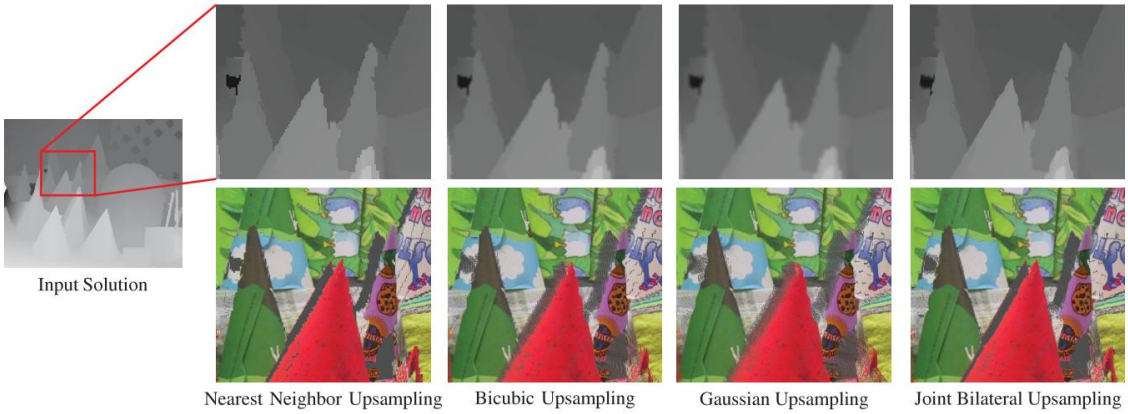


图 3.6 深度上采样的局部对比。普通插值主要依据邻近像素，联合双边上采样还参考彩色图中的边界，因此能保留更清楚的深度轮廓。

算法实现 ｜ 双边滤波

```
I = gray.astype(np.float32) / 255.0
denoised = cv.bilateralFilter(
    I, d=9, sigmaColor=0.08, sigmaSpace=3.0)
```

`sigmaColor` 使用像素值的单位。把 $[0,255]$ 改成 $[0,1]$ 时，它也应按比例缩小；`sigmaSpace` 仍以像素为单位。

模板匹配

把小图案当作核在大图中滑动，就能寻找相似区域。将模板 T 和同样大小的窗口 W 展开成向量，互相关得分就是点积 $\langle T, W\rangle$。点积会受到像素值大小的影响，整体较亮的窗口可能得到较高分数。为了减小整体增益的影响，先除以两个向量的长度，得到

$$\mathrm{NCC}(T,W)=\frac{\langle T,W\rangle}{\|T\|\,\|W\|}.$$

NCC 消除了正的整体增益。要进一步消除亮度偏移，可以减去各自均值，得到零均值归一化相关：

$$\mathrm{ZNCC}(T,W)=\frac{\sum_i (T_i-\bar T)(W_i-\bar W)}{\sqrt{\sum_i (T_i-\bar T)^2\sum_i (W_i-\bar W)^2}}. \tag{3.10}$$

若 $W=aT+b$、$a>0$，则 $W-\bar W=a(T-\bar T)$。只要模板方差非零，分子与分母都有相同的因子 $a\|T-\bar T\|^2$，约去后得分为 1。无纹理窗口中的像素几乎相同，方差接近零。实际计算时设置方差阈值，选取有足够灰度变化的窗口。

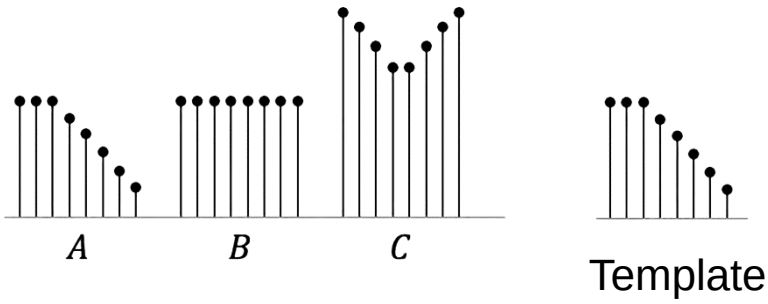


图 3.7 A 与模板形状相同，但原始点积可能让高数值的 B、C 得分更高。NCC 消除整体增益影响，ZNCC 再减去均值，从而更关注变化的形状。

在 OpenCV 中，`cv.matchTemplate(I, T, cv.TM_CCOEFF_NORMED)` 对应这种中心化后归一化的比较。响应图的极大位置给出模板左上角候选；若模板在图中发生旋转或缩放，还可以改变模板的角度和尺寸继续搜索，或使用下一章的局部特征。

3.4 采样、混叠与高斯金字塔

缩小图像会减少采样点。采样位置选得不同，结果可能相差很大。例如一串黑白交替的像素，灰度依次为 $1,0,1,0,\ldots$。每隔一个像素取样后，可能变成全白，也可能变成全黑。原来的条纹被误看成常量，这就是*混叠*的极端例子。

用原像素间距作单位，规则采样能够无歧义表示的角频率范围是 $[-\pi, \pi]$。每隔 s 个像素取样，同一波纹在新网格中的频率变成原来的 s 倍。要避免越过这个范围，降采样前应尽量滤掉 $|\omega| > \pi/s$ 的成分。这正是上一节低通滤波在缩图中的用途。

图 3.8 放大观察缩小后的图像。先滤波再抽样，能减轻条纹混叠和边缘锯齿，同时会使部分细节变平滑。

缩小图像时，先进行抗混叠滤波，再降采样。高斯滤波会逐渐衰减高频成分，是常用的近似低通方法。缩小倍数越大，需要去除的高频范围也越大，因此要相应增加滤波强度。

如果只生成一张小图，做一次“平滑再取样”即可。若要同时观察大结构和细节，就反复执行这一步，并保留各层，得到高斯金字塔。令 $I^{(0)} = I$，$\downarrow_2$ 表示每个方向隔一个像素取样，则

$$I^{(\ell+1)} = \downarrow_2 \left(G_{\sigma_0} * I^{(\ell)} \right). \tag{3.11}$$

每层宽高减半，像素总数满足

$$N + \frac{N}{4} + \frac{N}{16} + \cdots < \frac{4}{3} N.$$

金字塔总像素数约为原图的 $4/3$；各层越往上越小，因此增加层数不会成倍增加存储。

图 3.9 金字塔同时改变分辨率和观察尺度。左图每次先模糊再缩小；右图中，大脸缩小到合适的一层后，可以与固定大小的模板比较。

> **数学直觉 ｜ 多尺度搜索**
>
> 在图像金字塔的较粗层，同一个物体占据的像素更少。因此，用同一个模板依次搜索各层，就能找到原图中不同大小的物体。第 ℓ 层的一个像素约对应原图 2^ℓ 个像素宽，坐标映射回原图时也要乘以该尺度。

算法实现 | 构建高斯金字塔

```
G = [I.astype(np.float32)]
for _ in range(levels):
    G.append(cv.pyrDown(G[-1]))
```

每层先滤波再缩小。选择层数时，最小的一层应保留足够像素，供后续的邻域计算或模板匹配使用。

尺度空间可以在同一网格上改变 σ，金字塔还改变了像素间距。第 j 层的一个像素相当于原图的 2^j 个像素，因此在该层额外模糊 σ_0，相当于在原图单位下模糊 $2^j\sigma_0$。将这些新增方差相加，忽略离散与重采样误差，有

$$\sigma_{\mathrm{eff},\ell}^2 = \sigma_0^2 \sum_{j=0}^{\ell-1} 4^j = \sigma_0^2 \frac{4^\ell - 1}{3}.$$

同一个“像素单位”的 σ 在不同层代表不同的物理尺度。

3.5 图像插值

放大或旋转图像时，目标像素往往对应源图中两个像素之间的位置。我们可以利用附近已知的像素值，估计这个位置的数值，这就是插值。一维网格间隔为 1 时，每个样本 $I[n]$ 对位置 x 的贡献由权重 $\varphi(x-n)$ 决定：

$$\hat{I}(x) = \sum_{n\in\mathbb{Z}} I[n]\,\varphi(x-n). \tag{3.12}$$

若希望严格经过原采样点，需要 $\varphi(0)=1$ 且对非零整数 n 有 $\varphi(n)=0$。高斯加权平均在整数位置也会混合邻居的值，通常用于平滑。

最近邻插值使用离目标位置最近的采样值，适合离散标签，但会产生块状边缘。线性插值使用三角核 $\varphi(t)=\max(1-|t|,0)$，连续但一阶导数通常不连续。二维双线性插值可以分成两次一维插值。设 $(x,y)=(i+\alpha, j+\beta)$，$0\le\alpha,\beta\le 1$。先在上下两行沿水平方向插值，再把两行结果沿竖直方向混合：

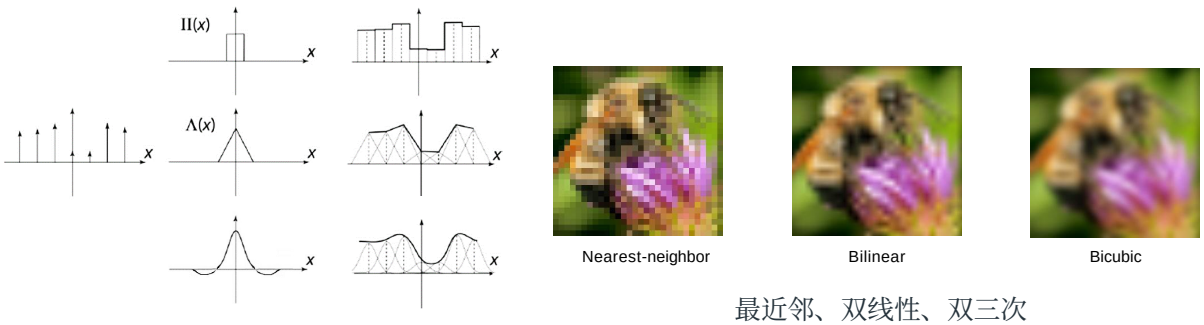
$$\begin{aligned} A &= (1-\alpha)I[i,j] + \alpha I[i+1,j], \\ B &= (1-\alpha)I[i,j+1] + \alpha I[i+1,j+1], \\ \hat{I}(x,y) &= (1-\beta)A + \beta B. \end{aligned} \tag{3.13}$$

例如四个像素依次为 $0, 10, 20, 30$，取 $\alpha=0.25, \beta=0.5$，先得 $A=2.5, B=22.5$，再得 $\hat{I}=12.5$。这里数学下标先横后纵，Python 数组应写成 `I[j, i]`。“双线性”表示固定一个坐标后，对另一个坐标是线性的；整体展开式中还含有 xy 项。

双三次插值在两个方向各取四个邻居，用 4×4 个像素共同估计新值：

$$\hat{I}(x,y) = \sum_{p=i-1}^{i+2}\sum_{q=j-1}^{j+2} I[p,q]\,\varphi_a(x-p)\varphi_a(y-q), \tag{3.14}$$

其中 $(i,j)=(\lfloor x\rfloor, \lfloor y\rfloor)$。$\varphi_a$ 是支撑在 $[-2,2]$ 的分段三次核，参数 a 调节其形状，常见选择为 -0.5。从图中可见，负瓣能增强锐利感，但也可能使边缘过冲。



插值核与重建曲线　　最近邻、双线性、双三次

图 3.10　左图由上至下是最近邻、线性、三次插值；右图是相应的放大效果。矩形核产生台阶，三角核产生连续折线，三次核产生更平滑的曲线。

像素对齐

把长度 N 的输入变成长度 M 的输出，输出整数位置 j 应映射到哪里？若让首尾像素中心一一对应，就要把 $0 \mapsto 0$、$M-1 \mapsto N-1$，得到第一式。若把像素视为方格，则输出中心的相对位置是 $(j+\tfrac{1}{2})/M$；乘以 N 后，再减去输入中心的半像素偏移，得到第二式：

$$x_{\text{in}} = j\,\frac{N-1}{M-1}, \qquad \text{首尾像素中心对齐,} \tag{3.15}$$

$$x_{\text{in}} = (j+\tfrac{1}{2})\frac{N}{M} - \tfrac{1}{2}, \qquad \text{像素边界对齐.} \tag{3.16}$$

这里 $N, M > 1$，并显式指定输出长度。例如 $N=2, M=4$ 时，输出位置 $j=0$ 在两种约定下分别映到 0 和 -0.25，后者需配合边界取值规则。多次变换出现半像素偏移，常常是这些约定不一致造成的。

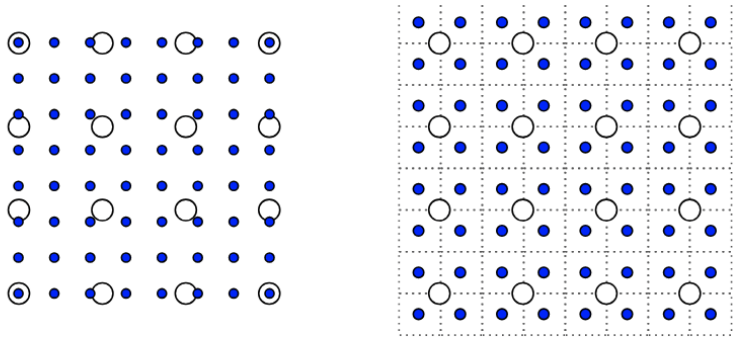


图 3.11　空心圆为输入像素中心，实心圆为输出像素中心。左图对齐首尾中心，右图对齐像素边界；两种约定会给出不同的取样位置。

> **备注 ｜ 放大与超分辨率**
> 插值按给定的核，利用附近像素计算中间值。学习型超分辨率还会利用训练图像中的纹理和形状，补充可能的细节。这些新增细节是模型的估计，评价结果时可以与真实高分辨率图像比较。

算法实现 | 双线性缩放

```
import torch.nn.functional as F

# x has shape (batch, channels, height, width).
y = F.interpolate(x, size=(new_h, new_w),
                  mode="bilinear", align_corners=False,
                  antialias=True)
```

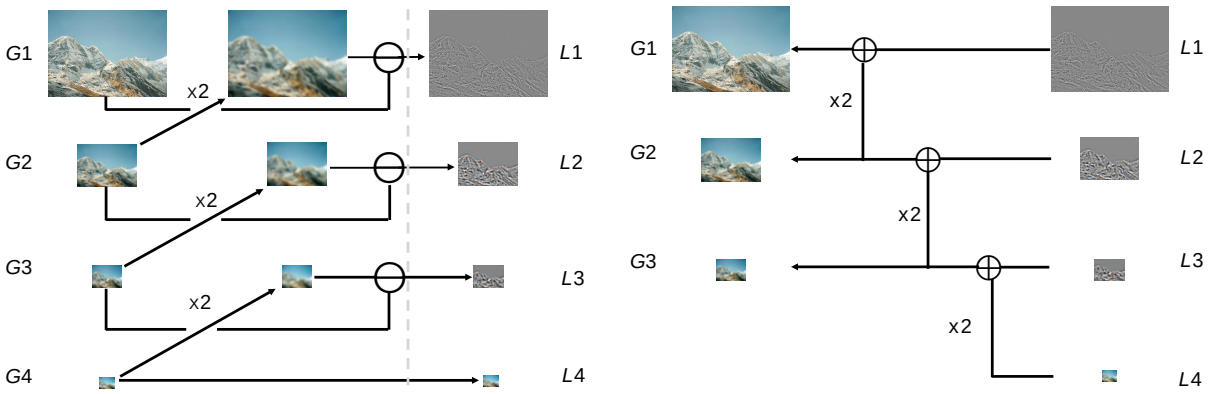
此处显式指定输出大小；缩小时启用抗混叠。参数的模式适用范围见 PyTorch 插值接口。

3.6 拉普拉斯金字塔与多频带融合

高斯金字塔的较粗层保留了大范围的亮度变化。为了逐层恢复细节，可以把细层与粗层上采样结果之间的差值另存下来。令 E_ℓ 把第 $\ell+1$ 层上采样到第 ℓ 层的尺寸，它给出了对细层的一个预测。细层减去预测值，便得到残差：

$$L^{(\ell)} = I^{(\ell)} - E_\ell(I^{(\ell+1)}), \quad \ell = 0, \dots, m-1; \qquad L^{(m)} = I^{(m)}. \tag{3.17}$$

这些残差加上最粗层图像，就构成拉普拉斯金字塔。靠近原图的层主要保存细节，较粗的层则保存更大范围的明暗变化。每层集中在一定的空间频率范围内，因此也可近似看作带通滤波的结果。



分解：保存粗层预测误差　　重建：逐层加回残差

图 3.12 拉普拉斯金字塔的分解与重建。图中从 1 开始编号，正文从 0 开始。

重建从最粗层开始：

$$\hat{I}^{(m)} = L^{(m)}, \qquad \hat{I}^{(\ell)} = L^{(\ell)} + E_\ell(\hat{I}^{(\ell+1)}). \tag{3.18}$$

最粗层已经完整保存。假设第 $\ell+1$ 层已正确重建，代入残差定义就有

$$\hat{I}^{(\ell)} = \underbrace{I^{(\ell)} - E_\ell(I^{(\ell+1)})}_{L^{(\ell)}} + E_\ell(I^{(\ell+1)}) = I^{(\ell)}.$$

从粗到细重复这一步，就能恢复原图。**分解和重建使用相同的上采样运算**，这样，上采样产生的差值就会在相加时抵消。

数学直觉 | 残差重建

例如，某个细层像素为 100，粗层上采样后得到 94，就保存残差 6。重建时重新计算上采样值 94，再加上 6，得到原来的 100。若为了压缩而将残差近似为 5，重建结果就会相差 1。

算法实现 | 金字塔分解与重建

```
def expand(coarse, shape):
    h, w = shape[:2]
    return cv.pyrUp(coarse, dstsize=(w, h))

L = [g - expand(h, g.shape) for g, h in zip(G[:-1], G[1:])]
L.append(G[-1])

recon = L[-1]
for detail in reversed(L[:-1]):
    recon = detail + expand(recon, detail.shape)
```

`G` 是浮点高斯金字塔，残差也用浮点数保存，以保留负值和小数部分。显示残差时，可以另取一份副本作平移或缩放。

拉普拉斯金字塔与对应高斯金字塔的像素数量相同。残差的数值通常集中在零附近，这有利于后续压缩；实际文件大小取决于精度和编码方法。

多频带融合

这种分层表示还能让图像融合更自然。设 A, B 已对齐，掩码 $M = 1$ 处取 A，$M = 0$ 处取 B，中间值表示混合。直接用 $MA + (1 - M)B$，窄过渡容易留下亮度接缝，宽过渡又可能把纹理混成重影。

解决办法是让不同频带使用不同的过渡宽度：为 A, B 构建拉普拉斯金字塔，为 M 构建高斯金字塔。每一层的图像与掩码尺寸相同，再逐位置混合：

$$L_C^{(\ell)} = M^{(\ell)} L_A^{(\ell)} + (1 - M^{(\ell)}) L_B^{(\ell)}. \tag{3.19}$$

由 L_C 重建最终图像。粗尺度的掩码在原图坐标中覆盖更宽的范围，使低频亮度差异缓慢过渡；细尺度的掩码变化更集中，使纹理和边缘少受模糊影响。

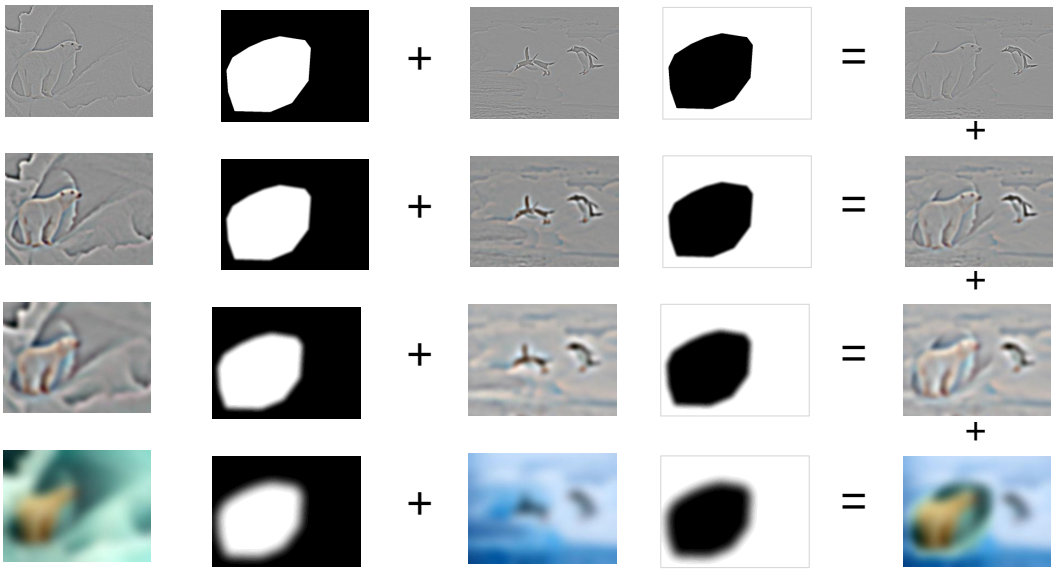


图 3.13 每一行混合一个频带，再逐层上采样重建。掩码从上到下越来越模糊：高频细节只在窄范围过渡，低频明暗差异则在宽范围过渡。

图 3.14 先对齐两幅图，再用多尺度融合减轻亮度接缝，并保留局部细节。

3.7 几何变换与图像形变

融合之前，需要把两幅图中的同一处物体对齐。几何变换规定像素应移动到哪里，插值则负责计算新位置上的数值。常用几何变换如下，其中 R 现在表示二维旋转矩阵。从刚性到单应，允许的形变逐渐增加，但都保持共线关系。

变换	形式	主要保留的几何性质
刚性	$\boldsymbol{x}' = R\boldsymbol{x} + \boldsymbol{t}$	长度、角度与形状；3 个自由度。
相似	$\boldsymbol{x}' = sR\boldsymbol{x} + \boldsymbol{t}$	角度与长度比例；4 个自由度。
仿射	$\boldsymbol{x}' = A\boldsymbol{x} + \boldsymbol{t}$	平行性、共线性与共线长度比；6 个自由度。
单应	$\tilde{\boldsymbol{x}}' \sim H\tilde{\boldsymbol{x}}$	共线性；非奇异 3×3 矩阵去掉整体尺度后有 8 个自由度。

平面场景的两个视图可以用一个单应联系，相机绕同一光心旋转时也是如此。相机平移并且场景有明显深度差时，各处的位移会随深度变化，通常需要更详细的几何模型。第五章会继续讨论单应的适用条件。

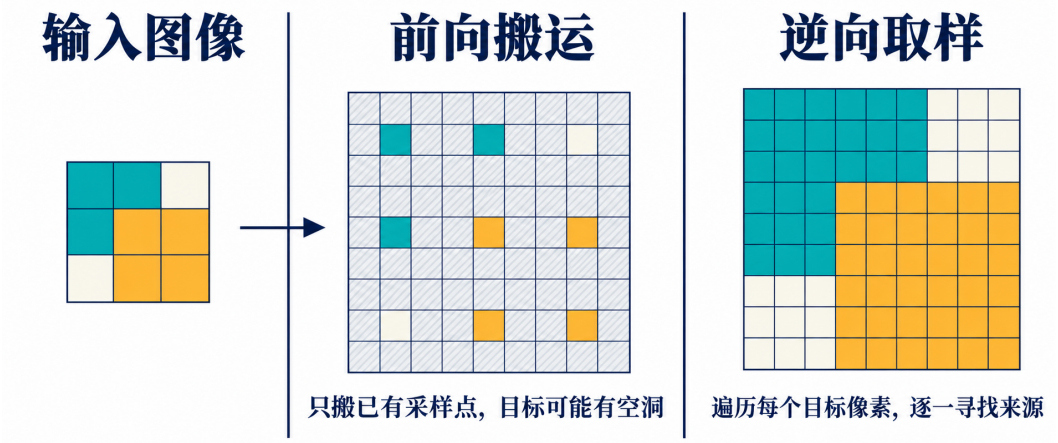


图 3.15 放大示意。前向搬运只把已有采样点推到目标位置，可能留下空洞；逆向取样逐个填满目标网格。右图用最近邻展示覆盖关系，也可换成双线性等插值。

前向映射与逆向采样

前向映射从源像素出发，计算它在目标图中的位置。逆向采样则遍历目标图的每个像素，计算它在源图中的取样位置。已知源到目标的映射 T，逆向采样写成

$$J(\boldsymbol{x}') = \hat{I}\big(T^{-1}(\boldsymbol{x}')\big), \tag{3.20}$$

这里假设 T 在所需区域可逆，$\hat{I}$ 是前一节构造的插值函数。例如坐标变换为 $u' = 2u$ 时，目标像素 $u' = 3$ 来自源位置 $u = 1.5$，需要在邻居之间插值。逐个遍历目标像素可避免漏填；映到源图之外的位置仍需设置边界值。

算法实现 ｜ 按单应变换重采样

```
# Hmat maps source coordinates to destination coordinates.
warped = cv.warpPerspective(
    image, Hmat, dsize=(out_w, out_h),
    flags=cv.INTER_LINEAR, borderMode=cv.BORDER_CONSTANT)
```

默认参数下，传入源到目标的矩阵，函数会在内部完成逆向采样。若某些区域经过透视变换后显著缩小，可在取样前加入适当的抗混叠滤波。

图像渐变（Morphing）

如果要把一张人脸逐渐变成另一张，仅平均像素会让两组五官同时出现。应先让它们移动到共同位置，再混合颜色。设两图为 A, B，对应特征点为 $\boldsymbol{p}_i^A, \boldsymbol{p}_i^B$，在变化进度 $0 \le t \le 1$ 时，中间位置取为

$$\boldsymbol{p}_i(t) = (1-t)\boldsymbol{p}_i^A + t\boldsymbol{p}_i^B.$$

用相同的顶点连接方式建立三角网格，就把整张脸拆成许多局部仿射变换。

接下来仍用逆向采样：对中间三角形内的目标像素 $\boldsymbol{x}$，先求它相对于三个顶点的权重

$$\boldsymbol{x} = \sum_{j=1}^{3} \lambda_j \boldsymbol{p}_j(t), \qquad \sum_{j=1}^{3} \lambda_j = 1, \quad \lambda_j \ge 0.$$

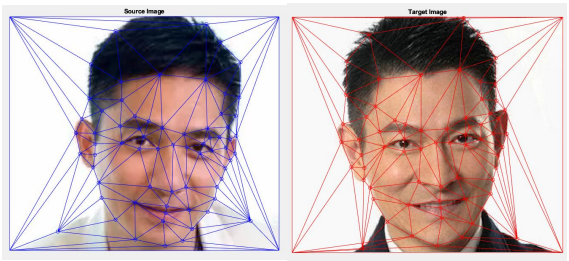


图 3.16 对应点确定三角网格。让同一对眼睛、鼻子先移动到共同位置，再插值颜色，可以减少直接平均造成的重影。

这组重心坐标描述点在三角形内部的相对位置。三个权重都是 $1/3$ 时位于重心；取 $(1/2, 1/4, 1/4)$ 时则更靠近第一个顶点。在两幅源图中保持同样的权重，就能找到对应位置，再分别插值取色：

$$\begin{aligned} \boldsymbol{x}_A &= \sum_{j=1}^{3} \lambda_j \boldsymbol{p}_j^A, \qquad \boldsymbol{x}_B = \sum_{j=1}^{3} \lambda_j \boldsymbol{p}_j^B, \\ J_t(\boldsymbol{x}) &= (1-t)\hat{A}(\boldsymbol{x}_A) + t\hat{B}(\boldsymbol{x}_B). \end{aligned} \tag{3.21}$$

每个三角形内使用仿射变换。为了让渐变平滑，对应点应准确，变形过程中的三角形也应保持正常朝向；发生遮挡变化的区域可另设掩码处理。

第 4 章　边缘、角点与局部特征

两张照片中的同一处物体，位置、大小和朝向都可能不同。本章介绍边缘、角点和局部特征，讨论怎样找到这些位置，以及怎样比较它们附近的图像内容。

假设拍了同一座建筑的两张照片，希望把它们拼成全景。可以先找窗角、屋檐等容易辨认的位置，再比较它们附近的纹理，寻找同一处物体。为了适应照片之间的缩放和旋转，还需要确定局部区域的大小和方向。

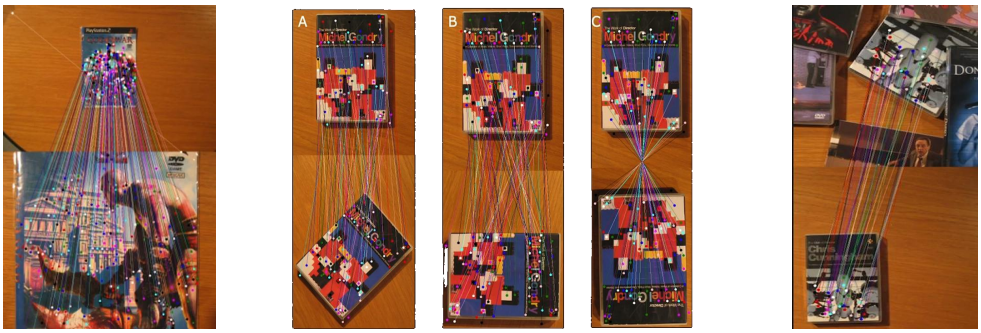


图 4.1　同一物体在大小、朝向和背景变化后，局部纹理仍能帮助我们找到对应位置。

边缘检测先找出明显的亮度变化，角点检测进一步寻找容易在两个方向上定位的点。讲过这两种方法后，我们再介绍尺度选择，以及 SIFT 如何检测和描述局部区域。实际使用 SIFT 时，从 DoG 检测开始完成它自己的流程。本章最后介绍 HOG，用一个窗口中的梯度分布来描述物体形状。

4.1　梯度与 Canny 边缘检测

物体轮廓、窗框和纹理通常伴随明显的亮度变化，边缘检测就是寻找这些变化。阴影和表面颜色的分界也会产生边缘，如下图所示。

图像梯度

先把图像看成一个可微的二维函数。横向导数 I_x 衡量左右变化，纵向导数 I_y 衡量上下变化，将两者放在一起得到梯度：

$$\nabla I = \begin{bmatrix} I_x \\ I_y \end{bmatrix}, \qquad m = \sqrt{I_x^2 + I_y^2}, \qquad \theta = \operatorname{atan2}(I_y, I_x). \tag{4.1}$$

为什么它指向变化最快的方向？沿单位方向 $\boldsymbol{n}$ 移动一个小距离 ε，一阶展开给出

$$I(\boldsymbol{x} + \varepsilon \boldsymbol{n}) - I(\boldsymbol{x}) \approx \varepsilon \, \nabla I(\boldsymbol{x})^{\mathsf{T}} \boldsymbol{n}.$$

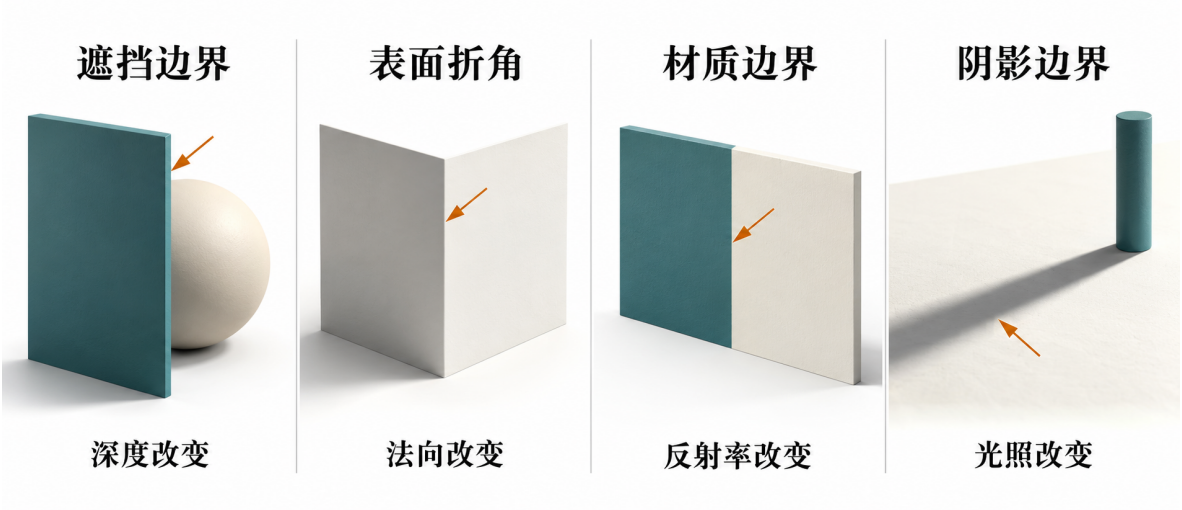


图 4.2 遮挡、表面折角、材质变化和阴影都可以在图像中形成边缘。

点积等于 $\|\nabla I\|\cos\angle(\nabla I, \boldsymbol{n})$，同向时增长最多。沿等亮度曲线移动则变化为零，所以**梯度沿边缘法向，边缘走向与它垂直**。例如一条竖直的明暗分界线，亮度主要左右变化，梯度就主要指向水平方向。

像素只在网格上给出数值，需要用差分近似导数。单位间距下，

$$I_x(x,y) \approx \frac{I(x+1,y) - I(x-1,y)}{2}.$$

分母 2 是两个采样点的间距。按互相关、图像纵轴向下的约定，常用 Sobel 核为

$$S_x = \frac{1}{8}\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \qquad S_y = \frac{1}{8}\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}.$$

把 S_x 看成 $(1,2,1)^{\mathsf{T}}(-1,0,1)/8$，就能看出它在横向做差分，同时沿纵向做平滑。$S_y$ 的作用相反；这与上一章核的可分离性完全一致。

平滑与求导

噪声会改变相邻像素的差值，求导时也会产生较强的响应。为了减轻它的影响，先计算平滑图像 $L = G_\sigma * I$，再求梯度。将求导作用到卷积核上，有

$$\partial_x L = (\partial_x G_\sigma) * I, \qquad \partial_y L = (\partial_y G_\sigma) * I. \tag{4.2}$$

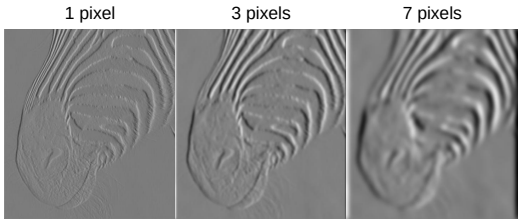
这把“平滑后求导”合成了一次滤波。σ 小时保留更多细边缘，也更容易留下噪声；σ 大时，细纹理和噪声会进一步减弱，相邻轮廓也可能合并。因此，σ 要根据希望保留的细节大小选择。

Canny 检测步骤

平滑后的阶跃变化会产生一条较宽的梯度带。如果直接把幅值较大的位置都当作边缘，轮廓会很粗；如果只用一个高阈值，又容易把较暗的轮廓截断。Canny 分别处理这两个问题：

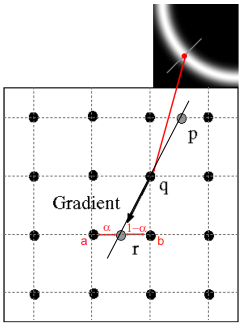
$$\text{平滑与求梯度} \longrightarrow \text{非极大值抑制} \longrightarrow \text{双阈值与连通追踪}.$$

I_x、I_y 与梯度幅值



更大的平滑尺度

图 4.3 左：不同方向的导数强调不同的边，幅值将它们合并。右：尺度增大后，细纹理与噪声减弱，但相邻轮廓也可能合并。



沿梯度方向比较

梯度幅值

非极大值抑制

双阈值筛选

图 4.4 先沿梯度方向取局部峰值，让粗边缘变细；再用阈值区分强、弱响应。下一步还需把与强边缘连通的弱边缘接回来。

对 $m(\boldsymbol{x}) > 0$ 的位置，令 $\boldsymbol{n} = \nabla L / \|\nabla L\|$。非极大值抑制比较 $m(\boldsymbol{x})$ 与 $m(\boldsymbol{x} \pm \boldsymbol{n})$，只留下法向上的局部峰值。非整数位置用插值取值。这个比较方向横穿边缘带，因此留下的是边缘带中间的峰值。

接着，在细化后的幅值图上设置 $T_{\text{low}} < T_{\text{high}}$。超过高阈值的点标为强边缘，低于低阈值的点丢弃，其余标为候选弱边缘。从强边缘出发，沿八邻域追踪候选点，只保留能通过一条候选路径连接到强边缘的弱点。沿候选路径继续追踪，就能保留离强边缘较远、但通过其他弱点与它相连的像素。

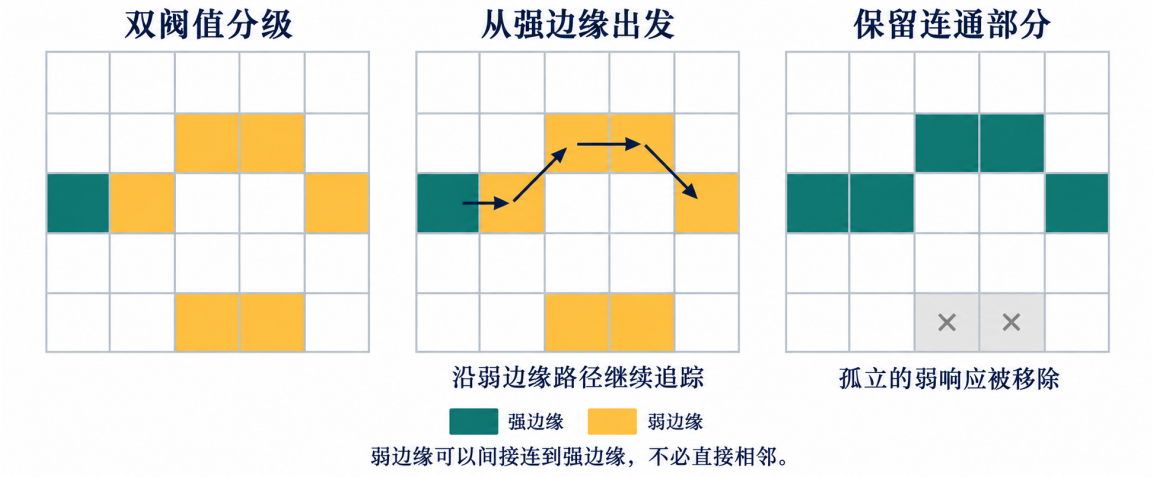


图 4.5 从强边缘出发，沿候选像素的八邻域持续追踪。远处的弱点也能通过路径保留；孤立弱点被删除。右图用绿色标出了最终保留的像素。

边缘可以帮助我们找到窗框的位置。若沿着一条长直窗框移动小窗口，看到的内容却可能几乎相同。窗角附近的情况不同：向各个方向移动，窗口中的两条边都会改变位置。角点检测利用的就是这种差别。

图 4.6 Canny 将较宽的梯度响应细化为轮廓，并通过连通追踪保留部分较弱的边缘。

算法实现 | Canny 边缘检测

```
blurred = cv.GaussianBlur(gray, (0, 0), 1.2)
edges = cv.Canny(blurred, 50, 120, L2gradient=True)
```

`gray` 为 `uint8` 灰度图，$50, 120$ 只是示例阈值。阈值与输入范围、梯度核的缩放有关；更换数值约定后需要重新选择。

4.2 Harris 角点检测

在两张照片中寻找同一个窗角时，我们希望它的位置容易确定。如果窗口稍微移动，外观就明显改变，就能较准确地判断原位置。Harris 计算窗口平移前后的差异，并据此选择角点。

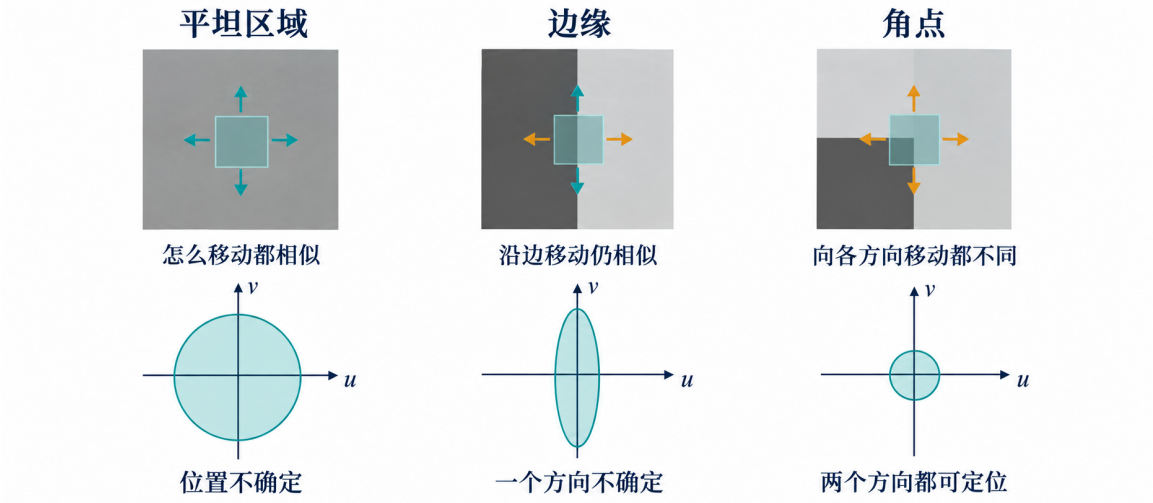


图 4.7 上排比较窗口移动前后的外观，下排标出外观变化较小的移动范围。平坦区域允许较大的移动；直边附近可以沿边移动；角点附近向各个方向移动都会产生明显差别。

窗口位移误差

固定一个候选窗口，用 $w(\boldsymbol{x}) \geq 0$ 给其中像素加权；通常中心权重大，边缘权重小。令小位移为 $\boldsymbol{\delta} = (u, v)^{\mathsf{T}}$，定义平方差

$$E(\boldsymbol{\delta}) = \sum_{\boldsymbol{x}} w(\boldsymbol{x}) \big[I(\boldsymbol{x} + \boldsymbol{\delta}) - I(\boldsymbol{x}) \big]^2. \tag{4.3}$$

原位置对应 $\boldsymbol{\delta} = 0$，此时平方差为零。接下来考察小位移产生的误差，看看窗口沿哪些方向移动时变化较大。

当位移足够小时，在每个像素处作一阶展开：$I(x+u,y+v)-I(x,y)\approx I_x u+I_y v$。代入并展开平方，

$$\begin{aligned}E(u,v)&\approx\sum_{\boldsymbol{x}} w(\boldsymbol{x})(I_x u+I_y v)^2\\&=u^2\sum_{\boldsymbol{x}} wI_x^2+2uv\sum_{\boldsymbol{x}} wI_xI_y+v^2\sum_{\boldsymbol{x}} wI_y^2.\end{aligned}$$

令这三个加权和分别为 A,B,C，就得到

$$E(\boldsymbol{\delta})\approx\boldsymbol{\delta}^{\mathsf{T}}M\boldsymbol{\delta},\qquad M=\begin{bmatrix}A & B\\ B & C\end{bmatrix}=\sum_{\boldsymbol{x}} w(\boldsymbol{x})\nabla I(\boldsymbol{x})\nabla I(\boldsymbol{x})^{\mathsf{T}}.\tag{4.4}$$

M 称为结构张量，也称二阶矩矩阵。它的三个独立元素分别累加了横向梯度平方、横纵梯度乘积和纵向梯度平方。由于 $\boldsymbol{\delta}^{\mathsf{T}}M\boldsymbol{\delta}$ 是一组平方项的非负加权和，M 必为半正定。

特征值与角点判据

对称矩阵 M 可以找到两个互相垂直的特征方向。把位移在这两个方向上的分量记为 ξ_1,ξ_2，若 $M=Q\,\mathrm{diag}(\lambda_1,\lambda_2)Q^{\mathsf{T}}$，则 $\boldsymbol{\xi}=Q^{\mathsf{T}}\boldsymbol{\delta}$，从而

$$E\approx\boldsymbol{\xi}^{\mathsf{T}}\,\mathrm{diag}(\lambda_1,\lambda_2)\boldsymbol{\xi}=\lambda_1\xi_1^2+\lambda_2\xi_2^2,\qquad\lambda_1\geq\lambda_2\geq 0.$$

每个 λ_i 就是沿相应方向的误差增长强度。固定容许误差 $E=c>0$，当两特征值为正时，等值线是半轴长 $\sqrt{c/\lambda_1}$、$\sqrt{c/\lambda_2}$ 的椭圆。两个值都小，位置在各方向都难确定；一大一小，只有一个方向定位可靠；两个都大，才是角点所需要的局部结构。

> **数学直觉 ｜ 位移误差椭圆**
>
> 若 $M=\mathrm{diag}(9,1)$，则 $E=9u^2+v^2$。在 $E\leq 1$ 的容许范围内，横向最多移动 $1/3$，纵向却可以移动 1。纵向允许更大的移动，对应图 4.7 中的长轴。要让两个方向都容易定位，需要较小的特征值也足够大。

对这个二阶矩矩阵，$\det M=AC-B^2=\lambda_1\lambda_2$，$\mathrm{tr}\,M=A+C=\lambda_1+\lambda_2$，只需三个矩阵元素就能求出。Harris 用这两个量构造分数：

$$R_H=\det M-\kappa(\mathrm{tr}\,M)^2=\lambda_1\lambda_2-\kappa(\lambda_1+\lambda_2)^2,\qquad\kappa\approx 0.04\text{–}0.06.\tag{4.5}$$

平坦区域两特征值都小，分数接近零；边缘上 $\lambda_2\approx 0$，分数约为 $-\kappa\lambda_1^2<0$；若两特征值都接近较大的 λ，则分数为 $(1-4\kappa)\lambda^2>0$。因此可用较大的正响应筛选角点，再用空间非极大值抑制保留局部代表。直接阈值化 $\lambda_{\min}(M)$ 也是一种常见选择，对应 Shi–Tomasi 的思想。

梯度窗口

计算 M 时会用到整个窗口的梯度。先以 σ_d 为标准差平滑图像并求导，再用标准差为 σ_i 的高斯窗口平均梯度外积：

$$L=G_{\sigma_d}*I,\qquad M=G_{\sigma_i}*\begin{bmatrix}L_x^2 & L_xL_y\\ L_xL_y & L_y^2\end{bmatrix}.\tag{4.6}$$

算法实现 | 直接计算 Harris 响应

```
I = gray.astype(np.float32) / 255.0
blur = lambda a, s: cv.GaussianBlur(a, (0, 0), s)
J = blur(I, 1.0)
Ix = cv.Sobel(J, cv.CV_32F, 1, 0, ksize=3, scale=1/8)
Iy = cv.Sobel(J, cv.CV_32F, 0, 1, ksize=3, scale=1/8)
A, B, C = [blur(a, 1.5) for a in (Ix*Ix, Ix*Iy, Iy*Iy)]
R = A*C - B*B - 0.04*(A + C)**2
peak = cv.dilate(R, np.ones((3, 3), np.uint8))
keep = (R == peak) & (R > 0.01 * max(float(R.max()), 0))
v, u = np.where(keep)
```

若出现等高平台，可再合并连通的极大值；精确定位还可做亚像素优化。

σ_d 控制求导前的平滑程度，σ_i 控制统计梯度的窗口大小。窗口过小会受噪声影响，过大又会把多个结构混在一起。

理想连续模型下，旋转只改变这些特征方向，不改变特征值，角点位置会随图像一起旋转。若亮度变为 $I' = aI + b$，梯度变为原来的 a 倍；M 含两个梯度相乘，所以变为 a^2M，Harris 分数进一步变为 a^4R_H。因此，若用绝对阈值筛选角点，阈值也应随对比度调整。

上面的计算预先选定了窗口大小。物体在照片中变大后，原窗口覆盖的范围就变小，可能只包含一条边或一块平坦区域。因此，在不同尺寸的照片中寻找同一结构时，还需要同时选择窗口的尺度。

4.3 尺度空间与 LoG

在多个尺度上运行检测器，可以寻找适合各个物体的观察范围。这里用斑点来说明具体方法：斑点是一块比周围更亮或更暗的区域，我们希望同时找到它的中心，并估计它的大小。

LoG 滤波

一阶导数在斑点的两侧变化最大，在对称中心反而可能为零。二阶导数描述斜率怎样改变：亮斑中心像一个向下弯的峰，通常具有负曲率；暗斑中心则相反。为减弱噪声，先构造一族平滑图像

$$L(\boldsymbol{x};\sigma) = G_\sigma * I, \qquad \sigma > 0.$$

位置 $\boldsymbol{x}$ 与模糊尺度 σ 一起构成*尺度空间*。二维情况下把两个方向的曲率相加，得到拉普拉斯算子 $\nabla^2 L = L_{xx} + L_{yy}$，它不偏好某一个边缘方向。

计算时可把求导移到高斯核上：$\nabla^2 L = (\nabla^2 G_\sigma) * I$。由 $\partial_x G_\sigma = -xG_\sigma/\sigma^2$ 再求一次导数，有

$$\partial_{xx} G_\sigma = \left(\frac{x^2}{\sigma^4} - \frac{1}{\sigma^2}\right) G_\sigma.$$

把 x, y 两个方向的结果相加，得到 LoG（Laplacian of Gaussian）核：

$$\nabla^2 G_\sigma(x,y) = \frac{x^2 + y^2 - 2\sigma^2}{\sigma^4} G_\sigma(x,y).$$

它的中心与外围符号相反；核在半径 $\sqrt{2}\sigma$ 的圆上过零。图 4.8 中的虚线圆正是在表示这个特征半径。

尺度归一化

把同一结构放大后，它的亮度变化分布在更宽的范围内，导数的幅值会相应减小。为了比较不同大小的结构，先计算这个幅值如何随缩放变化。设图像放大 $a>0$ 倍：$I_a(\boldsymbol{x}')=I(\boldsymbol{x}'/a)$。同时把模糊半径放大 a 倍，对应位置的平滑值不变，

$$L_a(\boldsymbol{x}';a\sigma)=L(\boldsymbol{x}'/a;\sigma).$$

令 $\boldsymbol{x}'=a\boldsymbol{x}$，链式法则给出 $\partial/\partial x'=a^{-1}\partial/\partial x$，二阶导数因此多出 a^{-2}。只要再乘以相应尺度的平方，两个因子就抵消：

$$(a\sigma)^2\nabla^2_{\boldsymbol{x}'}L_a(a\boldsymbol{x};a\sigma)=\sigma^2\nabla^2_{\boldsymbol{x}}L(\boldsymbol{x};\sigma). \tag{4.7}$$

所以使用**尺度归一化**的响应

$$\mathcal{L}(\boldsymbol{x};\sigma)=\sigma^2\nabla^2L(\boldsymbol{x};\sigma)=(\sigma^2\nabla^2G_\sigma)*I. \tag{4.8}$$

亮斑和暗斑产生相反符号的响应，因此同时寻找极小值和极大值。每个候选要和空间位置、尺度附近的响应比较，留下的极值便给出斑点的位置和检测尺度。

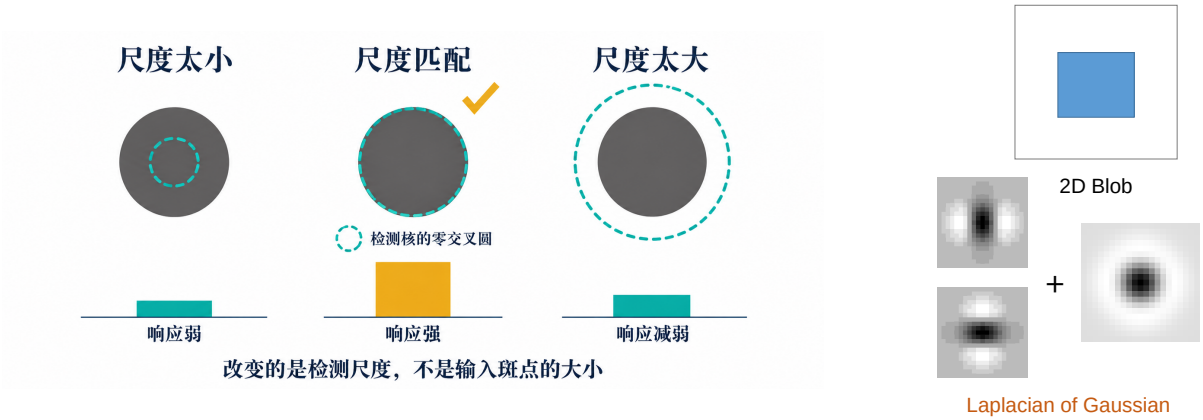


图 4.8 左：同一斑点，用不同尺度的核检测，匹配尺度附近的响应更强；柱形只示意强弱趋势。右：LoG 是两个方向二阶导数的和。

> **数学直觉 ｜ 缩放后的检测结果**
> 原图在 $(\boldsymbol{x},\sigma)$ 处的稳定极值，放大后应移到 $(a\boldsymbol{x},a\sigma)$，归一化响应保持相同。这体现了位置和尺度的等变性。把检测到的邻域缩放到统一大小以后，就可以比较两幅图中大小不同的同一个结构。

圆盘的检测尺度

考虑半径为 r、内部灰度为 1、外部为 0 的理想亮圆盘。在圆心处，卷积就是把核在圆盘内积分。用 ρ 表示积分半径，圆环面积为 $2\pi\rho\,\mathrm{d}\rho$。计算中令 $q=\rho^2/(2\sigma^2)$，则 $\mathrm{d}q=(\rho/\sigma^2)\mathrm{d}\rho$，于是

$$\begin{aligned}\mathcal{L}(0;\sigma)&=\sigma^2\int_0^r\frac{\rho^2-2\sigma^2}{\sigma^4}\frac{e^{-\rho^2/(2\sigma^2)}}{2\pi\sigma^2}2\pi\rho\,\mathrm{d}\rho\\&=2\int_0^{r^2/(2\sigma^2)}(q-1)e^{-q}\,\mathrm{d}q\\&=\left[-2qe^{-q}\right]_0^{r^2/(2\sigma^2)}=-\frac{r^2}{\sigma^2}e^{-r^2/(2\sigma^2)}.\end{aligned} \tag{4.9}$$

现在固定圆盘，改变检测尺度。令 $t = r^2/(2\sigma^2)$，响应的绝对值为 $2te^{-t}$，其导数为 $2e^{-t}(1-t)$，在 $t=1$ 处取得最大值，因此

$$\sigma^\star = \frac{r}{\sqrt{2}}, \qquad r = \sqrt{2}\,\sigma^\star.$$

此时核的零交叉圆恰好贴合斑点边界，解释了图 4.8 中的“尺度匹配”。这个比例适用于理想圆盘。形状更复杂的区域也能用响应极值选择检测尺度，所选尺度反映该区域适合用多大的邻域描述。

> **备注 | 边缘与斑点**
> 阶跃边缘处，一阶导数有峰值，二阶导数过零；斑点检测则寻找二阶导数响应在空间和尺度中的极值。前者可用来定位边缘，后者用来定位斑点及其尺度。

4.4 DoG 检测

LoG 可以同时检测位置和尺度。为了减小多尺度计算的开销，可以将两张相邻尺度的高斯图像相减，近似得到 LoG 响应。下面从高斯核对尺度的导数推导这个关系。

DoG 与 LoG 的关系

对二维高斯核直接求尺度导数，

$$\frac{\partial G_\sigma}{\partial \sigma} = \left(-\frac{2}{\sigma} + \frac{x^2+y^2}{\sigma^3}\right) G_\sigma = \sigma \nabla^2 G_\sigma.$$

令相邻尺度为 σ 和 $k\sigma$，其中 $k>1$。尺度增量是 $\Delta\sigma = (k-1)\sigma$，一阶泰勒展开给出

$$G_{k\sigma} - G_\sigma \approx \Delta\sigma\, \partial_\sigma G_\sigma = (k-1)\sigma^2 \nabla^2 G_\sigma.$$

再与图像卷积，利用线性性把相减移到滤波结果之间，就得到

$$D(\boldsymbol{x};\sigma) = L(\boldsymbol{x};k\sigma) - L(\boldsymbol{x};\sigma) \approx (k-1)\mathcal{L}(\boldsymbol{x};\sigma). \tag{4.10}$$

这就是 DoG（Difference of Gaussians）。在这一近似下，固定 k 对应一个相同的比例因子，极值位置保持不变；k 越接近 1，近似越准确，但需要计算的尺度层也越多。

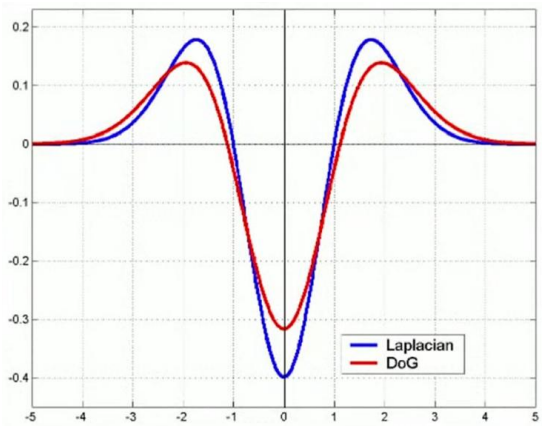


图 4.9 LoG 与 DoG 的核剖面具有相近的形状：中心与两侧符号相反，因此都能突出特定尺度的局部结构。两条曲线的幅值可以相差比例因子。

DoG 金字塔

相机拍到的物体可能放大两倍、四倍。为了用相同的比例比较大小，我们让相邻尺度之间保持固定倍率 k。一组（octave）覆盖从 σ_0 到 $2\sigma_0$ 的范围，分成 S 个间隔时，$k^S = 2$，所以 $k = 2^{1/S}$。常用 $S = 3$。

以 $S = 3$ 为例，从已有模糊尺度为 σ_0 的图像开始，一组内构造 6 幅高斯图像，尺度为 $\sigma_0, k\sigma_0, \ldots, k^5\sigma_0$；相邻相减得到 5 幅 DoG，只在中间 3 层检测，让每个候选都有上下尺度邻居。一般情形就是 $S+3$ 幅高斯图、$S+2$ 幅 DoG、S 层可检测响应。每个候选还要与本层的 8 个空间邻居、上下层各 9 个邻居比较，共 26 个。

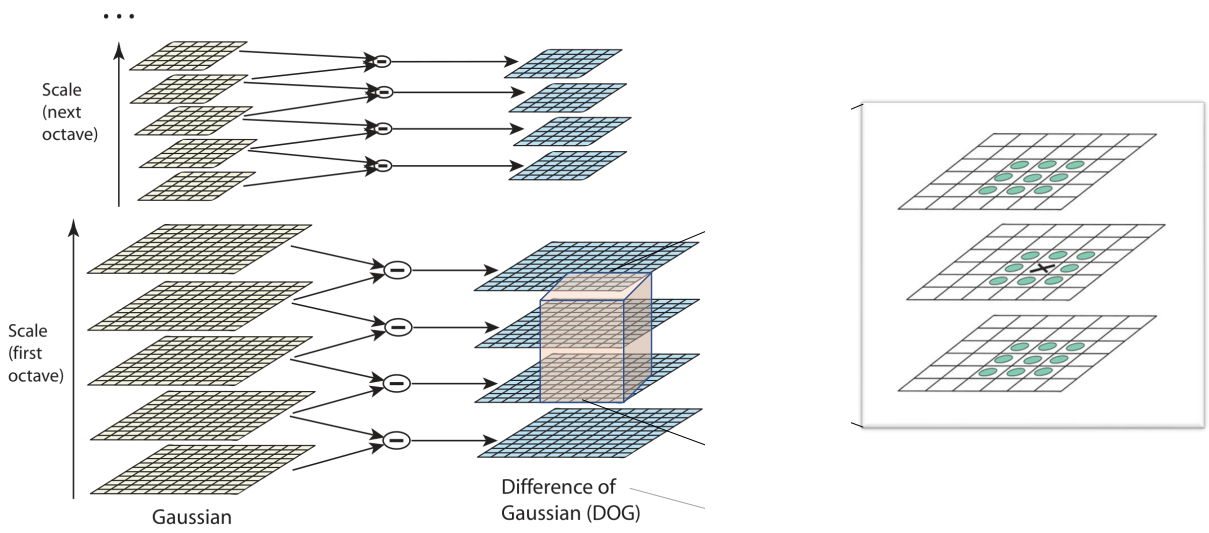


图 4.10 左：相邻高斯层相减得到 DoG。右：候选点同时与本层及相邻尺度的邻居比较。

到高斯尺度 $k^S\sigma_0 = 2\sigma_0$ 时，降采样一半作为下一组的起点。新网格的一个像素相当于原网格两个像素，局部尺度因而又变成 σ_0。组内的附加模糊量仍要按方差相加计算：$\sigma_{\text{inc}} = \sqrt{\sigma_{\text{next}}^2 - \sigma_{\text{current}}^2}$；这里用的是高斯平滑的方差相加关系。

关键点筛选与定位

得到离散极值后，先去掉响应过弱的点和主要来自直边的点。前一类容易受噪声影响，后一类沿边缘方向的定位较差。再用局部插值估计极值的精确位置，使坐标可以落在整数像素和尺度层之间。

把位置与层号写成 $\boldsymbol{q} = (x, y, \ell)^{\mathsf{T}}$，在候选附近把离散层号也视作可插值的坐标。记 D 在该点的梯度为 $\boldsymbol{g}$、二阶导数矩阵（Hessian）为 $H_3 \in \mathbb{R}^{3\times 3}$，其元素为 $\partial^2 D/\partial q_i \partial q_j$，二阶近似为

$$D(\boldsymbol{q} + \boldsymbol{\delta}) \approx D + \boldsymbol{g}^{\mathsf{T}}\boldsymbol{\delta} + \tfrac{1}{2}\boldsymbol{\delta}^{\mathsf{T}} H_3 \boldsymbol{\delta}.$$

令它对 $\boldsymbol{\delta}$ 的导数为零，得到 $H_3\hat{\boldsymbol{\delta}} = -\boldsymbol{g}$。再用 $\hat{\boldsymbol{\delta}}^{\mathsf{T}} H_3 \hat{\boldsymbol{\delta}} = -\boldsymbol{g}^{\mathsf{T}}\hat{\boldsymbol{\delta}}$ 代回，便有

$$\hat{\boldsymbol{\delta}} = -H_3^{-1}\boldsymbol{g}, \qquad D_{\text{refined}} = D + \tfrac{1}{2}\boldsymbol{g}^{\mathsf{T}}\hat{\boldsymbol{\delta}}. \tag{4.11}$$

这里假设 H_3 可逆，程序通过解线性方程求偏移。若偏移较大，可移动到新的候选位置后重新拟合；拟合失败或响应幅值过小时，则去掉这个候选。

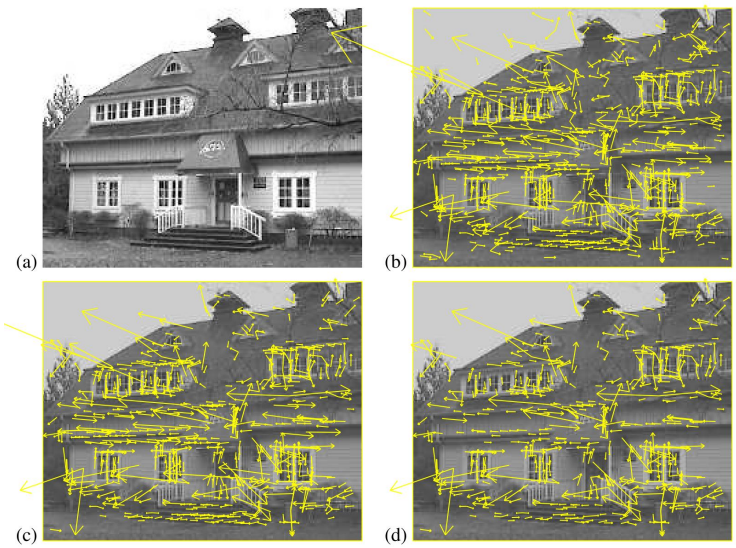


图 4.11 从左到右、从上到下依次为原图、初始候选、去除低对比度点后的结果，以及继续去除边缘响应后的结果。

判断是否像一条边，只需看空间方向的曲率矩阵

$$H_2 = \begin{bmatrix} D_{xx} & D_{xy} \\ D_{xy} & D_{yy} \end{bmatrix}.$$

若 $\det H_2 \le 0$，剔除。否则两特征值同号，令其绝对值为 $\alpha \ge \beta > 0$，比值 $\rho = \alpha/\beta$ 越大，两个方向的定位越不均衡。由于

$$\frac{(\operatorname{tr} H_2)^2}{\det H_2} = \frac{(\alpha+\beta)^2}{\alpha\beta} = \rho + 2 + \frac{1}{\rho},$$

右侧在 $\rho \ge 1$ 时随 ρ 增大。允许的最大曲率比为 r 时，可直接检测

$$\frac{(\operatorname{tr} H_2)^2}{\det H_2} < \frac{(r+1)^2}{r}, \qquad r > 1 \tag{4.12}$$

来筛选候选，常见示例取 $r = 10$。这里的 H_2 是 DoG 响应的二阶导数；Harris 的 M 则累积一阶梯度的外积。H_2 描述 DoG 在两个空间方向上的弯曲程度，M 则描述窗口平移后的灰度差异。

4.5 SIFT 描述子与匹配

经过检测和筛选，已经得到关键点的位置 $\boldsymbol{p}$ 和尺度 σ。接下来比较关键点附近的图像内容，判断两张照片中哪些点来自同一处物体。SIFT 会先将邻域的朝向和大小统一，再把其中的梯度分布写成向量。

主方向与局部坐标

尺度已由检测器给出，还缺一个参考方向。在对应的高斯层上计算梯度，按方向分箱，用梯度幅值和高斯空间权重投票。直方图主峰表示邻域最显著的方向，记为 θ_0。汇总一个邻域比只取单个像素方向更稳定；明显次峰也可产生额外方向，以减少主方向跳变。

对邻域位置和梯度方向作变换：

$$\boldsymbol{u} = \frac{1}{\sigma} R(-\theta_0)(\boldsymbol{x} - \boldsymbol{p}), \qquad \theta_{\mathrm{rel}}(\boldsymbol{x}) = \theta(\boldsymbol{x}) - \theta_0. \tag{4.13}$$

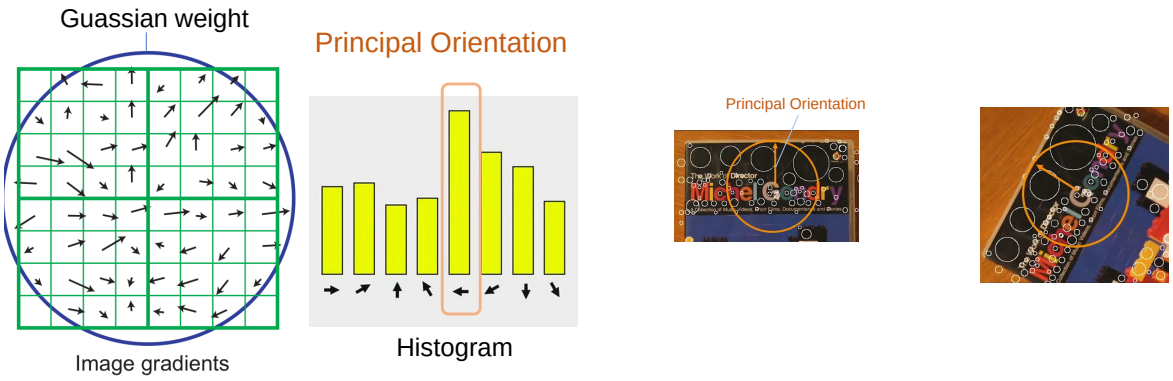


图 4.12 先找梯度方向直方图的主峰，再把邻域转到统一朝向。即使原图旋转，局部坐标系中的结构仍可相似。

减去 $\boldsymbol{p}$，把关键点移到原点；旋转 $-\theta_0$，把主方向转回统一朝向；再除以 σ，把邻域缩回统一大小。经过这些变换，两幅图中同一结构的局部位置就更接近，便于逐区比较。

SIFT 把这个邻域划分为 4×4 个空间单元，每个单元把 0 到 2π 的梯度方向分成 8 个箱。按固定顺序拼接 16 个八维直方图，就得到

$$\boldsymbol{d} \in \mathbb{R}^{128}, \qquad 128 = (4 \times 4) \times 8.$$

每个梯度还会向相邻空间格和方向格分配部分票数，避免稍微移动就突然换一个分箱。

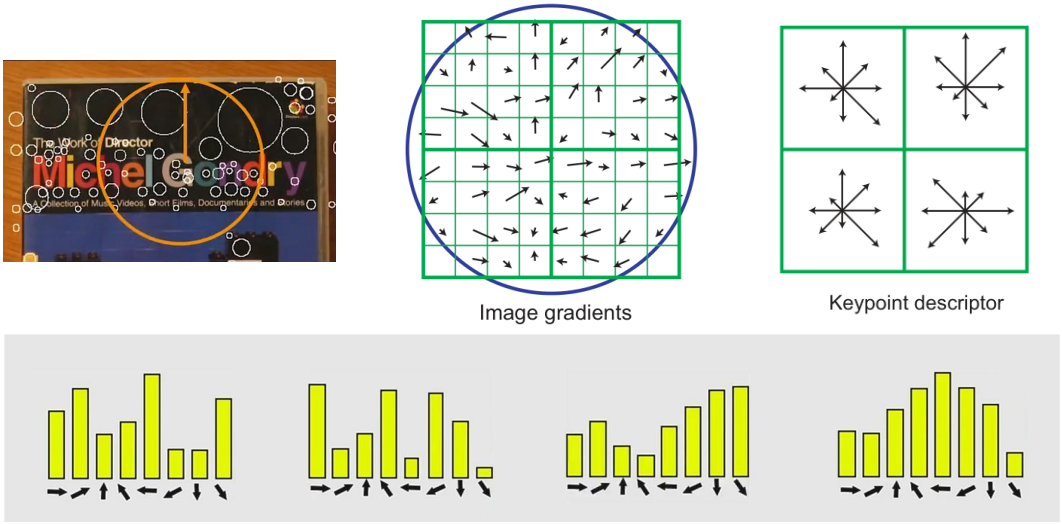


图 4.13 将邻域梯度分区统计，保留粗略的空间布局与方向分布。图中用 2×2 分区示意；标准描述子使用 4×4 分区。

> **数学直觉 ｜ 分区直方图**
> 每个分区分别统计方向，便保留了梯度在窗口中的大致位置。例如，上半部的水平边和下半部的竖直边，会记在不同的分区中。边缘在分区内略微移动时，直方图变化通常较小，因此这种描述也能容纳少量位置误差。

描述子归一化

在局部模型 $I' = aI + b$、$a > 0$ 下，求梯度消去了常量 b，所有梯度幅值乘以 a，所以未经归一化的直方图也近似乘以 a。对非零向量，$a\boldsymbol{d}/\|a\boldsymbol{d}\| = \boldsymbol{d}/\|\boldsymbol{d}\|$，解释了长度归一化为何能消除整体增益。标准做法还会限制过强分量，随后再次归一化：

$$\boldsymbol{d} \leftarrow \frac{\boldsymbol{d}}{\|\boldsymbol{d}\| + \varepsilon}, \quad d_i \leftarrow \min(d_i, 0.2), \quad \boldsymbol{d} \leftarrow \frac{\boldsymbol{d}}{\|\boldsymbol{d}\| + \varepsilon}. \tag{4.14}$$

截断可以减小少量强梯度对整个向量的影响。对于近似符合 $I' = aI + b$ 的亮度变化，这些处理能让描述子保持相近。明显的阴影、饱和或视角变化会进一步改变局部内容，匹配效果也会随之变化。

最近邻与比值检验

将两幅图的描述子按欧氏距离比较。对单位归一化的描述子，有

$$\|\boldsymbol{d}_1 - \boldsymbol{d}_2\|^2 = \|\boldsymbol{d}_1\|^2 + \|\boldsymbol{d}_2\|^2 - 2\boldsymbol{d}_1^{\mathsf{T}}\boldsymbol{d}_2 = 2 - 2\boldsymbol{d}_1^{\mathsf{T}}\boldsymbol{d}_2,$$

所以距离越小，对应的方向相似度越高。但重复纹理可能让多个候选都很接近。设最近邻、次近邻距离分别为 r_1, r_2，只在

$$r_1 < \tau r_2, \qquad 0 < \tau < 1 \tag{4.15}$$

时接受最近邻。比如距离为 $0.40, 0.41$ 时，两者几乎无法区分；若是 $0.40, 0.90$，最近邻就有明显优势。减小 τ 会要求最近邻有更明显的优势，因此保留的匹配更少。第五章会进一步用两幅图之间的几何关系筛选这些结果。

算法实现 | 提取 SIFT 并做比值筛选

```
sift = cv.SIFT_create()
k1, d1 = sift.detectAndCompute(gray1, None)
k2, d2 = sift.detectAndCompute(gray2, None)
good = []
if d1 is not None and d2 is not None and len(d2) >= 2:
    matcher = cv.BFMatcher(cv.NORM_L2)
    for pair in matcher.knnMatch(d1, d2, k=2):
        if len(pair) == 2:
            m, n = pair
            if m.distance < 0.75 * n.distance:
                good.append(m)
```

`gray1`，`gray2` 为灰度图；`d1`,`d2` 的每行是一个 128 维描述子，与关键点列表 `k1`,`k2` 一一对应。0.75 是比值阈值示例；匹配结果通过索引连接两边的关键点。接口见 OpenCV 的 SIFT 文档。

4.6 HOG 描述子

在行人检测中，我们会取一个图像窗口，判断里面是否有人。HOG 将窗口划分成规则网格，统计各处的梯度方向，以描述头部、躯干和四肢形成的轮廓。各个网格使用统一的方向基准，因此可以比较它们在整个窗口中的排列。

HOG 常把方向变为 $\theta \bmod \pi$：相反的梯度代表同一条边的两个明暗极性，因此可以统计到同一方向。整体旋转仍会改变方向分布，例如直立与横躺的人会得到不同的描述。

以高 128、宽 64 像素的检测窗口为例，按 8×8 像素划分，得到 16×8 个 cell，每个 cell 统计 9 个方向。再用 2×2 个 cell 组成一个 block，步长为一个 cell：竖向有 $16 - 2 + 1 = 15$ 个位置，横向有 $8 - 2 + 1 = 7$ 个位置。每个 block 拼接 4 个九维直方图，所以整窗维度为

$$15 \times 7 \times (2 \times 2 \times 9) = 3780. \tag{4.16}$$

对某个 block 的向量 $\boldsymbol{h}$，用

$$\hat{\boldsymbol{h}} = \frac{\boldsymbol{h}}{\sqrt{\|\boldsymbol{h}\|^2 + \varepsilon^2}}$$

归一化长度，必要时截断并再次归一化。这样局部变亮时，各方向的相对分布比绝对幅值更稳定。重叠 block 让同一个 cell 在不同局部背景下参与归一化。最后将全部 block 拼成整窗描述子 $\boldsymbol{z}$，交给线性分类器 $s(\boldsymbol{z}) = \boldsymbol{w}^{\top}\boldsymbol{z} + b$ 打分，并在图像金字塔上滑动窗口覆盖不同大小的目标。

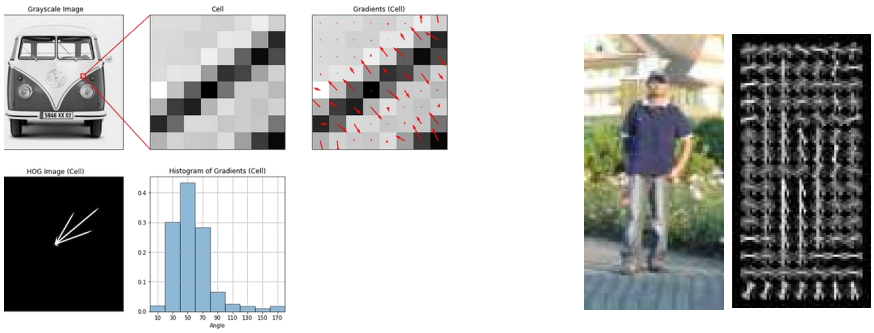


图 4.14 HOG 的可视化：每个网格中的短线表示梯度方向，线的强弱反映该方向的累计响应。

	SIFT	HOG
观察位置	稀疏关键点及其邻域	通常是密集规则网格
尺度与方向	检测尺度，对齐局部主方向	固定窗口与方向基准，尺度靠金字塔处理
比较目标	两个局部区域是否对应	一个窗口是否具有目标形状
典型用途	匹配、拼接、三维重建	目标检测与形状描述

第 5 章　图像拼接

图像拼接先利用匹配点估计两幅图之间的变换，将它们放到同一张画布上，再融合重叠区域。本章介绍单应估计、RANSAC，以及处理亮度接缝的几种方法。

两张风景照片中，同一座山的位置和形状可能不同，曝光也可能有差别。可以先用 SIFT 找到候选对应，再估计图像之间的变换，并筛掉错误匹配。将两幅图按这个变换对齐后，就可以用前面介绍的逆向采样和多尺度融合生成拼接结果。

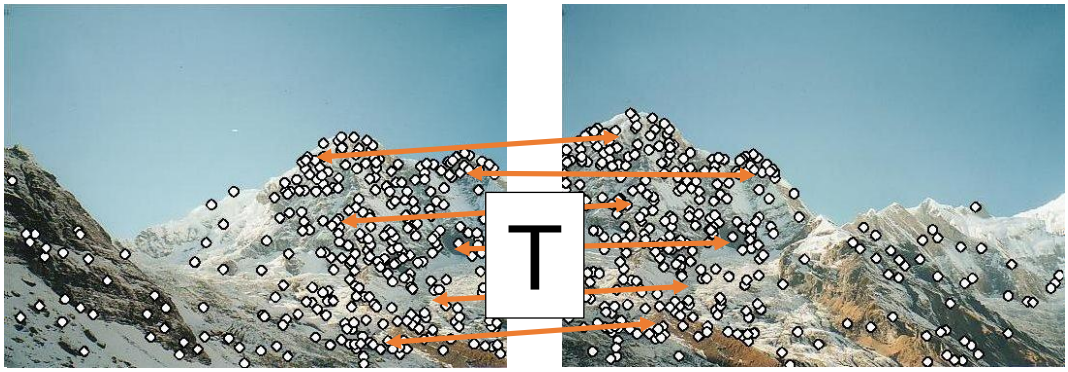


图 5.1　用局部特征找到两幅图中的对应点，再根据这些点估计整幅图像的变换。

5.1　图像间的变换

设源图中的点为 $\boldsymbol{x}_i = (x_i, y_i)^{\mathsf{T}}$，目标图中对应点为 $\boldsymbol{x}'_i = (u_i, v_i)^{\mathsf{T}}$。先假定对应关系正确，寻找满足 $\boldsymbol{x}'_i \approx T(\boldsymbol{x}_i)$ 的变换。变换的参数越多，能表示的形变也越多，求解时需要的独立对应关系相应增加。

仿射变换

仿射模型写成 $u = ax + by + c$、$v = dx + ey + f$。把六个未知数排成 $\boldsymbol{\beta} = (a, b, c, d, e, f)^{\mathsf{T}}$，一对点给出两个线性方程：

$$\begin{bmatrix} x_i & y_i & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & x_i & y_i & 1 \end{bmatrix} \boldsymbol{\beta} = \begin{bmatrix} u_i \\ v_i \end{bmatrix}.$$

叠起 n 对点得到 $A\boldsymbol{\beta} = \boldsymbol{b}$，其中 $A \in \mathbb{R}^{2n \times 6}$。三对不共线的点可确定仿射变换；若都在一条直线上，垂直于这条线的形变就缺少约束。有噪声时，多组方程通常无法同时满足，改为最小化

$$E(\boldsymbol{\beta}) = \|A\boldsymbol{\beta} - \boldsymbol{b}\|^2, \qquad \nabla E = 2A^{\mathsf{T}}(A\boldsymbol{\beta} - \boldsymbol{b}) = 0. \tag{5.1}$$

由此得到正规方程 $A^{\mathsf{T}}A\boldsymbol{\beta} = A^{\mathsf{T}}\boldsymbol{b}$。满列秩时解唯一；实现中用 QR 或 SVD 求最小二乘，例如 `np.linalg.lstsq(A, b, rcond=None)[0]`。

单应变换

透视图像中的平行线可以汇聚到消失点。为了表示这种变化，单应在仿射表达式中加入了随位置变化的分母。令 $\pi(a,b,c)=(a/c,b/c)$，则

$$\tilde{\boldsymbol{x}}' \sim H\tilde{\boldsymbol{x}}, \qquad \boldsymbol{x}' = \pi(H\tilde{\boldsymbol{x}}), \qquad H=\begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}. \tag{5.2}$$

也就是

$$u=\frac{h_{11}x+h_{12}y+h_{13}}{h_{31}x+h_{32}y+h_{33}}, \qquad v=\frac{h_{21}x+h_{22}y+h_{23}}{h_{31}x+h_{32}y+h_{33}}.$$

H 非奇异且只确定到一个非零倍数，因此有 $9-1=8$ 个自由度。当最后一行为 $(0,0,1)$ 时退化为仿射模型；新增的两项让图像不同位置具有不同的透视缩放。

单应的适用场景

下面两种常见的拍摄情形，都能由成像方程推导出单应关系。

所有对应点在同一平面上。将该平面选为世界坐标的 $Z=0$，相机 j 对平面点的投影为

$$\tilde{\boldsymbol{x}}_j \sim K_j[\boldsymbol{r}_{j1}\ \boldsymbol{r}_{j2}\ \boldsymbol{t}_j]\begin{bmatrix} X \\ Y \\ 1 \end{bmatrix} = H_j\begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}.$$

只要平面没有被看成一条线，H_j 可逆，消去平面坐标便得 $\tilde{\boldsymbol{x}}_2 \sim H_2H_1^{-1}\tilde{\boldsymbol{x}}_1$。两次拍摄允许存在平移，因为所有点共享同一平面。

相机绕同一光心旋转。场景中的点可以位于不同深度。若相机坐标之间满足 $\boldsymbol{X}_{c2}=R_{21}\boldsymbol{X}_{c1}$，把反投影射线 $\boldsymbol{X}_{c1}=Z_1K_1^{-1}\tilde{\boldsymbol{x}}_1$ 代入，有

$$\tilde{\boldsymbol{x}}_2 \sim K_2R_{21}K_1^{-1}\tilde{\boldsymbol{x}}_1, \qquad H=K_2R_{21}K_1^{-1}.$$

未知深度 Z_1 只贡献整体尺度，被齐次关系消去。

图 5.2 左：平面表面的不同视图，以及共享光心的不同朝向。右：具有深度层次的场景发生相机平移时，近处与远处的相对位移不同；对齐背景后，前景可能仍有重影。右图为效果示意。

相机平移时，近处和远处的物体会产生不同的图像位移，这种差异称为*视差*。若场景的深度差明显，按背景估计的单应就可能让前景错开。远景或较小的平移通常更适合用一个单应近似；视差较大时，可以分区域处理，或使用三维几何。

5.2 DLT 单应估计

给定对应点以后，可以求单应矩阵 H 的元素。将投影公式两边乘以分母，就得到关于这些元素的线性方程。这种求法称为直接线性变换（DLT）。

建立线性方程

对第 i 对点，将 u_i 的分母移到左侧并整理：

$$h_{11}x_i + h_{12}y_i + h_{13} - u_i h_{31}x_i - u_i h_{32}y_i - u_i h_{33} = 0.$$

v_i 同理。把 H 按行展开为九维向量 $\boldsymbol{h}$，这一对点对应

$$A_i\boldsymbol{h} = 0, \qquad A_i = \begin{bmatrix} x_i & y_i & 1 & 0 & 0 & 0 & -u_i x_i & -u_i y_i & -u_i \\ 0 & 0 & 0 & x_i & y_i & 1 & -v_i x_i & -v_i y_i & -v_i \end{bmatrix}. \tag{5.3}$$

叠起各个 A_i 得到 $A \in \mathbb{R}^{2n\times 9}$。四对一般位置的点可以提供八个独立约束：在这组最小样本中，两幅图里的点都应互异，且任意三点不共线。满足这些位置条件时，四对点可以确定一个单应。

为了在齐次方程中选出非零解，我们固定 $\|\boldsymbol{h}\| = 1$，然后求

$$\min_{\|\boldsymbol{h}\|=1} \|A\boldsymbol{h}\|^2. \tag{5.4}$$

若 $A = U\Sigma V^{\mathsf{T}}$，写 $\boldsymbol{h} = V\boldsymbol{z}$，正交矩阵不改变长度，因而 $\|A\boldsymbol{h}\|^2 = \|\Sigma\boldsymbol{z}\|^2 = \sum_j \sigma_j^2 z_j^2$。在 $\sum_j z_j^2 = 1$ 下，把向量放在最小奇异值对应的方向，能让这个和最小。所以取 V 的最后一列，重排为 3×3 矩阵。

> **数学直觉 ｜ 齐次方程的尺度**
>
> H 和 $2H$ 在透视除法后给出相同的像素坐标,因此可以从任意非零倍数中选一个代表。$\|\boldsymbol{h}\| = 1$ 就是一种选择。若求得的 $h_{33} \neq 0$，也可以将整个矩阵除以 h_{33}，使右下角元素等于一。有噪声时，SVD 在选定尺度下寻找方程残差平方和最小的解。

坐标归一化

像素坐标可能达到数千，A 同时包含 1、x_i 和 $u_i x_i$，各列的量级相差很大。这会放大舍入误差，也会让代数误差过分依赖坐标原点和单位。对两组点分别做同一种预处理：减去各自质心 $\bar{\boldsymbol{x}}$，再乘 $s = \sqrt{2}/\bar{r}$，其中 $\bar{r}$ 是点到质心的平均距离。对应的矩阵为

$$T = \begin{bmatrix} s & 0 & -s\bar{x} \\ 0 & s & -s\bar{y} \\ 0 & 0 & 1 \end{bmatrix}.$$

归一化后的平均距离为 $\sqrt{2}$。设两幅图分别用 T_1, T_2，在新坐标中求得 H_n，则

$$T_2\tilde{\boldsymbol{x}}' \sim H_n T_1 \tilde{\boldsymbol{x}} \quad\Longrightarrow\quad H = T_2^{-1} H_n T_1.$$

这个预处理改变坐标的原点和单位，用来改善线性方程的数值条件。

算法实现 | 归一化 DLT

`src_n`，`dst_n` 是按上述规则归一化后的 $N \times 2$ 点数组，`T1`，`T2` 是各自的坐标变换。

```
A = []
for (x, y), (u, v) in zip(src_n, dst_n):
    A += [[x,y,1,0,0,0,-u*x,-u*y,-u],
          [0,0,0,x,y,1,-v*x,-v*y,-v]]
_, _, Vt = np.linalg.svd(np.asarray(A), full_matrices=True)
Hn = Vt[-1].reshape(3, 3)
H = np.linalg.solve(T2, Hn) @ T1
H /= np.linalg.norm(H)
```

恰好四对点时 A 为 8×9。完整 SVD 保留全部九个右奇异向量，最后一个就是所需的零空间方向。代码要求对应点提供独立约束，且各组点的平均距离非零。

重投影误差

DLT 的目标是方程残差平方和。将这个残差展开，可以看出它与像素误差之间的关系。令 $w_i = h_{31}x_i + h_{32}y_i + h_{33}$，式 (5.3) 第一行的残差实际上是 $w_i(\hat{u}_i - u_i)$。不同点的像素误差被不同的 w_i 重新加权了。因此，用 DLT 初始化之后，可以直接最小化重投影误差

$$E_{\text{reproj}}(H) = \sum_i \left\| \boldsymbol{x}'_i - \pi(H\tilde{\boldsymbol{x}}_i) \right\|^2. \tag{5.5}$$

若两幅图的测量误差都需要考虑，也可使用对称传递误差，在每项中再加 $\|\boldsymbol{x}_i - \pi(H^{-1}\tilde{\boldsymbol{x}}'_i)\|^2$。这些目标包含透视除法，通常通过迭代优化求解。实际流程先用下一节的 RANSAC 筛选对应，再用内点优化重投影误差。

5.3 RANSAC

最小二乘适合处理小幅定位误差。特征匹配还可能把两扇相似的窗认成同一扇，这时误差会远大于定位噪声。一个偏差 100 像素的点，在平方误差中占 10000，相当于一万个偏差 1 像素的点。少量错配因此就可能明显改变拟合结果。

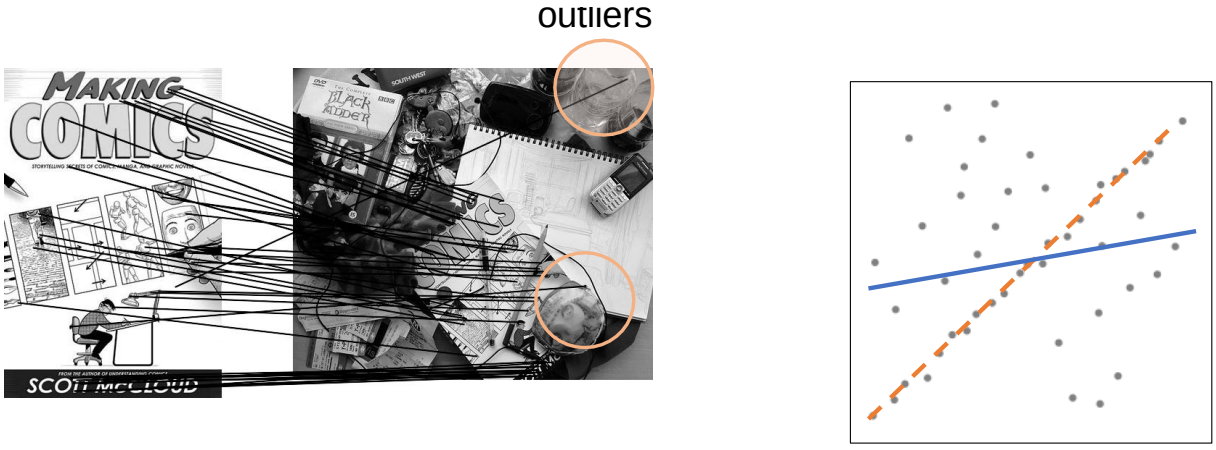


图 5.3 左：重复的外观容易产生错误匹配。右：一个偏离较大的点，使最小二乘直线偏离其余数据。

RANSAC 每次随机选取一小组对应点，先用它们估计变换，再检查其余点是否符合这个变换。对于单应，每次选四对提供独立约束的匹配来估计 H，然后计算所有点的重投影距离

$$e_i = \|\boldsymbol{x}_i' - \pi(H\tilde{\boldsymbol{x}}_i)\|, \qquad \mathcal{I}(H) = \{i : e_i < \tau\}.$$

$\mathcal{I}(H)$ 是该模型的内点集，τ 与正常定位误差处于相近的像素尺度。重复抽样后，保留内点最多的模型，最后**用全部内点重新估计并优化**。

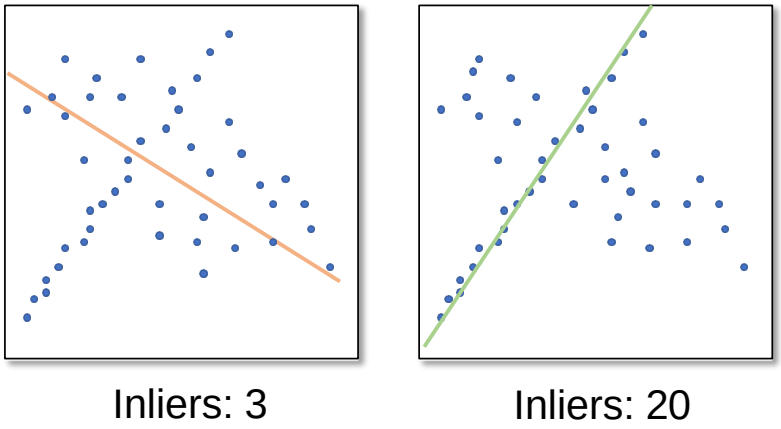


图 5.4 不同的随机样本产生不同的拟合直线。图中比较了每条直线附近的内点数量；单应估计也按同样的办法比较候选模型。

采样次数

设匹配中的内点比例为 w，最小样本大小为 s。近似独立抽样时，一次抽到全内点样本的概率为 w^s，一次失败的概率为 $1 - w^s$。连续 N 次都失败的概率是 $(1 - w^s)^N$。若要求成功概率至少为 p，就需要

$$(1 - w^s)^N \le 1 - p \quad \Longrightarrow \quad N \ge \frac{\log(1 - p)}{\log(1 - w^s)}. \tag{5.6}$$

两边取对数后，分母为负，所以除过去时不等号反向。单应取 $s = 4$、$p = 0.99$：若 $w = 0.5$，至少约需 72 次；若 $w = 0.3$，则约需 567 次。提高内点比例，可以明显减少所需的采样次数。上式计算抽到全内点样本的概率；求解时还要选取位置分布合适的样本，并使用适合场景的变换。

算法实现 ｜ 从候选匹配估计单应

`src`，`dst` 为 $N \times 2$ 浮点像素坐标，按同一顺序配对；至少需要四对。

```
H, mask = cv.findHomography(
    src, dst, cv.RANSAC, ransacReprojThreshold=3.0,
    maxIters=5000, confidence=0.995)
if H is None:
    raise ValueError("No reliable homography")
inliers = mask.ravel().astype(bool)
```

3 像素是示例阈值，可按定位精度调整。内点覆盖重叠区域的不同位置时，更有利于估计整幅图的变换；拼接后可用图像边缘的对应情况检查结果。

较小的阈值筛选更严格，较大的阈值则允许更多定位误差。重复纹理可能形成一组彼此一致的错配，因此还应结合场景内容检查，选出的变换是否对齐了希望拼接的区域。

5.4 画布与图像融合

有了 $H_{B\to A}$，就可以把图 B 变换到图 A 的坐标系。变换后的图像可能超出 A 的边界，甚至出现负坐标，因此需要计算一张能够容纳两图的画布。先变换两幅图的边界角点，求合并区域的包围盒 $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$，再平移整个结果。令整数左上界为 $a = \lfloor x_{\min} \rfloor$、$b = \lfloor y_{\min} \rfloor$，则

$$T_c = \begin{bmatrix} 1 & 0 & -a \\ 0 & 1 & -b \\ 0 & 0 & 1 \end{bmatrix}, \qquad H_A = T_c, \quad H_B = T_c H_{B\to A}.$$

输出宽高可由边界向外取整得到，并对所用像素中心或像素边界约定保留足够边缘。这里要求单应的分母在输入区域内保持同号且远离零，使变换后的边界位于有限范围内。

逐个遍历画布像素，用第三章的逆向采样填值。同时变换一幅全 1 的有效区域掩码，用最近邻插值记录哪些位置有来源。**有效区域由掩码记录**。这样即使图像内容是纯黑色，也能保留它的覆盖信息。

重叠区可用权重 w_A, w_B 混合：

$$C(\boldsymbol{x}) = \frac{w_A(\boldsymbol{x}) A_w(\boldsymbol{x}) + w_B(\boldsymbol{x}) B_w(\boldsymbol{x})}{w_A(\boldsymbol{x}) + w_B(\boldsymbol{x})},$$

其中 A_w, B_w 是对齐后的图像，无效区权重为零；分母为零处属于画布空白。权重可以从边界向内部逐渐增大，或采用第三章的多频带融合。

对齐后仍有接缝　　融合后的结果

图 5.5 两幅图对齐后，在重叠区域混合颜色，可以减轻亮度突变和可见的接缝。

多幅图拼接时，各图都应映射到同一参考坐标系。例如 $H_{C\to A} = H_{B\to A} H_{C\to B}$，矩阵顺序从右向左读取。长链相乘会累积误差；有多处重叠时，可以同时优化各图的变换，使全部匹配共同约束结果。视场很宽时，柱面或球面画布也常比单张透视平面更自然。

5.5 泊松融合

将一个物体粘贴到新背景中时，可以保留源图的边缘和纹理，并调整它的整体色调，使边界接上背景。泊松融合通过相邻像素的灰度差来保留这些局部细节。给整个区域加上同一个常数，会改变亮度，而像素之间的差值保持不变。

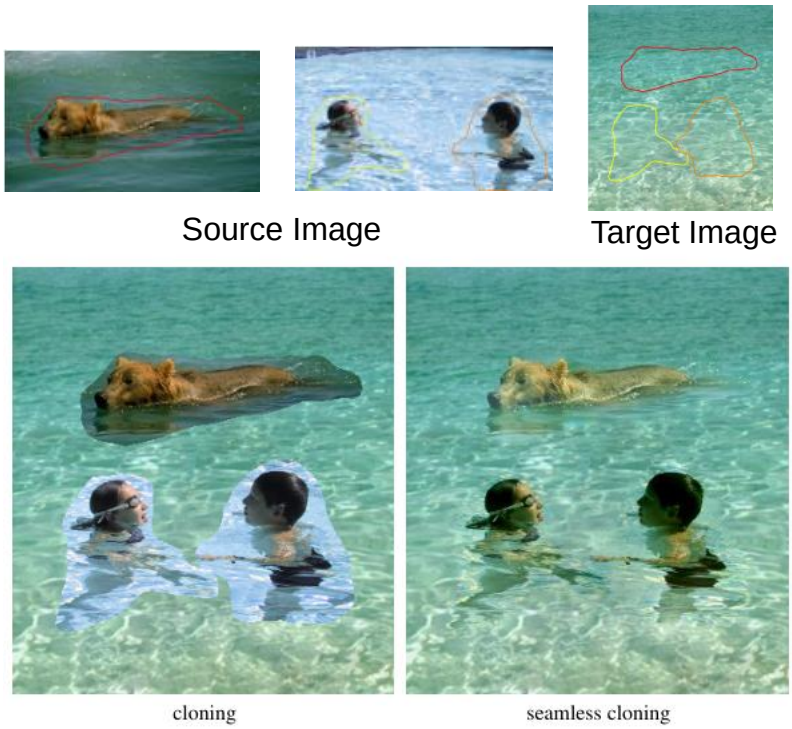


图 5.6 直接粘贴保留源图的绝对颜色，边界容易显露。梯度域融合让区域内部保留源图的局部变化，同时让边界接上目标背景；整体色调因而可以改变。

梯度与边界条件

设 g 是已经几何对齐的源图，f^* 是目标背景，Ω 是待融合区域，未知量 f 是最终输出。给定引导梯度 $\boldsymbol{v}$，求

$$\min_f \int_\Omega \|\nabla f - \boldsymbol{v}\|^2 \,\mathrm{d}\boldsymbol{x}, \qquad f\big|_{\partial\Omega} = f^*\big|_{\partial\Omega}. \tag{5.7}$$

普通泊松融合取 $\boldsymbol{v} = \nabla g$，使输出的梯度尽量接近源图。由于 f 和 $f + c$ 具有相同梯度，还需要边界条件来确定整体亮度。把边界固定为背景值，就让粘贴区域在边缘接上目标图像。若源梯度与这个边界无法同时精确满足，最小二乘会在区域内求一个整体误差较小的结果。

让 f 产生一个在边界为零的小变化 η。目标的一阶变化为

$$\delta E = 2\int_\Omega (\nabla f - \boldsymbol{v}) \cdot \nabla\eta = -2\int_\Omega (\Delta f - \nabla\cdot\boldsymbol{v})\eta.$$

第二步使用分部积分，边界项因 $\eta = 0$ 消失。最小值处的一阶变化对任意 η 都为零，因此

$$\Delta f = \nabla\cdot\boldsymbol{v}, \qquad \text{普通情形为} \Delta f = \Delta g. \tag{5.8}$$

这里的拉普拉斯与前面的 LoG 使用同一种二阶微分运算，但用途不同：LoG 测量局部结构，泊松方程则从局部差异恢复整片像素值。

离散泊松方程

在四邻域网格上，令有向边的引导差为 $v_{pq} = g_p - g_q$，并令 $v_{qp} = -v_{pq}$。最小化相邻像素差的误差 $\sum_{\{p,q\}} (f_p - f_q - v_{pq})^2$，对一个内部未知像素 f_p 求导并令其为零，得到

$$|N_p| f_p - \sum_{q\in N_p} f_q = \sum_{q\in N_p} v_{pq}.$$

N_p 是实际存在的四邻域。内部邻居也是未知数；区域外紧邻的像素已由背景固定，移到右侧：

$$|N_p| f_p - \sum_{q \in N_p \cap \Omega} f_q = \sum_{q \in N_p} v_{pq} + \sum_{q \in N_p \setminus \Omega} f_q^*. \tag{5.9}$$

每个方程最多连接自己和四个邻居，所以系数矩阵非常稀疏。逐行装配后求解 $L\boldsymbol{f} = \boldsymbol{b}$；每个连通区域有固定边界约束时，通常可获得唯一解。RGB 图像可以逐通道求解。若一个连通分量完全没有固定边界，还要补充锚点来消除常量零空间。

数学直觉 ｜ 单个像素的求解

设只有中心像素未知，四个已知邻居都是 100。若源图要求中心比每个邻居亮 10，则右侧梯度差之和为 40，边界值之和为 400，所以 $4f_p = 440$，得到 $f_p = 110$。源图原先的中心可能是 30、邻居是 20；保留下来的是差值 10，整体基准由目标背景决定。

算法实现 ｜ 泊松融合

```
out = cv.seamlessClone(src, dst, mask, center, cv.NORMAL_CLONE)
```

`src`,`dst` 为八位三通道图像，`mask` 为源图坐标中的八位单通道掩码，非零处表示选区；`center` 指定选区放到目标图中的中心位置。拼接时先完成几何变换，再把待融合区域与掩码放到正确位置。

图 5.7 泊松融合保留了源图的局部细节，并使边界接上背景。物体的整体色调也随背景发生变化。

对每条边，还可以从源图与目标图中选取幅值更大的梯度作为引导，形成混合梯度。此时引导场未必对应某幅图的精确梯度，最小二乘求的是尽量接近它、又满足边界的图像。

第 6 章　相机标定与三角测量

相机标定根据已知的空间点和像素位置求相机参数。三角测量则利用已知的相机参数，从多个视图计算空间点。这一章分别介绍这两个过程，最后说明怎样利用建筑物中的正交方向标定相机。

第二章的反投影公式将像素写成从光心出发的一条射线。把相机移到另一个位置，再找到同一物体点，就能得到第二条射线。已知两台相机的位置和朝向时，可以通过两条射线的交会计算空间点的位置。

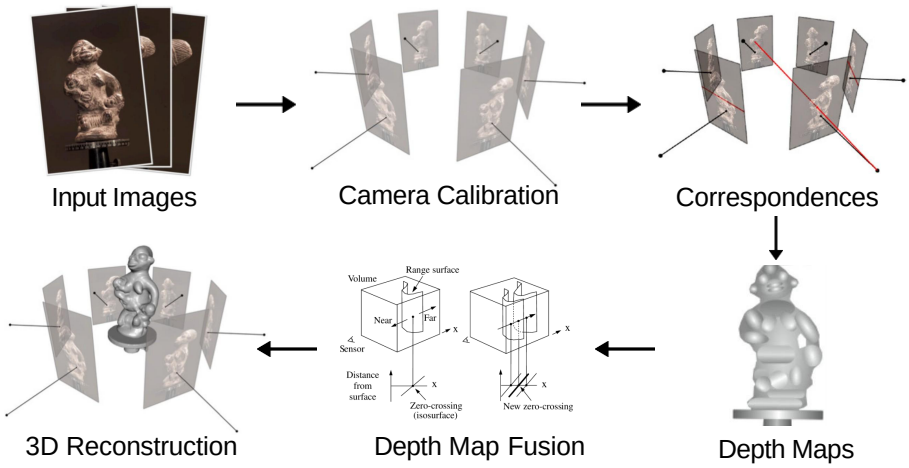


图 6.1　三维重建的一般过程：标定相机、匹配像点、计算深度，再将多个深度结果融合成表面。

6.1　相机矩阵估计

先拍摄一个尺寸已测量好的标定装置。每个标记点都有已知的三维坐标，在照片中也能找到它的像素位置。将这些对应代入成像方程，就可以求相机矩阵。先采用理想针孔模型：

$$\tilde{\boldsymbol{x}}_i \sim P\tilde{\boldsymbol{X}}_i, \qquad P = K[R \mid \boldsymbol{t}] \in \mathbb{R}^{3\times 4}.$$

这里的 $\boldsymbol{X}_i$ 和 $\boldsymbol{x}_i$ 已知，待求的是 P 的十二个元素。整体相差一个非零倍数的矩阵产生相同投影，所以 P 有十一个自由度。

三维 DLT

记 P 的三行为 $\boldsymbol{p}_1^{\mathsf{T}}, \boldsymbol{p}_2^{\mathsf{T}}, \boldsymbol{p}_3^{\mathsf{T}}$，$\tilde{\boldsymbol{X}}_i = (X_i, Y_i, Z_i, 1)^{\mathsf{T}}$，图像点为 (u_i, v_i)。于是

$$u_i = \frac{\boldsymbol{p}_1^{\mathsf{T}}\tilde{\boldsymbol{X}}_i}{\boldsymbol{p}_3^{\mathsf{T}}\tilde{\boldsymbol{X}}_i}, \qquad v_i = \frac{\boldsymbol{p}_2^{\mathsf{T}}\tilde{\boldsymbol{X}}_i}{\boldsymbol{p}_3^{\mathsf{T}}\tilde{\boldsymbol{X}}_i}.$$

与单应 DLT 完全相同，乘掉分母，再把未知元素按行堆成 $\boldsymbol{p} \in \mathbb{R}^{12}$，每个点给出

$$\begin{bmatrix} \tilde{\boldsymbol{X}}_i^{\mathsf{T}} & \boldsymbol{0}^{\mathsf{T}} & -u_i\tilde{\boldsymbol{X}}_i^{\mathsf{T}} \\ \boldsymbol{0}^{\mathsf{T}} & \tilde{\boldsymbol{X}}_i^{\mathsf{T}} & -v_i\tilde{\boldsymbol{X}}_i^{\mathsf{T}} \end{bmatrix} \boldsymbol{p} = \boldsymbol{0}. \tag{6.1}$$

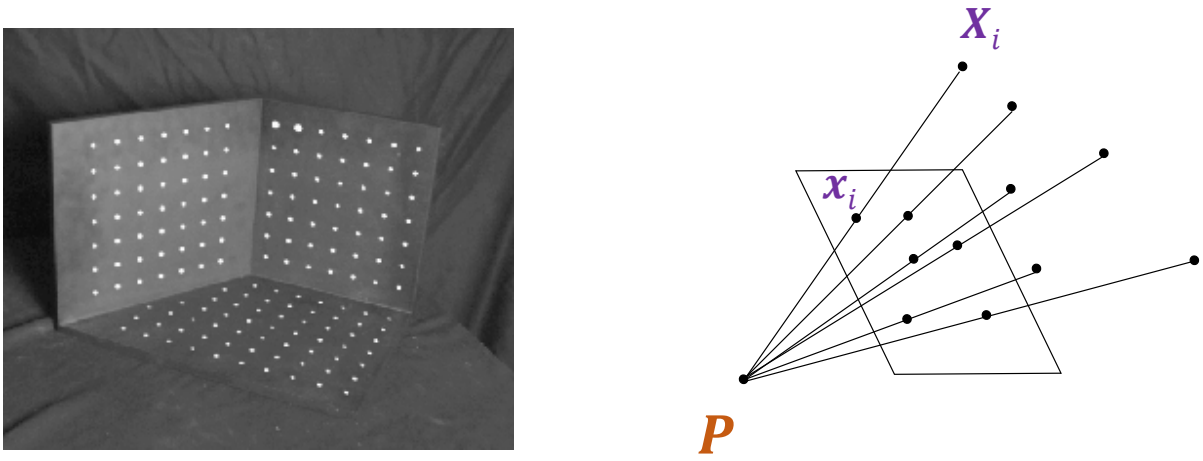


图 6.2 标定装置上的点具有已知三维坐标。选择分布在不同深度和图像位置的点，有助于估计相机参数。

这里每个块有四列，所以整体为 2×12。叠起各点得到 $A\boldsymbol{p} \approx 0$，求 $\min_{\|\boldsymbol{p}\|=1} \|A\boldsymbol{p}\|^2$，取最小右奇异向量并重排成 3×4 矩阵。最小右奇异向量的求法与式 (5.4)相同。

标定点的分布

每个点给出两个方程，因此至少需要六个三维点。点的位置还会影响方程是否独立。例如，若所有点都满足 $Z_i = 0$，P 的第三列始终乘以零，改变这一列也会得到相同投影。此时成像方程为

$$\tilde{\boldsymbol{x}}_i \sim K[\boldsymbol{r}_1\ \boldsymbol{r}_2\ \boldsymbol{t}] \begin{bmatrix} X_i \\ Y_i \\ 1 \end{bmatrix}.$$

它确定了从标定平面到图像的单应。要估计完整的 P，需要选择非共面的点，使无噪声时的设计矩阵满足 $\operatorname{rank}(A) = 11$。这时零空间为一维，$P$ 就确定到一个整体倍数。点分布过于集中或接近平面时，求解容易受定位误差影响，因此通常让标定点覆盖不同深度和图像位置。

数值处理也沿用前章：二维点居中并缩放到平均距离 $\sqrt{2}$，三维点居中并缩放到平均距离 $\sqrt{3}$。若二维归一化为 T、三维齐次归一化为 $U \in \mathbb{R}^{4\times4}$，解得 P_n 后

$$T\tilde{\boldsymbol{x}} \sim P_n U \tilde{\boldsymbol{X}} \quad \Longrightarrow \quad P = T^{-1} P_n U.$$

归一化改善各列的数值量级；点的空间分布则决定方程能否唯一确定 P。

6.2 内参与外参

求得 P 后，可以进一步分解出焦距、主点、旋转和平移。写 $P = [M \mid \boldsymbol{p}_4]$，与 $P = K[R \mid \boldsymbol{t}]$ 对照，得到

$$M = KR, \qquad \boldsymbol{p}_4 = K\boldsymbol{t}. \tag{6.2}$$

K 是上三角矩阵，R 是正交矩阵，所以应使用上三角乘正交的 RQ 分解。

RQ 分解

将矩阵按行展开，可以具体看到分解的过程。设 R 的三行是单位正交向量 $\boldsymbol{r}_1^{\mathsf{T}}, \boldsymbol{r}_2^{\mathsf{T}}, \boldsymbol{r}_3^{\mathsf{T}}$，则矩阵乘法逐行给出

$$\begin{aligned}\boldsymbol{m}_3 &= \boldsymbol{r}_3,\\ \boldsymbol{m}_2 &= f_y\boldsymbol{r}_2 + c_y\boldsymbol{r}_3,\\ \boldsymbol{m}_1 &= f_x\boldsymbol{r}_1 + s\boldsymbol{r}_2 + c_x\boldsymbol{r}_3.\end{aligned}$$

这里 $\boldsymbol{m}_i^{\mathsf{T}}$ 为 M 的第 i 行，已采用 $K_{33}=1$ 的尺度。最后一行直接给出第三个旋转方向；第二行沿它的投影是主点 c_y，剩余部分的长度是 f_y：

$$c_y = \boldsymbol{m}_2^{\mathsf{T}}\boldsymbol{r}_3, \qquad \boldsymbol{u}_2 = \boldsymbol{m}_2 - c_y\boldsymbol{r}_3, \qquad f_y = \|\boldsymbol{u}_2\|, \quad \boldsymbol{r}_2 = \boldsymbol{u}_2/f_y.$$

再从第一行扣掉已有两个方向，

$$\begin{aligned}c_x &= \boldsymbol{m}_1^{\mathsf{T}}\boldsymbol{r}_3, \qquad s = \boldsymbol{m}_1^{\mathsf{T}}\boldsymbol{r}_2,\\ \boldsymbol{u}_1 &= \boldsymbol{m}_1 - c_x\boldsymbol{r}_3 - s\boldsymbol{r}_2, \qquad f_x = \|\boldsymbol{u}_1\|, \quad \boldsymbol{r}_1 = \boldsymbol{u}_1/f_x.\end{aligned}$$

这个过程相当于从最后一行开始做 Gram–Schmidt 正交化。各行之间的投影给出主点和偏斜参数，剩余向量的长度给出焦距，单位方向则组成旋转矩阵。程序通常直接调用 RQ 分解完成计算。

尺度与符号

DLT 的输出可以整体乘以任意非零数。采用第二章的正焦距约定时，可先选取使 $\det M > 0$ 的整体符号，再将整个 P 除以 M 第三行的长度。这样 $K_{33}=1$，配合正的 f_x, f_y，能得到 $\det R = 1$。也可以在 RQ 分解后统一调整对角符号与尺度。缩放时对整个 P 使用同一比例，使前三列和第四列保持一致。

在同一尺度下，$\boldsymbol{t} = K^{-1}\boldsymbol{p}_4$。另一个更直接的量是光心 $\boldsymbol{C}$：它满足 $M\boldsymbol{C} + \boldsymbol{p}_4 = 0$，所以

$$\boldsymbol{C} = -M^{-1}\boldsymbol{p}_4, \qquad \boldsymbol{t} = -R\boldsymbol{C}. \tag{6.3}$$

P 整体缩放时，M 和 $\boldsymbol{p}_4$ 的倍数在上式中抵消，所以光心坐标保持不变。

算法实现 ｜ 分解投影矩阵

```
P = np.asarray(P, dtype=np.float64)
if np.linalg.det(P[:, :3]) < 0:
    P = -P
K, R, C_h, *_ = cv.decomposeProjectionMatrix(P)
K = K / K[2, 2]
C = (C_h[:3] / C_h[3]).reshape(3)
t = -R @ C
```

该接口的四维输出 `C_h` 表示齐次光心位置。计算 $\boldsymbol{t}$ 后，检查 $K[R \mid \boldsymbol{t}]$ 是否与原 P 成比例，以及标定点是否位于相机前方。

非线性标定

可以把线性解作为初值，再加入镜头畸变，按像素位置的误差继续优化。调整相机参数后，将标定点重新投影到图像上，并比较预测位置与观测位置。对第 j 张标定图，设畸变参数为 $\boldsymbol{d}$，带畸变的投影函数为 $\Pi(K, \boldsymbol{d}, R_j, \boldsymbol{t}_j; \boldsymbol{X}_i)$，优化

$$\min_{K,\boldsymbol{d},\{R_j,\boldsymbol{t}_j\}} \sum_{j,i} \left\| \boldsymbol{x}_{ij} - \Pi(K, \boldsymbol{d}, R_j, \boldsymbol{t}_j; \boldsymbol{X}_i) \right\|^2 . \tag{6.4}$$

在焦距等设置不变时，多幅图共享内参和畸变，每幅图有自己的外参。旋转可用三个参数的旋转向量表示，更新这些参数后再生成旋转矩阵，从而保持正交性。整个过程用非线性最小二乘迭代求解。

平面棋盘格也可以用于标定，做法是从多个不同的倾斜方向拍摄。各幅图的单应都含有同一个内参矩阵 K，旋转矩阵的列又要满足单位正交条件。把多幅图的条件联合起来，就能求出内参。拍摄时让棋盘格覆盖图像中心和边缘，并改变它相对于相机的角度。

算法实现 ｜ 棋盘格标定

```
rms, K, dist, rvecs, tvecs = cv.calibrateCamera(
    object_points, image_points, (width, height), None, None)
```

两组输入均为按视图组织的列表：每幅图对应 $N_j \times 3$ 的已知棋盘格坐标和 $N_j \times 2$ 的像素坐标，使用 `float32`；格点顺序必须一致。棋盘格可以取 $Z = 0$，格宽的物理单位也决定平移的单位。这里采用多视图平面标定的初始化；任意非平面标定物需要合适的内参初值。

评价标定结果时，可以同时查看平均误差和各个点的残差。若误差在图像边缘随半径有规律地变化，可检查畸变模型；若只有个别点误差较大，则检查这些点的定位和对应关系。改变焦距、裁剪或缩放图像后，内参也要按相应成像设置更新。

6.3 三角测量

标定完成后，已知两台相机的 P_1, P_2。给定同一个物体点在两幅图中的像素位置，现在要求它的三维坐标 $\boldsymbol{X}$。下面采用理想针孔矩阵，因此先将观测像素去畸变。

图 6.3 从两幅图中找到同一个物体点，结合相机参数，就可以计算两条反投影视线。

射线与中点法

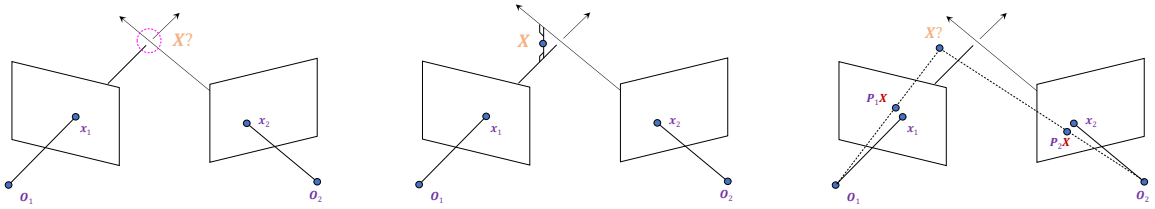
由第二章的反投影公式，第 j 台相机对应一条世界坐标射线

$$\boldsymbol{X} = \boldsymbol{C}_j + \lambda_j \boldsymbol{a}_j, \qquad \boldsymbol{a}_j = \frac{R_j^{\mathsf{T}} K_j^{-1} \tilde{\boldsymbol{x}}_j}{\| R_j^{\mathsf{T}} K_j^{-1} \tilde{\boldsymbol{x}}_j \|}, \qquad \lambda_j > 0.$$

无噪声时，两条射线相交于真实点；有噪声时，空间中的两条线通常不再相交。一种直观近似是求两条直线上距离最近的点，再取其连线中点。令 $D = [\boldsymbol{a}_1 \ -\boldsymbol{a}_2]$，则先解一个两未知量的最小二乘问题：

$$\min_{\lambda_1,\lambda_2} \left\| D \begin{bmatrix} \lambda_1 \\ \lambda_2 \end{bmatrix} - (\boldsymbol{C}_2 - \boldsymbol{C}_1) \right\|^2 . \tag{6.5}$$

将解代回两条线得到 $\boldsymbol{Q}_1, \boldsymbol{Q}_2$，取 $\hat{\boldsymbol{X}} = (\boldsymbol{Q}_1 + \boldsymbol{Q}_2)/2$。最后检查候选点在两台相机中的深度，保留两侧深度均为正的结果。



噪声使射线错开　　最短连线的中点　　比较重投影位置

图 6.4　左：噪声使两条视线错开。中：取最短连线的中点估计空间点。右：将候选点投回图像，计算它与观测像素之间的距离。

线性三角测量

设相机 j 的三行为 $\boldsymbol{p}_{j1}^{\mathsf{T}}, \boldsymbol{p}_{j2}^{\mathsf{T}}, \boldsymbol{p}_{j3}^{\mathsf{T}}$。对观测 (u_j, v_j)，乘去透视分母后得到

$$(u_j \boldsymbol{p}_{j3}^{\mathsf{T}} - \boldsymbol{p}_{j1}^{\mathsf{T}})\tilde{\boldsymbol{X}} = 0, \qquad (v_j \boldsymbol{p}_{j3}^{\mathsf{T}} - \boldsymbol{p}_{j2}^{\mathsf{T}})\tilde{\boldsymbol{X}} = 0.$$

这里 P_j 已知，方程的四个未知数是齐次坐标 $\tilde{\boldsymbol{X}}$ 的分量。两台相机叠成 4×4 矩阵，更多相机则继续增加两行：

$$A\tilde{\boldsymbol{X}} = 0, \qquad A = \begin{bmatrix} u_1 \boldsymbol{p}_{13}^{\mathsf{T}} - \boldsymbol{p}_{11}^{\mathsf{T}} \\ v_1 \boldsymbol{p}_{13}^{\mathsf{T}} - \boldsymbol{p}_{12}^{\mathsf{T}} \\ u_2 \boldsymbol{p}_{23}^{\mathsf{T}} - \boldsymbol{p}_{21}^{\mathsf{T}} \\ v_2 \boldsymbol{p}_{23}^{\mathsf{T}} - \boldsymbol{p}_{22}^{\mathsf{T}} \end{bmatrix} . \tag{6.6}$$

取 $\min_{\|\tilde{\boldsymbol{X}}\|=1} \|A\tilde{\boldsymbol{X}}\|^2$ 的最小右奇异向量。若解为 $(X_h, Y_h, Z_h, W)^{\mathsf{T}}$ 且 $W \neq 0$，欧氏坐标为 $(X_h/W, Y_h/W, Z_h/W)$。$W = 0$ 表示无穷远点。W 很小时，除法会放大误差，因此程序会对 W 设置阈值，并结合视线夹角筛选深度结果。

线性解可以作为非线性精化的初值，再最小化

$$\min_{\boldsymbol{X}} \sum_j \left\| \boldsymbol{x}_j - \pi\left(P_j \begin{bmatrix} \boldsymbol{X} \\ 1 \end{bmatrix} \right) \right\|^2 . \tag{6.7}$$

这里使用与 P_j 匹配的无畸变像素坐标；若直接使用原始观测，投影函数也应包含畸变。在像素噪声近似各向同性时，重投影误差比空间中点距离更贴近测量过程。

算法实现 | 三角测量

```
q1 = cv.undistortPoints(uv1.reshape(-1,1,2), K1, dist1)
q2 = cv.undistortPoints(uv2.reshape(-1,1,2), K2, dist2)
P1 = np.column_stack((R1, t1.reshape(3)))
P2 = np.column_stack((R2, t2.reshape(3)))
Xh = cv.triangulatePoints(P1, P2,
                          q1.reshape(-1,2).T, q2.reshape(-1,2).T)
valid = np.abs(Xh[3]) > 1e-10
X = (Xh[:3, valid] / Xh[3:4, valid]).T
```

输入像素坐标用浮点数组。`undistortPoints` 返回归一化相机坐标，与之配套的投影矩阵为 $[R \mid \boldsymbol{t}]$。计算后检查 $R_j\boldsymbol{X}+\boldsymbol{t}_j$ 的第三分量是否为正，并计算重投影误差。

视差与深度误差

考虑最简单的平行双目：左右相机焦距相同，主点已扣除，光心沿水平方向相距 B。对空间点 (X,Y,Z)，两幅图的横坐标为

$$u_L = f\frac{X}{Z}, \qquad u_R = f\frac{X-B}{Z}.$$

两式相减，把未知的横向位置 X 消去，得到视差 $d = u_L - u_R$ 和深度

$$d = \frac{fB}{Z}, \qquad Z = \frac{fB}{d}, \qquad |\delta Z| \approx \frac{Z^2}{fB}|\delta d|. \tag{6.8}$$

最后一式由对 d 求导得到。f 和 d 都用像素单位，B 的单位决定 Z 的单位。例如 $f=800$ 像素、$B=0.1$ 米，$Z=2$ 米时视差为 40 像素，一像素误差对应约 0.05 米深度误差；$Z=10$ 米时视差仅 8 像素，同样误差约对应 1.25 米。

数学直觉 | 基线长度

两条射线夹角很小时，轻微的方向误差就可能使交点移动很远。适当增大基线，可以增大视线夹角，改善深度测量；基线过大又容易增加遮挡，使对应更难寻找。相机绕同一光心旋转时，同一物体点的两次观测仍在同一条空间射线上。这种拍摄适合旋转全景拼接，测量深度则需要移动光心。

6.4 消失点标定

建筑物中的墙角和地面常提供三组互相垂直的方向。从照片中的平行线可以找到它们的消失点。如果选对内参矩阵 K，将这些消失点反投影后，得到的方向就应互相垂直。我们可以利用这个条件来求内参。

正交方向的约束

将世界坐标轴与三个正交方向对齐。第 i 个方向是单位基向量 $\boldsymbol{e}_i$，其消失点满足第二章的关系 $\tilde{\boldsymbol{v}}_i \sim KR\boldsymbol{e}_i$。左乘 K^{-1} 后，得到与旋转后的轴同方向的射线 $K^{-1}\tilde{\boldsymbol{v}}_i \sim R\boldsymbol{e}_i$。旋转保持夹角，因此对 $i \neq j$，

$$(R\boldsymbol{e}_i)^{\mathsf{T}}(R\boldsymbol{e}_j) = \boldsymbol{e}_i^{\mathsf{T}}R^{\mathsf{T}}R\boldsymbol{e}_j = 0.$$

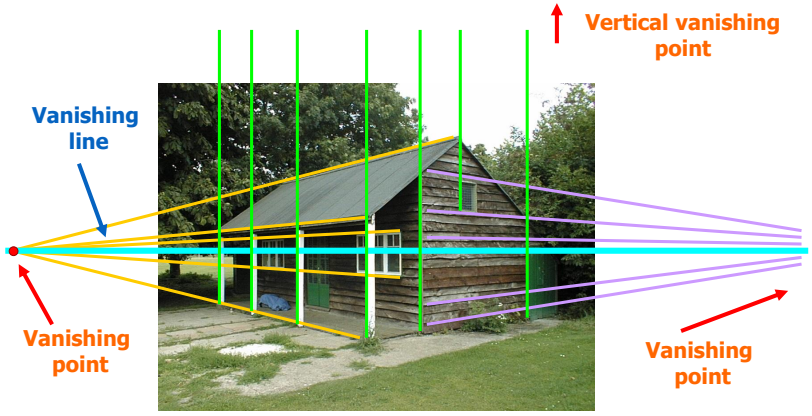


图 6.5 延长图像中的平行线，可以求出相应的消失点。图中的竖线保持平行，因此竖直方向的消失点位于无穷远。

未知的两个非零比例因子不影响“点积为零”，于是

$$\tilde{\boldsymbol{v}}_i^{\mathsf{T}}\omega\tilde{\boldsymbol{v}}_j = 0, \qquad \omega = K^{-\mathsf{T}}K^{-1}. \tag{6.9}$$

矩阵 ω 将两次乘以 K^{-1} 的运算合在一起。因此，上式可以直接用像素坐标表示反投影方向的正交条件，其中的未知量就是内参。

焦距与主点

为求出焦距和主点，这里采用**零偏斜、正方形像素**的假设，将内参写成

$$K = \begin{bmatrix} f & 0 & c_x \\ 0 & f & c_y \\ 0 & 0 & 1 \end{bmatrix}, \qquad f > 0.$$

若三个消失点都有限，把它们写成 $\tilde{\boldsymbol{v}}_i = (a_i, b_i, 1)^{\mathsf{T}}$。则 $K^{-1}\tilde{\boldsymbol{v}}_i = ((a_i - c_x)/f, (b_i - c_y)/f, 1)^{\mathsf{T}}$。将两条射线点积置零，再乘以 f^2，得到

$$(a_i - c_x)(a_j - c_x) + (b_i - c_y)(b_j - c_y) + f^2 = 0. \tag{6.10}$$

这里的平方项以同一个组合出现，可以将它们合记为一个未知量。令 $q = c_x^2 + c_y^2 + f^2$，展开后变成

$$(a_i + a_j)c_x + (b_i + b_j)c_y - q = a_i a_j + b_i b_j.$$

对 $(i, j) = (1,2), (1,3), (2,3)$ 各写一行，就能线性求出 (c_x, c_y, q)，再计算

$$f = \sqrt{q - c_x^2 - c_y^2}.$$

系数矩阵满秩、根号内为正时，便得到这三个内参。若条件不满足，可检查平行线是否归到正确的方向，以及所选方向是否确实正交。

数学直觉 ｜ 标定计算示例

设三个消失点为 $(1,0)$、$(-1,2)$、$(-1,-1)$。对应三条线性方程为 $2c_y-q=-1$、$-c_y-q=-1$、$-2c_x+c_y-q=-1$。先由前两式得 $c_y=0, q=1$，再得 $c_x=0$，故 $f=1$。反投影后的三个方向为 $(1,0,1)$、$(-1,2,1)$、$(-1,-1,1)$，两两点积确实为零。

无穷远消失点

为了同时处理有限点和无穷远点，保留齐次形式 $\tilde{\boldsymbol{v}}_i=(a_i,b_i,w_i)^{\mathsf{T}}$。正交约束统一写成

$$(a_i-c_xw_i)(a_j-c_xw_j)+(b_i-c_yw_i)(b_j-c_yw_j)+f^2w_iw_j=0. \tag{6.11}$$

若其中一个消失点在无穷远处，$w_i=0$，与它相关的方程就不再含 f。例如竖直无穷远点为 $(0,1,0)$，另两个有限点会落在同一条水平消失线上，与竖直方向的两个约束都只是在确定同一个 c_y。这时还剩 c_x 和 f 两个未知数，而剩余约束只有一个。若主点已知，就可以继续用这一方程求焦距。

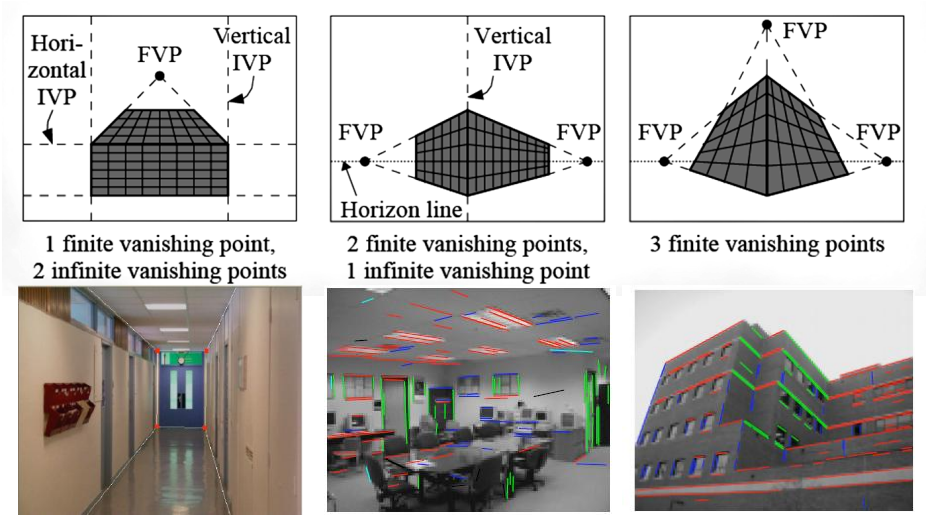


图 6.6 从左到右：两个、一个、零个消失点位于无穷远。在零偏斜、正方形像素假设下，三个有限且布局非退化的正交消失点可约束 f,c_x,c_y；出现无穷远点时，部分约束会失去独立性。

若两个正交消失点都在无穷远处，它们张成像平面方向，第三个方向沿光轴；其有限消失点就是主点。此时方程可确定主点位置，焦距还需要其他数据。如果主点已经可靠已知，则任意一对合适的有限正交消失点可由式 (6.10) 给出 f^2。

相机朝向

已知 K 后，每个旋转列的方向为 $K^{-1}\tilde{\boldsymbol{v}}_i$，所以

$$\boldsymbol{r}_i=\pm\frac{K^{-1}\tilde{\boldsymbol{v}}_i}{\|K^{-1}\tilde{\boldsymbol{v}}_i\|},\qquad R=[\boldsymbol{r}_1\ \boldsymbol{r}_2\ \boldsymbol{r}_3].$$

消失点描述的是无向轴，正负号需要结合所选世界轴的朝向、右手系与场景信息确定。有噪声时，三个单位向量未必严格正交。设初值 $R_0=U\Sigma V^{\mathsf{T}}$，用正交因子近似旋转，并修正最后一轴以排除镜面反射：

$$R=U\,\mathrm{diag}(1,1,\det(UV^{\mathsf{T}}))V^{\mathsf{T}}.$$

以上计算得到相机的朝向。消失点方程中的平移项已经消去，因此计算位置时需要另外加入位置观测。例如，已知世界原点的投影可以约束平移方向，再用已知长度或标定点确定距离。

第 7 章　对极几何与双视图重建

上一章用已知的相机参数计算空间点的位置。这一章从两幅照片出发，先找出相机之间的旋转和平移，再计算空间点。我们先来看，同一个物体点在两幅图中的位置有什么联系。

在左图中选中建筑物的一个角点，它在右图中会出现在哪里？左图的这个像素确定了一条从光心出发的射线，角点就在这条射线上。随着角点的深度变化，它在右图中的投影会沿一条直线移动。这条直线叫作*极线*。有了极线，我们就可以沿线寻找对应点。反过来，收集多组对应点，也能求出两台相机之间的关系。

7.1　极平面、极线与极点

记两台相机的光心为 O, O'，像平面为 π, π'。空间点 X 在两幅图中的投影分别为 x, x'。连接两个光心的直线 OO' 叫作*基线*。

先考虑 X 位于基线之外的情形。三个点 O, O', X 确定一个平面 Π，称为*极平面*。两条成像视线 OX 和 $O'X$ 都在这个平面内，所以它们与像平面的交点 x, x' 也在其中。这两个像点分别落在极平面与两个像平面的交线上：

$$l = \Pi \cap \pi, \qquad l' = \Pi \cap \pi'.$$

l, l' 就是一对极线。

基线同样在极平面中。它与两个像平面的交点记作 e, e'，称为*极点*。从左相机看，e 是右光心 O' 的投影；从右相机看，e' 是左光心 O 的投影。当空间点 X 改变位置时，极平面可以绕基线转动，像一本书的书页绕书脊翻动。每个极平面都包含基线，所以它在左图中的极线都经过 e，在右图中的极线都经过 e'。

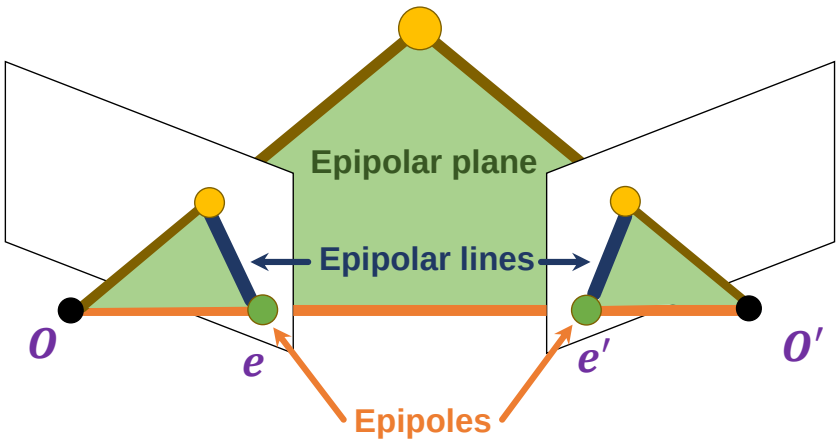


图 7.1　绿色区域是极平面，蓝色线段是它与两个像平面的交线。橙色基线与像平面相交于两个绿色极点。相机位置固定时，各图中的极线都通过各自的极点。

极线上的搜索

现在固定左像点 x。我们沿着从 O 经过 x 的射线，选取几个不同深度的空间点。只要这条射线与基线不同，它就和右光心 O' 确定一个极平面。刚才选取的空间点全在这个平面内，因此它们在右图中的投影也全在同一条极线 l' 上。

这说明，确定左像点以后，我们就可以把右图的搜索范围缩小到一条线。再比较线上的图像内容，例如角点附近的纹理，就能选择更合适的对应位置。

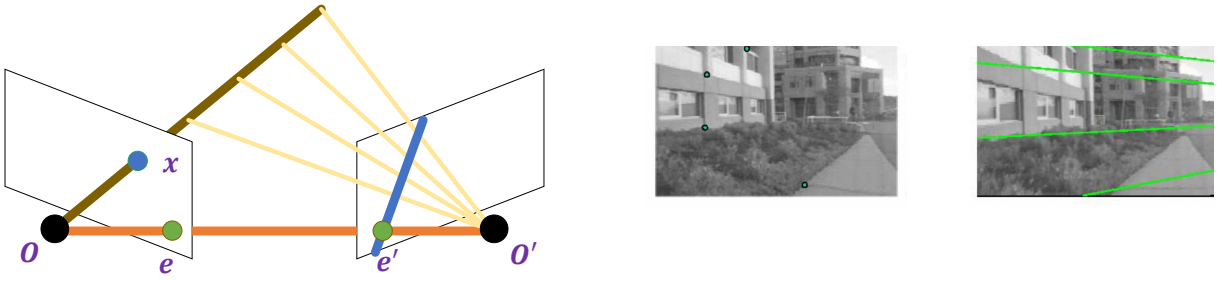


图 7.2 左：沿左视线选取不同深度的点，它们的右投影排在同一条极线上。右：左图中的几个标记点，各自对应右图中的一条绿色极线。

若已知物体的深度范围，还可以把搜索限制在极线的一段。例如，一段从两米到五米的空间射线，投到右图后也对应一段有限的搜索区间。这一过程还要求空间点同时位于两台相机前方，后面求位姿时会用到这个条件。

图像匹配

下图中的蓝、红像点恰好落在一对极线上，但它们分别来自两个不同的物体点。它们的视线会相交，交点却只是把这两个像点配成一对后算出来的位置。所以实际匹配时，我们既要看像点是否接近极线，也要比较两处的图像内容。

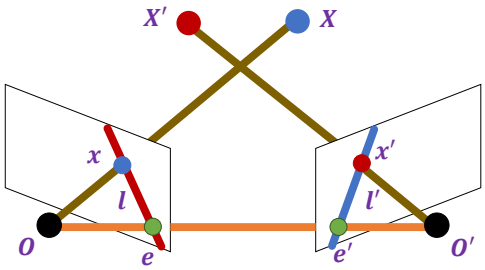


图 7.3 蓝、红像点分别来自场景中的 X, X'。两条视线在途中相交，把两个像点错误地配成一对，就会把交点当成待重建的物体点。

从几何上说，满足极线约束的两条视线位于同一个平面。对于其中两条不同、非平行的直线，可以求出它们的交点，再检查交点是否位于两台相机前方。若两条直线平行，则交点位于射影空间的无穷远处。拍摄和定位还会带来误差，实际的两条视线往往略微错开。因此，处理图像数据时，我们会计算像点偏离极线的距离，并允许一定误差。

7.2 本质矩阵

接下来把刚才的几何关系写成方程。推导从两个相机的成像方程开始，先保留空间点的深度，再用叉乘将深度消去。

坐标变换与深度

把第一台相机的坐标系作为世界坐标系。记空间点在左相机中的三维坐标为 $\boldsymbol{X}$，在右相机中的三维坐标为 $\boldsymbol{X}'$。两组坐标之间的变换，以及两台相机的投影矩阵分别为

$$\boldsymbol{X}' = R\boldsymbol{X} + \boldsymbol{t}, \qquad P = K[I \mid \boldsymbol{0}], \qquad P' = K'[R \mid \boldsymbol{t}]. \tag{7.1}$$

K, K' 是内参矩阵。R 将第一相机系中的方向转到第二相机系中，满足 $R^{\mathsf{T}}R = I$ 和 $\det R = 1$；$\boldsymbol{t}$ 是坐标变换中的平移项。

平移项的含义可以通过光心来理解。第一光心在自己的坐标系中是 $\boldsymbol{X} = \boldsymbol{0}$，代入后得到 $\boldsymbol{X}' = \boldsymbol{t}$。所以 $\boldsymbol{t}$ 就是第一光心在第二相机系中的坐标。若要描述第二光心在第一相机系中的位置，记它为 $\boldsymbol{C}'$。它在第二相机系中的坐标为零，因而 $R\boldsymbol{C}' + \boldsymbol{t} = 0$，解得 $\boldsymbol{C}' = -R^{\mathsf{T}}\boldsymbol{t}$。

记去除镜头畸变后的齐次像素坐标为 $\tilde{\boldsymbol{x}} = (u, v, 1)^{\mathsf{T}}$、$\tilde{\boldsymbol{x}}' = (u', v', 1)^{\mathsf{T}}$。乘以内参矩阵的逆，就得到归一化像平面上的坐标：

$$\boldsymbol{q} = K^{-1}\tilde{\boldsymbol{x}} = (a, b, 1)^{\mathsf{T}}, \qquad \boldsymbol{q}' = K'^{-1}\tilde{\boldsymbol{x}}' = (a', b', 1)^{\mathsf{T}}. \tag{7.2}$$

这两个向量分别给出了两条视线的方向。例如，$\boldsymbol{q} = (a, b, 1)^{\mathsf{T}}$ 对应的空间点可以写成 $\boldsymbol{X} = \lambda(a, b, 1)^{\mathsf{T}}$。它的第三个坐标正好是 λ，所以这里的参数 λ 就是沿光轴测量的深度 Z。相应地，右相机中的深度记作 $\lambda' = Z'$。

将两条视线的表示代入坐标变换，得到

$$\boldsymbol{X} = \lambda\boldsymbol{q}, \qquad \boldsymbol{X}' = \lambda'\boldsymbol{q}', \qquad \lambda'\boldsymbol{q}' = \lambda R\boldsymbol{q} + \boldsymbol{t}. \tag{7.3}$$

我们取相机前方为 $Z > 0$，因此同时被两台相机看见的点满足 $\lambda > 0$、$\lambda' > 0$。两台相机各自测量深度，式中保留了这两个参数。若要计算空间点到左光心的距离，则为 $\lambda\|\boldsymbol{q}\|$。

> **数学直觉 ｜ 深度与视差**
> 将式 (7.3)整体除以 λ，再取齐次比例关系，可得
>
> $$\boldsymbol{q}' \sim R\boldsymbol{q} + \frac{\boldsymbol{t}}{\lambda}.$$
>
> 固定左像点时，$R\boldsymbol{q}$ 保持不变。改变深度 λ，就会改变平移项所占的比例，右图中的投影也随之移动。点越远，$\boldsymbol{t}/\lambda$ 越小，投影越接近方向 $R\boldsymbol{q}$。这就是两台相机之间的平移产生视差的方式。

共面条件

在第二相机坐标系中，$\boldsymbol{t}$ 从右光心 O' 指向左光心 O，$R\boldsymbol{q}$ 沿左视线，$\boldsymbol{q}'$ 沿右视线。这三个向量都在极平面内。若 $\boldsymbol{t}$ 和 $R\boldsymbol{q}$ 不平行，它们的叉乘 $\boldsymbol{t} \times (R\boldsymbol{q})$ 垂直于极平面，因此也垂直于 $\boldsymbol{q}'$。

同样的结论也能从成像方程算出来。先用 $\boldsymbol{t}$ 从左侧与式 (7.3)叉乘。平移项变成 $\boldsymbol{t} \times \boldsymbol{t} = 0$，剩下

$$\lambda'(\boldsymbol{t} \times \boldsymbol{q}') = \lambda(\boldsymbol{t} \times R\boldsymbol{q}).$$

再与 $\boldsymbol{q}'$ 作点积。左侧的叉乘与 $\boldsymbol{q}'$ 垂直，所以

$$0 = \lambda\,\boldsymbol{q}'^{\mathsf{T}}(\boldsymbol{t} \times R\boldsymbol{q}).$$

由于 $\lambda > 0$，两侧除以 λ，得到

$$\boldsymbol{q}'^{\mathsf{T}}(\boldsymbol{t} \times R\boldsymbol{q}) = 0. \tag{7.4}$$

式中两个深度都已消去，留下了两条成像方向与相机运动之间的关系。左侧是三个向量的混合积，其绝对值等于它们张成的平行六面体体积。体积为零，正好对应前面看到的共面关系。

叉乘矩阵

对固定的 $\boldsymbol{t} = (t_x, t_y, t_z)^{\mathsf{T}}$，定义

$$[\boldsymbol{t}]_\times = \begin{bmatrix} 0 & -t_z & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix}.$$

将这个矩阵乘以任意向量 $\boldsymbol{v}$，结果就是 $\boldsymbol{t} \times \boldsymbol{v}$。因此可以把式 (7.4)写成

$$\boldsymbol{q}'^{\mathsf{T}} E \boldsymbol{q} = 0, \qquad E = [\boldsymbol{t}]_\times R. \tag{7.5}$$

E 称为本质矩阵。

给定左像点 $\boldsymbol{q}$，先计算 $E\boldsymbol{q} = (\alpha, \beta, \gamma)^{\mathsf{T}}$。右像点 $\boldsymbol{q}' = (a', b', 1)^{\mathsf{T}}$ 就要满足 $\alpha a' + \beta b' + \gamma = 0$。这是一条直线的方程，所以 $E\boldsymbol{q}$ **给出了右极线的三个系数**。以后即使空间点的深度未知，也可以直接用它算出对应的极线。

> **数学直觉 ｜ 极平面与极线**
>
> $E\boldsymbol{q} = \boldsymbol{t} \times (R\boldsymbol{q})$ 是极平面的法向量，记作 $\boldsymbol{n}'$。在第二相机坐标系中，这个平面经过原点，方程为 $\boldsymbol{n}'^{\mathsf{T}}\boldsymbol{X}' = 0$。归一化像平面位于 $Z' = 1$。将 $\boldsymbol{X}' = (a', b', 1)^{\mathsf{T}}$ 代入平面方程，就得到两平面的交线。这解释了为什么同一组三个系数，既能表示空间中的平面法向量，又能表示图像中的极线。

秩与奇异值

假设两光心不同，即 $\boldsymbol{t} \neq 0$。与 $\boldsymbol{t}$ 平行的向量经过 $[\boldsymbol{t}]_\times$ 后变成零；与它垂直的非零向量经过叉乘后仍有非零长度。因此 $[\boldsymbol{t}]_\times$ 的零空间恰为 $\operatorname{span}(\boldsymbol{t})$，维数为一，矩阵的秩为二。$R$ 可逆，所以 $E = [\boldsymbol{t}]_\times R$ 的秩也为二。

为了求奇异值，我们计算 EE^{T}。由 $RR^{\mathsf{T}} = I$，有 $EE^{\mathsf{T}} = [\boldsymbol{t}]_\times [\boldsymbol{t}]_\times^{\mathsf{T}}$。对任意向量 $\boldsymbol{v}$，向量三重积公式给出

$$[\boldsymbol{t}]_\times [\boldsymbol{t}]_\times^{\mathsf{T}} \boldsymbol{v} = -\boldsymbol{t} \times (\boldsymbol{t} \times \boldsymbol{v}) = \|\boldsymbol{t}\|^2 \boldsymbol{v} - \boldsymbol{t}(\boldsymbol{t}^{\mathsf{T}}\boldsymbol{v}).$$

由于这对所有 $\boldsymbol{v}$ 都成立，得到矩阵等式

$$EE^{\mathsf{T}} = \|\boldsymbol{t}\|^2 I - \boldsymbol{t}\boldsymbol{t}^{\mathsf{T}}. \tag{7.6}$$

右侧乘以 $\boldsymbol{t}$ 得到零，乘以任意 $\boldsymbol{v} \perp \boldsymbol{t}$ 得到 $\|\boldsymbol{t}\|^2 \boldsymbol{v}$。因此 EE^{T} 有一个零特征值和两个等于 $\|\boldsymbol{t}\|^2$ 的特征值。取平方根，就得到 E 的三个奇异值：

$$\|\boldsymbol{t}\|, \qquad \|\boldsymbol{t}\|, \qquad 0.$$

本质矩阵的两个非零奇异值相等。后面从带噪声的数据估计 E 时，我们会用到这个性质。

式 (7.5)的右侧为零，所以将 E 整体乘以非零数，仍然得到同一条约束。这意味着图像中的对应只能确定 E 到一个非零倍数。旋转有三个自由度；平移的长度留在这个未知倍数中，方向有两个自由度。因此，用于表示对极约束的本质矩阵共有五个自由度。

7.3 基础矩阵

本质矩阵的方程使用归一化像平面坐标。实际做特征匹配时，我们首先得到的是像素坐标 $\tilde{\boldsymbol{x}}, \tilde{\boldsymbol{x}}'$。把两种坐标之间的关系代入，就能写出像素坐标下的对极方程：

$$0 = (K'^{-1}\tilde{\boldsymbol{x}}')^{\mathsf{T}} E (K^{-1}\tilde{\boldsymbol{x}}) = \tilde{\boldsymbol{x}}'^{\mathsf{T}} K'^{-\mathsf{T}} E K^{-1} \tilde{\boldsymbol{x}}.$$

将中间三个矩阵的乘积记作 F，得到

$$F = K'^{-\mathsf{T}} E K^{-1}, \qquad \tilde{\boldsymbol{x}}'^{\mathsf{T}} F \tilde{\boldsymbol{x}} = 0. \tag{7.7}$$

F 叫作基础矩阵。以后即使内参未知，也可以用像素对应直接估计这个矩阵。

由于 K, K' 可逆，F 与 E 的秩相同，都是二。内参矩阵会改变奇异值，所以基础矩阵的两个非零奇异值可以不同。它的九个元素有一个任意的整体倍数，另外还要满足 $\det F = 0$。在秩为二时，这个行列式条件提供一个独立约束，因此 F 有七个自由度。

极线方程

先固定左像点，把式 (7.7)看成关于右像点的方程。再固定右像点，把它看成关于左像点的方程，就分别得到

$$\boldsymbol{l}' = F\tilde{\boldsymbol{x}}, \qquad \boldsymbol{l} = F^{\mathsf{T}}\tilde{\boldsymbol{x}}'. \tag{7.8}$$

例如，若算出 $\boldsymbol{l}' = (2, -1, 3)^{\mathsf{T}}$，右极线就是 $2u' - v' + 3 = 0$。直线的三个系数整体乘以非零数，仍表示同一条线。交换两幅图的顺序，相应使用 F^{T}。

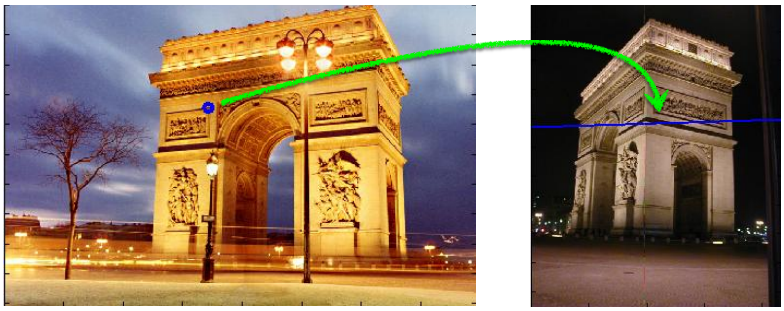


图 7.4 左图选定一个像点后，计算 $F\tilde{\boldsymbol{x}}$，就得到右图中的蓝色极线。接下来沿这条线比较图像内容，寻找同一个物体点。

极点坐标

左光心在右相机中的坐标为 $\boldsymbol{t}$，右光心在左相机中的坐标为 $-R^{\mathsf{T}}\boldsymbol{t}$。将它们分别投影到像平面，可得

$$\boldsymbol{e} \sim K(-R^{\mathsf{T}}\boldsymbol{t}), \qquad \boldsymbol{e}' \sim K'\boldsymbol{t}.$$

将这两式代入基础矩阵的表达式，利用 $[\boldsymbol{t}]_\times \boldsymbol{t} = 0$ 和 $[\boldsymbol{t}]_\times^{\mathsf{T}} \boldsymbol{t} = 0$，得到

$$F\boldsymbol{e} = \boldsymbol{0}, \qquad F^{\mathsf{T}}\boldsymbol{e}' = \boldsymbol{0}. \tag{7.9}$$

因此，求极点可以归结为求矩阵的零空间。设 $F = U\,\mathrm{diag}(\sigma_1, \sigma_2, 0)V^{\mathsf{T}}$，$V$ 的第三列 $\boldsymbol{v}_3$ 满足 $F\boldsymbol{v}_3 = 0$，可作为左极点的齐次坐标；U 的第三列 $\boldsymbol{u}_3$ 满足 $F^{\mathsf{T}}\boldsymbol{u}_3 = 0$，可作为右极点的齐次坐标。第三个分量非零时，除以它就得到像素坐标；第三个分量为零时，极点位于无穷远。

这个计算也能说明极线为什么共点。对任意左像点，右极线 $\boldsymbol{l}' = F\tilde{\boldsymbol{x}}$ 都满足 $\boldsymbol{e}'^{\mathsf{T}}\boldsymbol{l}' = \boldsymbol{e}'^{\mathsf{T}}F\tilde{\boldsymbol{x}} = 0$。也就是说，每条右极线都经过右极点。

左像点恰好等于 $\boldsymbol{e}$ 时，计算结果为 $F\boldsymbol{e} = 0$。此时左视线就是基线，所有极平面都包含这条视线，因此所有右极线都有可能。这也是零向量在这里的含义：它的三个系数都为零，无法指定一条直线。

> **备注 ｜ 单应矩阵与对极矩阵**
>
> 第五章的单应矩阵 H 将左图的点映射到右图的点，适用于平面场景或相机绕同一光心旋转等情形。本章的 F 将左像点映射为右极线，适用于具有不同深度的空间点。E 则在归一化相机坐标中表示极线关系。已知内参时，两者可以通过 $E = K'^{\mathsf{T}}FK$ 转换，进一步求相机的旋转和平移。

7.4 三种常见的相机位置

极点是另一个光心的投影。因此，相机的位置和朝向决定了极点的位置，也决定了极线的排列方式。下面看三个具体例子。

相机朝向彼此

两台相机朝向彼此时，基线通常会与像平面相交于有限位置，极点也就位于有限位置。各条极线都通过这个交点。若交点落在照片范围之外，我们看到的是从画面外延伸进来的若干条线。当交点离画面很远时，这些线在照片中看起来会接近平行。

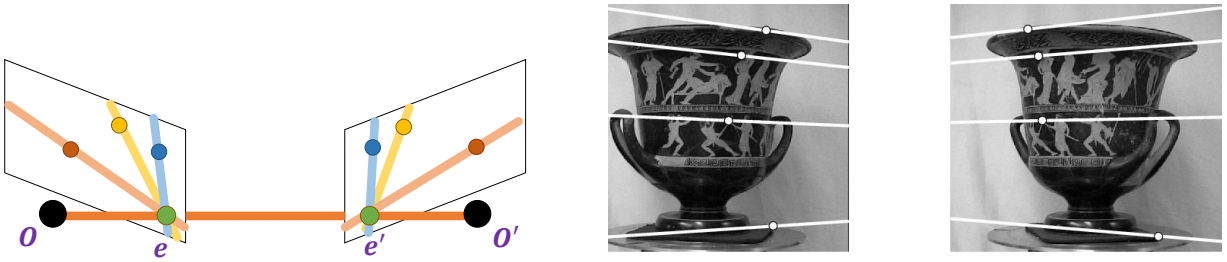


图 7.5 左：两相机朝向彼此，各图中的极线汇聚于极点。右：同一个花瓶的两个视图，白色线段标出了几组极线。

水平双目

设两台相机朝向相同，右光心位于左相机坐标系中的 $(B, 0, 0)^{\mathsf{T}}$，其中 $B > 0$ 是基线长度。这时 $R = I$、$\boldsymbol{t} = (-B, 0, 0)^{\mathsf{T}}$。代入 $E = [\boldsymbol{t}]_\times R$，得到

$$E = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & B \\ 0 & -B & 0 \end{bmatrix}, \qquad E\boldsymbol{q} = (0, B, -Bb)^{\mathsf{T}}.$$

右像点满足 $\boldsymbol{q}'^{\mathsf{T}} E \boldsymbol{q} = B(b' - b) = 0$，所以 $b' = b$。也就是说，在归一化像平面中，两侧像点的纵坐标相同，匹配只需沿水平方向搜索。这些水平极线的公共方向是 $(1, 0, 0)^{\mathsf{T}}$，因此极点位于无穷远。

若两相机有相同的纵向焦距和主点纵坐标，$b' = b$ 还会给出像素坐标中的 $v' = v$。实际双目系统也可以通过极线校正，把图像变换到便于按行匹配的坐标中。

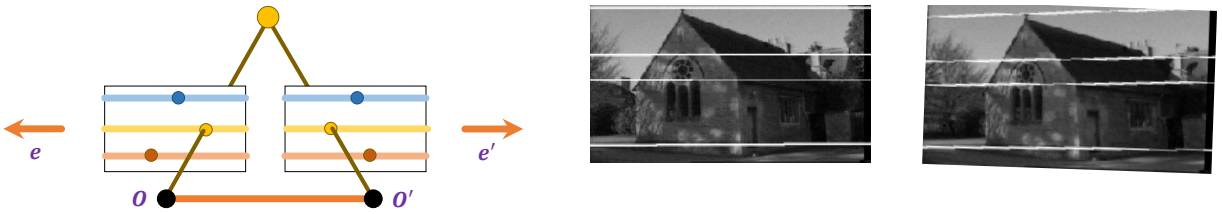


图 7.6 基线平行于两个像平面时，极点位于无穷远，各图中的极线互相平行。标准水平双目可以沿图像的每一行寻找对应。

纵坐标相同以后，横坐标之差就提供了深度。对空间点 $(X, Y, Z)^{\mathsf{T}}$，两图的归一化横坐标为 $a = X/Z$、$a' = (X - B)/Z$。相减得

$$a - a' = \frac{B}{Z}, \qquad Z = \frac{B}{a - a'}.$$

这就是第六章视差公式在归一化坐标中的写法。

例如，取 $B = 0.2$ 米，左像点为 $\boldsymbol{q} = (0.3, 0.1, 1)^{\mathsf{T}}$。若右像点为 $\boldsymbol{q}' = (0.2, 0.1, 1)^{\mathsf{T}}$，则 $Z = 0.2/(0.3 - 0.2) = 2$ 米，进而有 $\boldsymbol{X} = Z\boldsymbol{q} = (0.6, 0.2, 2)^{\mathsf{T}}$。若右横坐标改为 $a' = 0.25$，求得的深度就是 4 米。这两个右像点在同一条极线上，给出的深度却不同。**选定极线上的具体匹配位置以后，才能通过视差计算深度**。

另外两种取值也有明确的几何含义。若 $a' = 0.4$，求得 $Z = -2$ 米，交点位于相机后方；若 $a' = a$，两条视线平行，对应无穷远点。

沿光轴前进

设两相机的朝向相同，右光心在左相机坐标系中为 $(0, 0, B)^{\mathsf{T}}$。这时 $R = I$、$\boldsymbol{t} = (0, 0, -B)^{\mathsf{T}}$，右极点为 $\boldsymbol{e}' \sim K'\boldsymbol{t} \sim (c'_x, c'_y, 1)^{\mathsf{T}}$，正好位于主点。左图同理。将平移代入对极方程，得到

$$a'b - b'a = 0.$$

因此 (a, b) 与 (a', b') 位于同一条过原点的直线上。换回像素坐标后，极线就从主点向四周展开。在主点附近，沿光轴前进造成的图像位移很小，深度测量也更容易受到定位误差的影响。

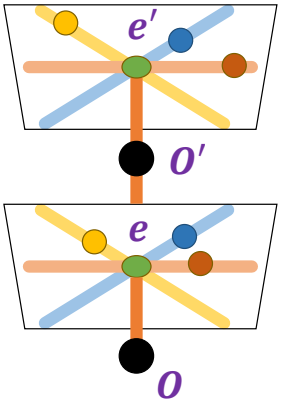


图 7.7 相机沿共同光轴移动时，极点位于主点。图中每种颜色表示一对极线，它们都从各自的主点向外展开。

7.5 八点法

前面用相机的旋转、平移和内参写出了 F。现在来看怎样从图像中求它。假设已经找到了若干对正确的匹配点，每对点都应满足 $\tilde{\boldsymbol{x}}'^{\mathsf{T}} F \tilde{\boldsymbol{x}} = 0$。把这些方程放在一起，就可以求 F 的九个元素。

建立线性方程

将 F 按行展开成一个向量：$\boldsymbol{f} = (f_{11}, f_{12}, f_{13}, f_{21}, \ldots, f_{33})^{\mathsf{T}}$。对一对像素 $(u, v) \leftrightarrow (u', v')$，展开对极方程得

$$u'(f_{11}u + f_{12}v + f_{13}) + v'(f_{21}u + f_{22}v + f_{23}) + (f_{31}u + f_{32}v + f_{33}) = 0.$$

将各个 f_{jk} 的系数依次排列，就得到设计矩阵的一行：

$$A_i = \begin{bmatrix} u'u & u'v & u' & v'u & v'v & v' & u & v & 1 \end{bmatrix}, \qquad A_i \boldsymbol{f} = 0. \tag{7.10}$$

n 对点给出 n 行，合起来是 $A\boldsymbol{f} = 0$，其中 $A \in \mathbb{R}^{n \times 9}$。求这个齐次方程时，$\boldsymbol{f}$ 的整体倍数可以任意选取。因此，只要有八条独立方程，就能将解限制在一个一维零空间内。这就是八点法所需的条件：对应点至少有八对，且在无噪声时使 $\operatorname{rank}(A) = 8$。

实际测量有误差。对应点多于八对时，方程通常无法同时精确成立，我们便求一个让总残差尽量小的向量：

$$\min_{\|\boldsymbol{f}\| = 1} \|A\boldsymbol{f}\|^2.$$

$\|\boldsymbol{f}\| = 1$ 为解选定了一个尺度，也排除了全零解。按第五章式 (5.4)的推导，取 A 的最小右奇异向量即可。将它重新排成 3×3 矩阵，就得到了 F 的线性估计。

这里暂时只使用了线性方程，后面还会加入 F 的秩二条件。若一开始就利用行列式约束，也可以采用七点法：七对点留下二维线性解空间，代入 $\det F = 0$ 后求解一个三次方程，可能得到多个候选结果。

坐标归一化

像素坐标可能有几千，而 A 中还包含坐标的乘积和常数 1。例如 $u = u' = 1000$ 时，乘积列中的数是 10^6。这些量级相差很大的列会让求解更容易受数值误差影响。因此，我们先把点移到原点附近，再统一缩放。

与第五章一样，对左图的点求质心 $\bar{\boldsymbol{p}} = (\bar{u}, \bar{v})^{\mathsf{T}}$，再求它们到质心的平均距离 $\bar{r}$：

$$\bar{\boldsymbol{p}} = \frac{1}{n}\sum_{i}(u_i, v_i)^{\mathsf{T}}, \qquad \bar{r} = \frac{1}{n}\sum_{i}\|(u_i, v_i)^{\mathsf{T}} - \bar{\boldsymbol{p}}\|.$$

取 $s = \sqrt{2}/\bar{r}$，用下面的变换将质心移到原点，并将平均距离变为 $\sqrt{2}$：

$$T = \begin{bmatrix} s & 0 & -s\bar{u} \\ 0 & s & -s\bar{v} \\ 0 & 0 & 1 \end{bmatrix}.$$

右图的点也分别求质心和缩放系数，得到另一个矩阵 T'。然后用 $T\tilde{\boldsymbol{x}}_i$、$T'\tilde{\boldsymbol{x}}'_i$ 建立 A，求出归一化坐标中的初值 $\widehat{F}_n$。

这里的归一化服务于数值计算，T, T' 根据这批匹配点的位置求得。前面用 K^{-1}, K'^{-1} 做的相机归一化，则根据焦距和主点将像素还原为视线方向。两者各有明确的用途。

秩二约束

线性估计得到了一个矩阵初值，接下来让它满足基础矩阵的秩二条件。设初值的 SVD 为

$$\widehat{F}_n = U\,\mathrm{diag}(\sigma_1, \sigma_2, \sigma_3)V^{\mathsf{T}}, \qquad \sigma_1 \geq \sigma_2 \geq \sigma_3 \geq 0.$$

将最小奇异值置零，得到

$$F_n = U\,\mathrm{diag}(\sigma_1, \sigma_2, 0)V^{\mathsf{T}}.$$

当 $\sigma_2 > 0$ 时，这个矩阵的秩恰为二。而且，在所有秩至多为二的矩阵中，它与初值的 Frobenius 距离最近。

下面证明这个结论。任取一个秩至多为二的矩阵 G。它有非零零空间，因此可以取一个单位向量 $\boldsymbol{v}$，使 $G\boldsymbol{v} = 0$。矩阵的 Frobenius 范数不小于它作用在任意单位向量上得到的长度，于是

$$\|\widehat{F}_n - G\|_F \geq \|(\widehat{F}_n - G)\boldsymbol{v}\| = \|\widehat{F}_n\boldsymbol{v}\| \geq \sigma_3.$$

最后一步用了最小奇异值的定义。另一方面，$\widehat{F}_n - F_n = \sigma_3 \boldsymbol{u}_3 \boldsymbol{v}_3^{\mathsf{T}}$，其 Frobenius 范数恰为 σ_3。所以 F_n 达到了所有候选矩阵都必须满足的下界，也就完成了证明。

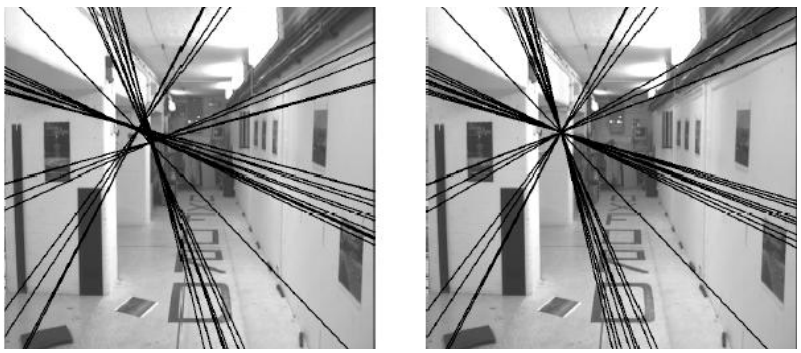


图 7.8 左：线性初值算出的极线略微错开。右：将最小奇异值置零后，极线通过同一个极点。

这一步选择了对矩阵元素改动最小的秩二矩阵。得到它以后，还可以进一步调整矩阵，使像点到极线的距离减小。后一种误差更接近我们在图像中测量位置的方式，下一节会具体讨论。

还原像素坐标

现在的 F_n 适用于经过 T, T' 变换的点。要将它用于原图中的像素，直接把坐标变换代回方程：

$$(T'\tilde{\boldsymbol{x}}')^{\mathsf{T}} F_n (T\tilde{\boldsymbol{x}}) = 0 \quad \Longrightarrow \quad \tilde{\boldsymbol{x}}'^{\mathsf{T}} (T'^{\mathsf{T}} F_n T)\tilde{\boldsymbol{x}} = 0.$$

所以原始像素坐标下的基础矩阵为

$$F = T'^{\mathsf{T}} F_n T. \tag{7.11}$$

右侧的 T 先变换左像点，左侧的 T'^{T} 来自对右像点的转置。T, T' 都可逆，所以这个变换也保留了矩阵的秩。

> **算法实现 ｜ 归一化八点法**
>
> `pts1_n`，`pts2_n` 是归一化后的 $N \times 2$ 点数组，`T1`，`T2` 分别是 T, T'。这里取 $N \geq 8$，并假设点的平均距离非零、对应点提供足够的独立约束。

```
A = []
for (u, v), (up, vp) in zip(pts1_n, pts2_n):
    A.append([up*u, up*v, up, vp*u, vp*v, vp, u, v, 1])
_, _, Vt = np.linalg.svd(np.asarray(A), full_matrices=True)
Fn = Vt[-1].reshape(3, 3)
U, s, Vt = np.linalg.svd(Fn)
s[-1] = 0
Fn = U @ np.diag(s) @ Vt
F = T2.T @ Fn @ T1
F /= np.linalg.norm(F)
```

> 恰好八对点时，A 是 8×9 矩阵。`full_matrices=True` 会保留全部九个右奇异向量，其中第九个向量正是这里要取的零空间方向。

7.6 误差与错误匹配

实际检测到的角点位置会有偏差，特征匹配也可能把两个不同的物体点配成一对。我们先计算像点偏离极线的程度，再用这个误差筛选匹配。

点到极线的距离

对第 i 对点，记 $r_i = \tilde{\boldsymbol{x}}_i'^{\mathsf{T}} F \tilde{\boldsymbol{x}}_i$，右极线为 $\boldsymbol{l}_i' = F\tilde{\boldsymbol{x}}_i = (\alpha_i', \beta_i', \gamma_i')^{\mathsf{T}}$。$r_i$ 就是将右像点代入直线方程得到的数。为了把它变成像素距离，我们将直线的法向量 $(\alpha_i', \beta_i')^{\mathsf{T}}$ 归一化，得到

$$d_i' = \frac{|r_i|}{\sqrt{\alpha_i'^2 + \beta_i'^2}}, \qquad d_i = \frac{|r_i|}{\sqrt{\alpha_i^2 + \beta_i^2}}, \qquad \boldsymbol{l}_i = F^{\mathsf{T}}\tilde{\boldsymbol{x}}_i'. \tag{7.12}$$

第二个式子同样计算左像点到左极线的距离，两个分母都要求非零。直线系数整体乘以某个非零数时，分子、分母会一起缩放，距离保持不变。在去畸变后的像素坐标中，d_i, d_i' 的单位都是像素。将两侧的平方距离相加，就得到对称极线误差 $d_i^2 + d_i'^2$。

同时修正两侧像点

两幅图中的位置都有测量误差，因此还可以让两个像点一起移动，用总移动量最小的方式，使它们满足对极方程。记原始像素为 $\boldsymbol{p}_i = (u_i, v_i)^\mathsf{T}$、$\boldsymbol{p}'_i = (u'_i, v'_i)^\mathsf{T}$，修正后的位置为 $\widehat{\boldsymbol{p}}_i, \widehat{\boldsymbol{p}}'_i$。给修正后的点添上第三个分量 1，得到齐次坐标 $\widehat{\widehat{\boldsymbol{x}}}_i, \widehat{\widehat{\boldsymbol{x}}}'_i$。

如果同时调整 F 和所有对应点，便可求解

$$\min_{F,\{\widehat{\boldsymbol{p}}_i,\widehat{\boldsymbol{p}}'_i\}} \sum_i \left(\|\widehat{\boldsymbol{p}}_i - \boldsymbol{p}_i\|^2 + \|\widehat{\boldsymbol{p}}'_i - \boldsymbol{p}'_i\|^2 \right),$$

并要求

$$\widehat{\widehat{\boldsymbol{x}}}_i'^\mathsf{T} F \widehat{\widehat{\boldsymbol{x}}}_i = 0, \qquad \operatorname{rank}(F) = 2, \qquad \|F\|_F = 1.$$

最后一个条件为 F 选定尺度。这个目标把两幅图中的定位误差都考虑进来。移动左像点时，它在右图中对应的极线也会改变，因此两侧的修正需要一起求解。

Sampson 误差

在误差较小时，可以先将对极方程作一阶展开，近似计算上面的最小移动量。把一对像点的四个坐标排成 $\boldsymbol{z} = (u, v, u', v')^\mathsf{T}$，记对极方程的左侧为 $g(\boldsymbol{z}) = r$。设左、右极线系数分别为 $(\alpha, \beta, \gamma)^\mathsf{T}$ 和 $(\alpha', \beta', \gamma')^\mathsf{T}$，对四个坐标求偏导得

$$\nabla g = (\alpha, \beta, \alpha', \beta')^\mathsf{T}.$$

例如，对 u 求导得到 $(F^\mathsf{T}\tilde{\boldsymbol{x}}')_1 = \alpha$，对 u' 求导得到 $(F\tilde{\boldsymbol{x}})_1 = \alpha'$。

给四个坐标加上小改变量 $\delta\boldsymbol{z}$，一阶近似为

$$g(\boldsymbol{z} + \delta\boldsymbol{z}) \approx r + \nabla g^\mathsf{T} \delta\boldsymbol{z}.$$

要让修正后的点满足方程，我们要求右侧为零。假设四个坐标的误差同样重要，就希望 $\|\delta\boldsymbol{z}\|^2$ 尽量小。由柯西不等式，满足该条件的改变量必有 $|r| \le \|\nabla g\| \|\delta\boldsymbol{z}\|$。当

$$\delta\boldsymbol{z} = -\frac{r}{\|\nabla g\|^2} \nabla g$$

时取到等号，因此最小平方移动量为

$$d_S^2 = \frac{r^2}{\alpha^2 + \beta^2 + \alpha'^2 + \beta'^2}. \tag{7.13}$$

这就是 Sampson 误差。它通过一次一阶计算，近似表示两侧像点共同修正所需的平方距离。使用这个近似时，需要改变量较小，且梯度长度明显大于零。当分母接近零时，可以改用前面的非线性修正，或将该点留待进一步检查。

RANSAC

现在处理错误匹配。沿用第五章的方法，每次抽取一小组对应点估计 F，再用所有对应点检查这个结果。若采用八点法，每次抽取八对能够提供独立约束的点，完成归一化、线性求解和秩二修正。然后计算其余点的误差，把低于阈值的点记为内点。例如，采用 Sampson 误差时可以用 $d_S^2 < \tau^2$ 作为内点条件。

重复采样后，选出内点较多、误差较小的结果。接着**用全部内点重新估计** F，让更多正确对应参与计算。还可以根据更新后的矩阵重新筛选内点，最后用上一部分的几何误差继续优化。

算法实现 | 用 RANSAC 估计基础矩阵

uv1，uv2 是 $N \times 2$ 浮点数组，已经去畸变并保留像素坐标。

```
F, mask = cv.findFundamentalMat(
    uv1, uv2, cv.FM_RANSAC, 1.0, 0.999, 5000)
if F is None or F.shape != (3, 3) or mask is None:
    raise ValueError("No valid fundamental matrix")
inlier = mask.ravel().astype(bool)
```

这里的阈值 1.0 以像素距离计量，可以随角点定位精度调整。函数内部的采样求解器由 OpenCV 实现决定；上面的八点法推导也可以用于自行实现 RANSAC。得到结果以后，检查内点是否分布在图像的不同区域，以及剩余误差主要出现在哪里。

纯旋转

有些拍摄方式会让求解出现多种解释，首先看相机绕同一光心旋转的情形。此时 $\boldsymbol{t} = 0$，所以 $E = 0$。由成像方程可得 $\tilde{\boldsymbol{x}}' \sim K' R K^{-1} \tilde{\boldsymbol{x}}$，两幅图之间是一个旋转单应。

对于同一物体点，两次观测从相同的光心出发，沿着同一条空间射线。改变点在射线上的深度，两幅图中的投影都保持不变。因此，这种拍摄可以用来估计旋转；测量深度还需要移动光心，取得带平移的另一幅图像。

平面场景

若所有对应点都来自同一平面，两图之间满足某个可逆单应 H。这时有一族不同的基础矩阵，都能满足这些对应点的方程。下面直接构造它们。

任取一个非零向量 $\boldsymbol{e}'$，令

$$F_{\boldsymbol{e}'} = [\boldsymbol{e}']_{\times} H.$$

$[\boldsymbol{e}']_{\times}$ 的秩为二，H 可逆，所以 $F_{\boldsymbol{e}'}$ 的秩也为二。对平面上的每个对应点，可以写成 $\tilde{\boldsymbol{x}}' = c_i H \tilde{\boldsymbol{x}}$，其中 $c_i \neq 0$。于是

$$\tilde{\boldsymbol{x}}'^{\mathsf{T}} F_{\boldsymbol{e}'} \tilde{\boldsymbol{x}} = c_i (H\tilde{\boldsymbol{x}})^{\mathsf{T}} [\boldsymbol{e}']_{\times} (H\tilde{\boldsymbol{x}}) = 0.$$

最后一个等号来自 $\boldsymbol{y}^{\mathsf{T}} [\boldsymbol{e}']_{\times} \boldsymbol{y} = 0$：$\boldsymbol{e}' \times \boldsymbol{y}$ 与 $\boldsymbol{y}$ 垂直。

选择不同方向的 $\boldsymbol{e}'$，就得到了不同的基础矩阵，而所有平面对应都满足它们。因此，要唯一估计一般场景的 F，需要有平面之外的对应点提供更多约束。若已知内参，也可以利用单应分解来求平面场景中的相机运动，并进一步选择候选解。

接近平面的点集、很小的基线，也会让估计对定位误差更敏感。采集图像时，让匹配点覆盖不同深度和图像位置，并使相机有适当的平移，通常有助于改善估计。计算时则可以检查 A 的奇异值，以及三角测量中两条视线的夹角。

7.7 相机位姿

现在假设两台相机的内参已知，也拍到了有足够视差的对应点。我们可以先估计 F，再通过 $E = K'^{\mathsf{T}} F K$ 得到本质矩阵；也可以先把像素变成归一化相机坐标，直接估计 E。下面从 E 中求出旋转 R 和平移方向。

修正奇异值

带有测量误差的估计值 $\widehat{E}$ 通常只有近似相等的前两个奇异值。根据式 (7.6)，我们把它们调整为相等，并将第三个奇异值设为零。设 $\widehat{E} = U\,\mathrm{diag}(\sigma_1, \sigma_2, \sigma_3)V^{\mathsf{T}}$，修正后写成

$$E_* = U\,\mathrm{diag}(s, s, 0)V^{\mathsf{T}}.$$

保持 U, V 不变时，矩阵改动的 Frobenius 范数平方为 $(\sigma_1 - s)^2 + (\sigma_2 - s)^2 + \sigma_3^2$。对 s 求导并令其为零，得到 $s = (\sigma_1 + \sigma_2)/2$。所以将前两个奇异值替换为它们的平均值，就能以最小的改动完成这一步。

下面取 $s > 0$。利用 E 的整体倍数可以任意选取这一性质，将 E_* 除以 s，得到 $E = UDV^{\mathsf{T}}$，其中 $D = \mathrm{diag}(1, 1, 0)$。这个选择相当于把基线长度暂时定为一。我们会先在这个长度单位下求平移和三维点，最后再说明怎样换成实际长度。

平移方向

从 $E = [\boldsymbol{t}]_\times R$ 出发，用 $\boldsymbol{t}^{\mathsf{T}}$ 左乘，有

$$\boldsymbol{t}^{\mathsf{T}}E = \boldsymbol{t}^{\mathsf{T}}[\boldsymbol{t}]_\times R = 0.$$

因此 $\boldsymbol{t}$ 在 E 的左零空间中。对于 $E = UDV^{\mathsf{T}}$，U 的第三列 $\boldsymbol{u}_3$ 正好满足 $E^{\mathsf{T}}\boldsymbol{u}_3 = 0$。将平移长度定为一以后，便得到两个方向候选：

$$\widehat{\boldsymbol{t}} = \pm\boldsymbol{u}_3.$$

两个符号都需要保留，因为 E 和 $-E$ 给出相同的对极方程。后面会通过空间点在相机前方还是后方来选择符号。

旋转矩阵

先把 SVD 中的 U, V 都调整为行列式等于一的正交矩阵。若某个矩阵的行列式为负，就将它的第三列乘以 -1。由于 D 的第三个对角元为零，这个调整保留了乘积 UDV^{T}。调整以后，U, V 本身也都是旋转矩阵。

令 $\boldsymbol{k} = (0, 0, 1)^{\mathsf{T}}$，并定义

$$J = [\boldsymbol{k}]_\times = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \qquad W = \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

W 表示绕第三轴旋转 $-90°$。直接做矩阵乘法，可以验证 $JW = D$、$JW^{\mathsf{T}} = -D$。

先取平移方向 $\widehat{\boldsymbol{t}} = \boldsymbol{u}_3 = U\boldsymbol{k}$。旋转保持叉乘关系，所以 $[U\boldsymbol{k}]_\times = U[\boldsymbol{k}]_\times U^{\mathsf{T}} = UJU^{\mathsf{T}}$。我们要找一个 R，使它与这个叉乘矩阵相乘后得到 E 或 $-E$。利用刚才得到的 $JW = D$ 和 $JW^{\mathsf{T}} = -D$，可以分别算出

$$[\widehat{\boldsymbol{t}}]_\times(UWV^{\mathsf{T}}) = UJWV^{\mathsf{T}} = UDV^{\mathsf{T}} = E,$$
$$[\widehat{\boldsymbol{t}}]_\times(UW^{\mathsf{T}}V^{\mathsf{T}}) = UJW^{\mathsf{T}}V^{\mathsf{T}} = -UDV^{\mathsf{T}} = -E.$$

于是有两个旋转候选 $R_1 = UWV^{\mathsf{T}}$、$R_2 = UW^{\mathsf{T}}V^{\mathsf{T}}$。它们都是旋转矩阵，因为各个因子都正交且行列式为一。再加上平移方向的两个符号，一共得到四组：

$$R_1 = UWV^{\mathsf{T}}, \qquad R_2 = UW^{\mathsf{T}}V^{\mathsf{T}}, \qquad (R, \widehat{\boldsymbol{t}}) \in \{(R_1, \pm\boldsymbol{u}_3), (R_2, \pm\boldsymbol{u}_3)\}. \tag{7.14}$$

还可以验证，这已经包含了全部旋转候选。固定 $\widehat{\boldsymbol{t}} = \boldsymbol{u}_3$，将未知旋转写成 $R = UQV^{\mathsf{T}}$，其中 Q 也必须是旋转矩阵。方程 $[\widehat{\boldsymbol{t}}]_{\times} R = E$ 随即变成 $JQ = D$。按行相乘可知，Q 的第一行必须为 $(0, 1, 0)$，第二行必须为 $(-1, 0, 0)$。第三行要与前两行正交，且让行列式等于一，因此为 $(0, 0, 1)$。这样就得到了 $Q = W$。同样，从 $JQ = -D$ 得到 $Q = W^{\mathsf{T}}$。

正深度条件

要从四组结果中选出实际相机的位姿，我们把图像中的点还原到三维，检查它们是否位于两台相机前方。

对每个候选，取归一化相机矩阵 $[I \mid 0]$ 和 $[R \mid \widehat{\boldsymbol{t}}]$，用第六章式 (6.6)对同一组内点作三角测量。得到欧氏坐标 $\boldsymbol{X}_i$ 后，分别取两个相机坐标系中的第三个分量：

$$z_i = (\boldsymbol{X}_i)_3, \qquad z_i' = (R\boldsymbol{X}_i + \widehat{\boldsymbol{t}})_3. \tag{7.15}$$

$z_i > 0$、$z_i' > 0$ 表示这个点在两台相机前方。在无噪声、具有有限交点且不发生退化的情形下，这个条件通常能选出唯一候选。实际计算时会统计多个内点的正深度结果，同时检查重投影误差和视差大小。若满足条件的点很少，就需要重新检查匹配，或增加有更明显视差的图像。

用一个水平双目例子来看四组结果。取单位基线，左、右归一化像点为 $\boldsymbol{q} = (0.3, 0.1, 1)^{\mathsf{T}}$、$\boldsymbol{q}' = (0.2, 0.1, 1)^{\mathsf{T}}$。实际的相机关系为 $R = I$、$\boldsymbol{t} = (-1, 0, 0)^{\mathsf{T}}$。另一个旋转候选是 $R_h = \mathrm{diag}(1, -1, -1)$，即绕第一轴转 180°。

对每组候选求解成像方程 $\lambda' \boldsymbol{q}' = \lambda R\boldsymbol{q} + \boldsymbol{t}$，会得到下表中的深度。

旋转	平移	两侧深度 (λ, λ')	点的位置
I	$(-1, 0, 0)^{\mathsf{T}}$	$(10, 10)$	两侧均在前方
I	$(1, 0, 0)^{\mathsf{T}}$	$(-10, -10)$	两侧均在后方
R_h	$(-1, 0, 0)^{\mathsf{T}}$	$(2, -2)$	右侧在后方
R_h	$(1, 0, 0)^{\mathsf{T}}$	$(-2, 2)$	左侧在后方

例如，对第三行，成像方程的第三个分量给出 $\lambda' = -\lambda$，第一个分量给出 $0.2\lambda' = 0.3\lambda - 1$。将前一式代入后一式，解得 $\lambda = 2$、$\lambda' = -2$。按照这一组相机位姿还原的点位于右相机后方。表中只有第一行让两侧深度同时为正，因此我们选取第一组位姿。

算法实现 | 估计两相机的位姿

先去除两幅图各自的畸变，并转成归一化相机坐标，然后估计 E。

```
q1 = cv.undistortPoints(uv1.reshape(-1,1,2), K1, dist1)
q2 = cv.undistortPoints(uv2.reshape(-1,1,2), K2, dist2)
q1, q2 = q1.reshape(-1,2), q2.reshape(-1,2)
E, mask = cv.findEssentialMat(
    q1, q2, np.eye(3), method=cv.RANSAC,
    prob=0.999, threshold=tau_n)
if E is None or E.shape != (3, 3) or mask is None:
    raise ValueError("No single essential matrix")
n_ok, R, t, pose_mask = cv.recoverPose(
    E, q1, q2, np.eye(3), mask=mask.copy())
```

`tau_n` 使用归一化坐标的单位。若各方向焦距都接近 f，且像素误差在各方向大小相近，可以先用像素容差除以 f 来设置它。焦距差异较大时，应按各方向的误差大小调整阈值。内参已经在前两行处理过，因此后续函数中的相机矩阵取 I。

`findEssentialMat` 可以在 RANSAC 中使用五点法求解本质矩阵。`recoverPose` 则对候选位姿作三角测量和正深度检查，返回旋转 R 和单位平移方向 $\boldsymbol{t}$。通过检查的对应记录在 `pose_mask` 中。选出这些点，再用第六章的三角测量方法计算三维坐标即可。

实际长度

上面的计算把基线长度选成了一。要将结果换成米或厘米，还需要一个实际测得的长度，例如两光心的距离。这个额外长度为何必要，可以从成像方程看出来。

假设 $R, \boldsymbol{t}, \boldsymbol{X}_i$ 已经满足观测。对任意 $c > 0$，将平移和所有三维点同时乘以 c，有

$$R(c\boldsymbol{X}_i) + c\boldsymbol{t} = c(R\boldsymbol{X}_i + \boldsymbol{t}).$$

整个相机坐标向量只变了一个倍数，透视除法后的像素位置保持不变。左相机的投影也是如此。因此，所有这样缩放得到的相机和场景，都能产生相同的两幅图像。

若测得实际基线长度为 B，就把单位基线下求出的平移和全部三维坐标一起乘以 B。例如，基线实际长 0.2 米时，前面表中第一组解的深度 10 就对应 2 米。

第 8 章　双视图立体匹配

上一章介绍了两台相机之间的几何关系，也说明了怎样用一对对应点求空间位置。这一章把问题扩展到整幅图像：为尽可能多的像素找到对应点，再逐点计算深度。

拍摄同一个场景时，近处的物体在左右两幅图中的位置变化较大，远处的物体变化较小。如果已知两台相机的内参、相对旋转和平移，就可以利用这种位置差计算深度。把各像素的深度排回图像中的位置，得到的二维数组称为*深度图*。它描述从这个视角能看到的物体表面。将深度图中的像素反投影到三维空间，就能得到一片点云。

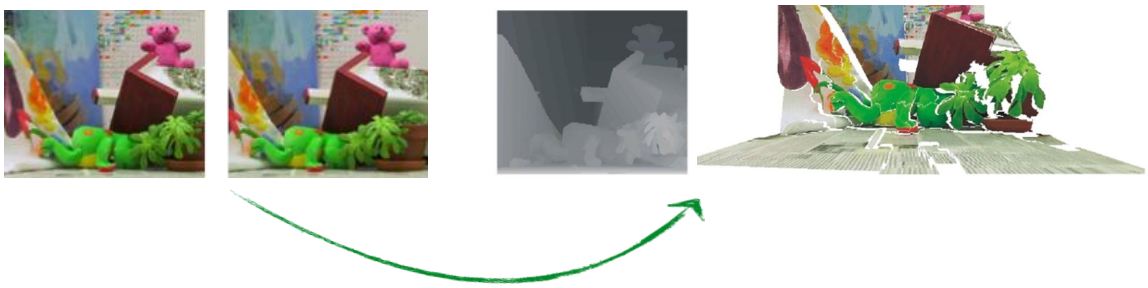


图 8.1　从左到右依次是两幅输入图像、深度图和重建的表面。左右图中都能看见的区域，可以通过匹配与三角测量求深度。

本章先假设场景静止、两台相机已经标定，图像中的镜头畸变已经去除。我们先推导最简单的双目成像关系，再说明怎样把一般的双目图像变换到这种形式。之后的主要工作是寻找对应点：从比较两个小窗口开始，再把相邻像素的关系和遮挡情况加入计算。

8.1　视差与深度

平行双目相机

以左相机坐标系作为世界坐标系，令右光心位于 $(B,0,0)^{\mathsf{T}}$，其中 $B>0$ 是两光心之间的距离，称为*基线长度*。两台相机的坐标轴方向相同，光轴均沿 Z 轴。空间点在左相机中的坐标记为 $\boldsymbol{X}=(X,Y,Z)^{\mathsf{T}}$，则它在右相机中的坐标为

$$\boldsymbol{X}'=(X-B,Y,Z)^{\mathsf{T}}.$$

因此，沿用上一章的约定 $\boldsymbol{X}'=R\boldsymbol{X}+\boldsymbol{t}$，这里有 $R=I$、$\boldsymbol{t}=(-B,0,0)^{\mathsf{T}}$。

先取两台相机相同的内参：水平方向和竖直方向的像素焦距分别为 f_x,f_y，主点为 (c_x,c_y)，像素轴之间没有斜切。透视投影给出

$$\begin{aligned} u_L &= f_x\frac{X}{Z}+c_x, & v_L &= f_y\frac{Y}{Z}+c_y,\\ u_R &= f_x\frac{X-B}{Z}+c_x, & v_R &= f_y\frac{Y}{Z}+c_y. \end{aligned} \tag{8.1}$$

两式中的竖直坐标相同，所以对应点位于同一行。固定左图中的一个像素后，只需要沿右图的这一行搜索。

这个结果也可以从本质矩阵得到。此时 $E=[\boldsymbol{t}]_\times$，将内参归一化后的方向 $\boldsymbol{q}_L=(a_L,b_L,1)^\top$、$\boldsymbol{q}_R=(a_R,b_R,1)^\top$ 代入极线约束，有

$$E=\begin{bmatrix}0&0&0\\0&0&B\\0&-B&0\end{bmatrix},\qquad \boldsymbol{q}_R^\top E\boldsymbol{q}_L=B(b_R-b_L)=0.$$

于是 $b_R=b_L$。由于两幅图使用相同的 f_y,c_y，像素坐标也满足 $v_R=v_L$。

深度公式

定义左图像素的*视差*为两个水平像素坐标之差：

$$d=u_L-u_R. \tag{8.2}$$

将式 (8.1)的两项相减，主点和 X 项分别抵消，得到

$$d=\frac{f_xB}{Z},\qquad \boxed{Z=\frac{f_xB}{d}}. \tag{8.3}$$

在当前约定下，有限的前方空间点满足 $d>0$。视差随深度增大而减小；当深度趋向无穷大时，视差趋向零。这里 f_x 与 d 都以像素为单位，所以 Z 与 B 的长度单位相同。若基线长度用米表示，计算出的深度也以米表示。

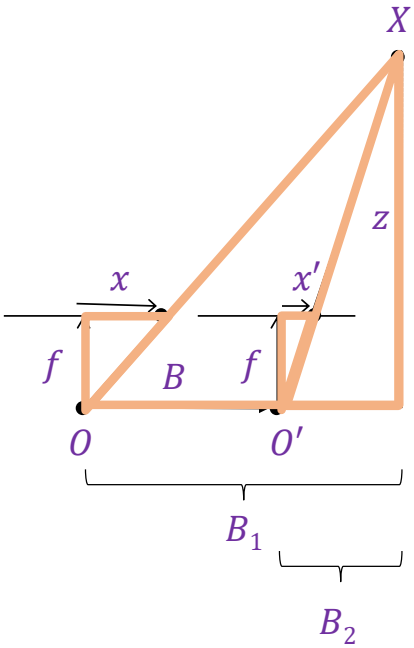


图 8.2 平行双目的三角测量。课件图中的 x,x' 是相对各自主点的水平坐标，z 对应正文的 Z。两条视线的水平位置差随深度变化，图中两个相似三角形给出同一个深度公式。

求出深度后，继续利用左图的成像方程，便可恢复三维坐标：

$$X=\frac{(u_L-c_x)Z}{f_x},\qquad Y=\frac{(v_L-c_y)Z}{f_y},\qquad \boldsymbol{X}=(X,Y,Z)^\top. \tag{8.4}$$

Z 是沿光轴测量的深度，空间点到光心的直线距离为 $\sqrt{X^2+Y^2+Z^2}$。若标定或其他测量给出了实际的基线长度，三维坐标就有相应的物理尺度。若只从本质矩阵恢复平移方向，则需要另外测量一个长度来确定尺度。

> **备注 ｜ 两幅图的主点**
> 校正后的两幅图可以具有相同的焦距，却保留不同的水平主点 c_{xL}, c_{xR}。这时直接相减得到
>
> $$d = \frac{f_x B}{Z} + c_{xL} - c_{xR}, \qquad Z = \frac{f_x B}{d - (c_{xL} - c_{xR})}.$$
>
> 分母是扣除主点偏移后的视差。下面讲匹配时采用两幅图主点相同的设置；实际计算则使用校正后投影矩阵中的参数。

深度误差与基线

设 $f_x = 800$ 像素，$B = 0.12$ 米。深度为 4 米的点产生 24 像素的视差，深度为 8 米的点只产生 12 像素的视差。同样大小的像素定位误差，对远处物体的深度影响会更大。

要算出这种影响，对式 (8.3)求导：

$$\frac{\partial Z}{\partial d} = -\frac{f_x B}{d^2} = -\frac{Z^2}{f_x B}.$$

当视差误差相对较小时，一阶近似给出

$$|\Delta Z| \approx \frac{Z^2}{f_x B}|\Delta d|, \qquad \frac{|\Delta Z|}{Z} \approx \frac{|\Delta d|}{d}. \tag{8.5}$$

在刚才的参数下，0.5 像素的视差误差在 4 米处约对应 0.083 米的深度误差，在 8 米处约对应 0.33 米。这一估计只考虑视差误差，焦距和基线的标定误差还会带来额外影响。

图 8.3 左侧基线较长，两条视线的夹角较大；右侧基线较短，夹角较小。视线方向有相近的偏差时，右侧交点沿深度方向的变化更明显。

在焦距、物体深度和视差定位误差相同的条件下，增大基线可以减小深度误差。同时，两个视角看到的物体外观会有更大差别，互相遮挡的区域也往往增多，匹配因此更困难。实际选择基线时，需要结合目标距离、图像分辨率和两相机的共同视野。

8.2 立体校正

真实的两台相机通常存在少量旋转，左右图中的极线也就可能倾斜。*立体校正*为两幅图分别选择一个新的像平面，并重新采样图像，使对应点具有相同的行坐标。这相当于保持两个光心的位置，给它们选择相同的观察方向和合适的内参。

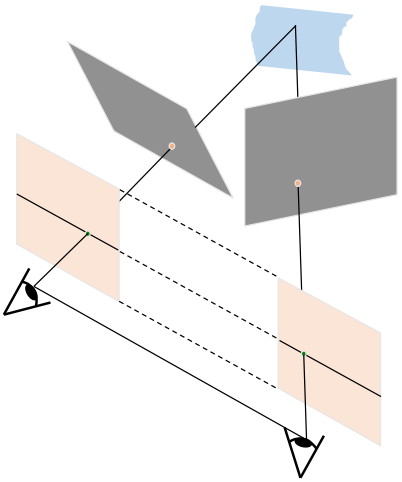


图 8.4 灰色区域表示原来的像平面，浅橙色区域表示校正后的像平面。新像平面彼此平行，并且都与基线平行，因此极线沿水平方向延伸。

共同的相机坐标轴

继续采用 $\boldsymbol{X}' = R\boldsymbol{X} + \boldsymbol{t}$，并假设 $K_L, K_R, R, \boldsymbol{t}$ 已知。右光心在左相机坐标系中的位置是 $\boldsymbol{C}_R = -R^{\mathsf{T}}\boldsymbol{t}$。令 $B = \|\boldsymbol{C}_R\| > 0$，把新相机的水平轴选为从左光心指向右光心的方向：

$$\boldsymbol{r}_1 = \frac{\boldsymbol{C}_R}{B}.$$

为了让新相机仍然朝向原来的场景，再选一个大致沿原光轴的参考方向 $\boldsymbol{a}$。例如，当两相机都朝向前方时，可以从左光轴 $\boldsymbol{a} = (0,0,1)^{\mathsf{T}}$ 开始。将它在基线方向上的分量去掉，再归一化，得到新的光轴：

$$\boldsymbol{r}_3 = \frac{\boldsymbol{a} - (\boldsymbol{a}^{\mathsf{T}}\boldsymbol{r}_1)\boldsymbol{r}_1}{\|\boldsymbol{a} - (\boldsymbol{a}^{\mathsf{T}}\boldsymbol{r}_1)\boldsymbol{r}_1\|}, \qquad \boldsymbol{r}_2 = \boldsymbol{r}_3 \times \boldsymbol{r}_1, \qquad R_c = \begin{bmatrix} \boldsymbol{r}_1^{\mathsf{T}} \\ \boldsymbol{r}_2^{\mathsf{T}} \\ \boldsymbol{r}_3^{\mathsf{T}} \end{bmatrix}. \tag{8.6}$$

$\boldsymbol{r}_1, \boldsymbol{r}_2, \boldsymbol{r}_3$ 是在左相机坐标系中写出的三个新坐标轴。它们两两垂直、长度为一，且 $\boldsymbol{r}_1 \times \boldsymbol{r}_2 = \boldsymbol{r}_3$，所以 R_c 是一个旋转矩阵。乘以 R_c，就把左相机坐标改写到这组新坐标轴下。

式 (8.6)要求 $\boldsymbol{a}$ 与基线不平行。若二者接近平行，可以换一个参考方向；这类拍摄布局在校正时往往也会产生较大的图像拉伸。新光轴的朝向还应使要重建的共同可见区域位于新相机前方。

图像变换

记新内参为 K_c，并让左右两幅图使用相同的 K_c。左图的齐次像素 $\tilde{\boldsymbol{x}}_L$ 对应方向 $K_L^{-1}\tilde{\boldsymbol{x}}_L$。先用 R_c 转到新坐标轴，再用 K_c 投影，得到

$$\tilde{\boldsymbol{x}}_{Lc} \sim K_c R_c K_L^{-1} \tilde{\boldsymbol{x}}_L.$$

右图方向最初写在右相机系中，需要先乘 R^{T} 转到左相机轴向，再乘 R_c。因此两个图像变换矩阵为

$$H_L = K_c R_c K_L^{-1}, \qquad H_R = K_c R_c R^{\mathsf{T}} K_R^{-1}. \tag{8.7}$$

它们都是 3×3 单应矩阵。这里每幅图都绕自己的光心改变观察方向，一条视线上的点始终投到同一个新像素，因此变换对不同深度的场景点都适用。

接着检查变换后的相机关系。左、右新相机中的三维坐标分别为

$$\boldsymbol{X}_{Lc} = R_c\boldsymbol{X}, \qquad \boldsymbol{X}_{Rc} = R_cR^{\mathsf{T}}\boldsymbol{X}' = R_c\boldsymbol{X} + R_cR^{\mathsf{T}}\boldsymbol{t}.$$

由 $R^{\mathsf{T}}\boldsymbol{t} = -\boldsymbol{C}_R$ 和 $R_c\boldsymbol{C}_R = (B,0,0)^{\mathsf{T}}$，得到

$$\boldsymbol{X}_{Rc} = \boldsymbol{X}_{Lc} + (-B,0,0)^{\mathsf{T}}. \tag{8.8}$$

这正是上一节的平行双目形式。两台新相机的旋转相同，平移只沿水平轴，又使用同一组内参，所以对应点的行坐标相同。两个极点位于水平方向的无穷远处，图中的极线便成为一组平行横线。

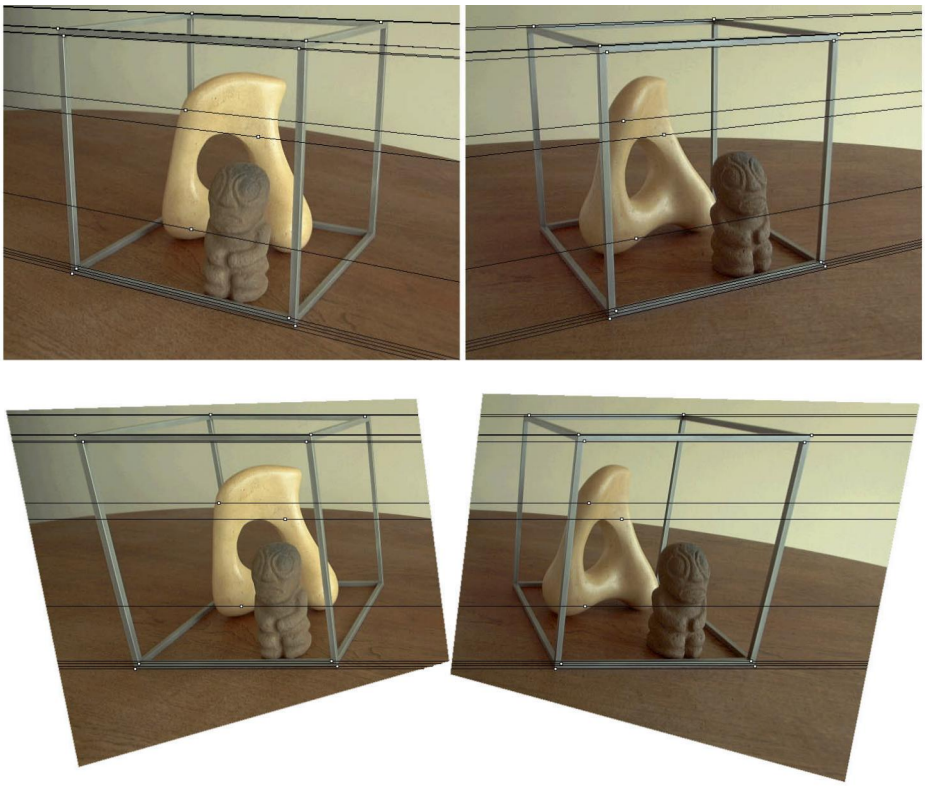


图 8.5 上：原始双目图像。下：校正后的双目图像。校正改变了图像边界和物体的投影形状，使左右对应点落在同一水平线上。

采样与有效区域

求得 H_L, H_R 后，可以使用第五章介绍的逆向映射：遍历新图像的像素，映回原图，并通过插值取得颜色。镜头畸变也可以合并到这一步的采样映射中处理。校正后有些像素会落在原图范围之外，这些位置应保留为无效区域。匹配窗口需要同时落在两幅图的有效区域内。

新焦距、主点和图像尺寸决定了保留多少视野，以及周围留下多少空白。选定它们以后，计算深度要使用新的投影参数。如果之后又对图像缩放或裁剪，也要相应调整焦距、主点和视差的像素单位。

只有基础矩阵时，也能求出使极线对齐的射影校正变换。上面具有明确三维旋转含义的构造还用到了内参和相对位姿；要把结果换算成有物理尺度的深度，则进一步需要基线长度。

8.3 局部立体匹配

完成校正以后，左图位置 (u, v) 在右图中的候选位置可以写为 $(u-d, v)$。于是寻找对应点，就变成了为每个像素选择一个视差 d。设候选集合为整数视差 $\mathcal{D} = \{d_{\min}, \dots, d_{\max}\}$。若已知场景深度约在 $[Z_{\min}, Z_{\max}]$ 内，便可用 $[f_x B/Z_{\max}, f_x B/Z_{\min}]$ 估计所需的视差范围，并留出适当的误差余量。

窗口代价

单个像素只提供一个灰度值，许多不同位置都可能有相同灰度。在像素附近取一个 $k \times k$ 窗口，可以同时比较纹理的形状。将左窗口和候选右窗口依次展开，得到向量 $\boldsymbol{w}_L, \boldsymbol{w}_R(d) \in \mathbb{R}^{k^2}$。平方差之和（SSD）定义为

$$C_{\mathrm{SSD}}(u, v, d) = \|\boldsymbol{w}_L - \boldsymbol{w}_R(d)\|^2 = \sum_{(s,t)\in W} \big[I_L(u+s, v+t) - I_R(u-d+s, v+t)\big]^2. \tag{8.9}$$

W 是窗口内相对中心的偏移集合，I_L, I_R 是左右图的灰度。代价越小，两个窗口的灰度排列越接近。

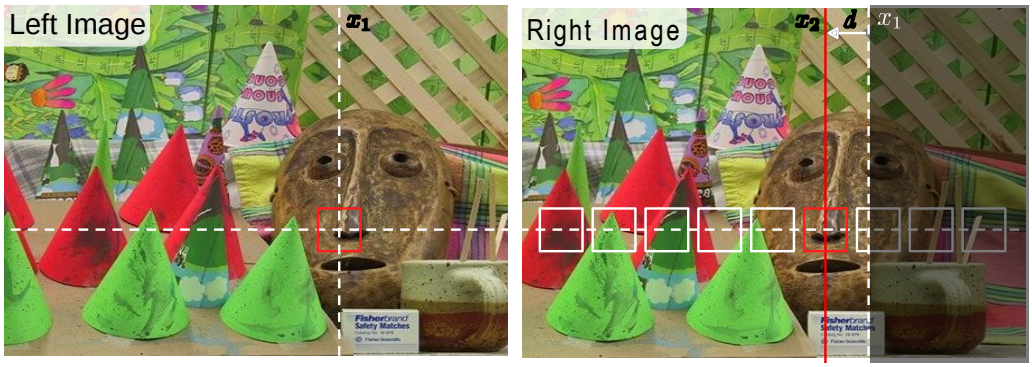


图 8.6 固定左图窗口，在右图的同一行移动候选窗口。每移动到一个位置，就计算一次匹配代价；横向移动的距离对应一个候选视差。

如果两幅图的曝光不同，可以先减去窗口均值，再比较归一化后的向量。记 $\bar{w}_L, \bar{w}_R$ 为窗口灰度均值，$\mathbf{1}$ 为全一向量，令 $\boldsymbol{a} = \boldsymbol{w}_L - \bar{w}_L\mathbf{1}$、$\boldsymbol{b} = \boldsymbol{w}_R - \bar{w}_R\mathbf{1}$。零均值归一化互相关（ZNCC）为

$$\rho(u, v, d) = \frac{\boldsymbol{a}^{\top}\boldsymbol{b}}{\|\boldsymbol{a}\|\,\|\boldsymbol{b}\|}, \qquad C_{\mathrm{ZNCC}} = 1 - \rho. \tag{8.10}$$

当两个窗口都有灰度变化时，$\rho \in [-1, 1]$，数值越大表示越相似。若 $\boldsymbol{w}_R = \alpha\boldsymbol{w}_L + \beta\mathbf{1}$ 且 $\alpha > 0$，减均值会消去 β，归一化又会消去 α，因此得到 $\rho = 1$。这解释了 ZNCC 对局部亮度平移和正比例变化的适应性。常量窗口的分母为零；实际实现可以将低方差窗口标为低置信度，并用小量稳定除法。

将每个位置、每个候选视差的代价存起来，就得到一个形如“图像高度 × 图像宽度 × 候选数”的数组，称为*代价体*。最直接的局部方法逐像素选择代价最小的候选：

$$d_L(u, v) = \arg\min_{d\in\mathcal{D}} C(u, v, d). \tag{8.11}$$

若只有一个候选明显优于其他候选，匹配就较明确；若几个候选的代价很接近，还需要邻域信息来帮助选择。

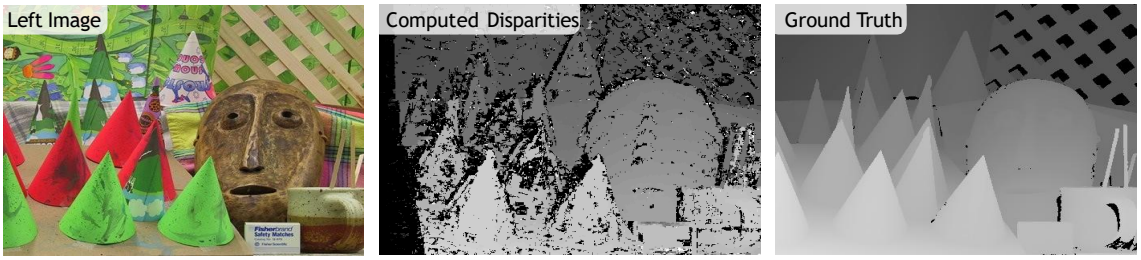


图 8.7 左：输入图像。中：各窗口独立选择得到的视差图。右：参考视差。物体轮廓已经可见，纹理不足和遮挡区域中仍有较多错误。

窗口大小与图像纹理

窗口计算默认邻域内的像素可以用同一个视差对齐。对于近似正对相机的小表面，这个假设通常较好；对于斜面，视差会逐渐变化；对于物体边缘，窗口还可能同时包含前景和背景。

较大的窗口能比较更多纹理，随机灰度噪声对总分的影响相对减小。但它也更容易跨过深度边界，使前景的视差向背景扩散。较小的窗口能保留细窄物体和边缘细节，同时需要局部有足够的纹理来确定位置。

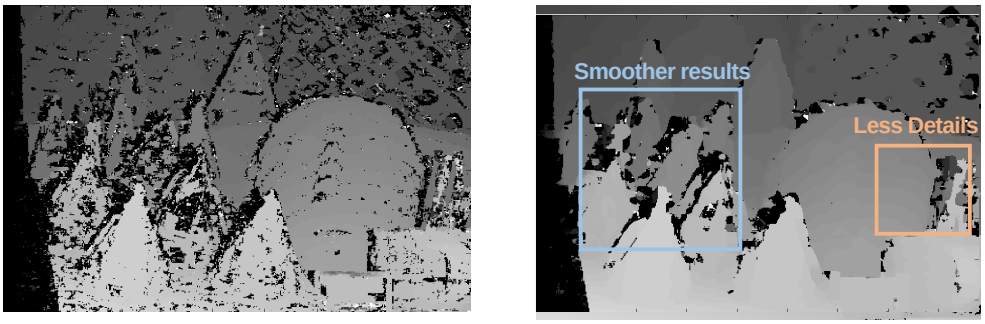


图 8.8 左侧使用 5×5 窗口，右侧使用 11×11 窗口。右图的平坦区域更平滑，部分细节和边界也被抹平。

无纹理区域中，窗口移动一段距离后仍然很相似，代价曲线会比较平。重复纹理则会产生多个低代价位置，例如相邻的栅栏条可能被互相配对。高光还会随观察方向移动，使同一物体点在左右图中呈现不同的亮度。这些现象分别对应信息不足、多个相似候选和外观随视角变化。

亚像素视差

真实对应点经常落在两个像素之间。在代价曲线有一个清楚的局部最小值时，可以用最小值附近的三个采样点拟合抛物线。记整数最优视差为 d_0，三个代价为 $c_- = C(d_0 - 1)$、$c_0 = C(d_0)$、$c_+ = C(d_0 + 1)$。将局部曲线写成 $a\delta^2 + b\delta + c_0$，代入 $\delta = -1, 1$ 得到

$$a = \frac{c_- + c_+ - 2c_0}{2}, \qquad b = \frac{c_+ - c_-}{2}.$$

抛物线的最低点位于 $\delta = -b/(2a)$，所以

$$d_{\text{sub}} = d_0 + \frac{c_- - c_+}{2(c_- - 2c_0 + c_+)}. \tag{8.12}$$

这一拟合要求 d_0 的两侧都有有效候选，且局部曲率为正、数值足够稳定。在平坦或多峰的代价曲线上，应同时保留较低的匹配置信度。

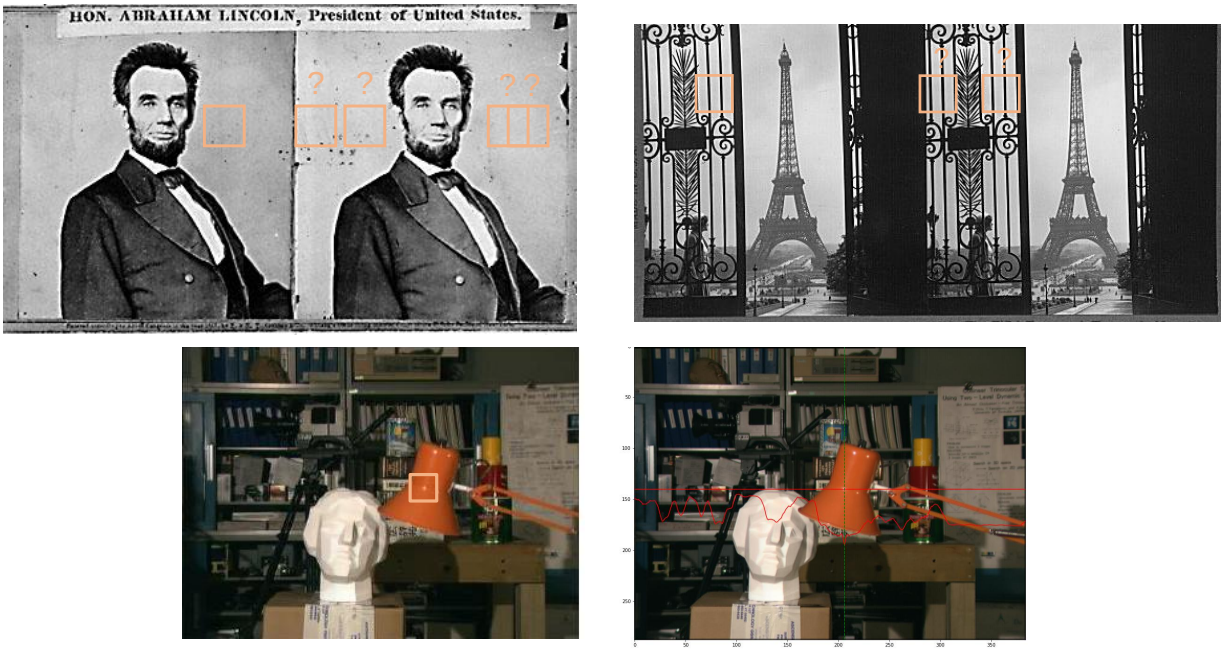


图 8.9 左上：墙面缺少可定位的纹理。右上：栅栏形成重复图案。下：高光和反射随视角变化，左右窗口的亮度排列也会改变。

8.4 遮挡与匹配约束

半遮挡

在前景物体的边缘附近，一部分背景可能只被左相机看见，另一部分可能只被右相机看见。这种只在一幅图中可见的区域称为*半遮挡区域*。其中的像素在另一幅图中没有真实的对应点，匹配结果需要为它们保留“无对应”的状态。

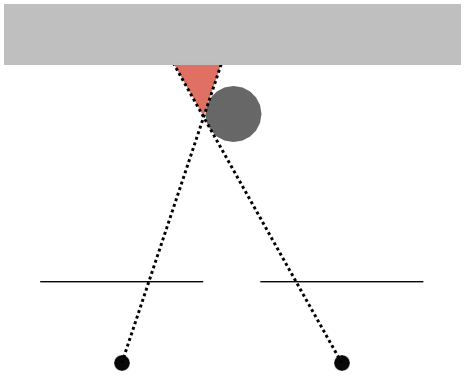


图 8.10 灰色圆形物体挡住了后面的表面。红色背景区域能被左相机看见，却被右相机前方的圆形物体挡住，因此在右图中没有对应像素。

左右一致性

可以从左图出发求一次视差，再从右图出发求一次。统一采用 $u_L - u_R$ 的符号，记左图视差为 $d_L(u_L, v)$，右图位置上记录的视差为 $d_R(u_R, v)$。从左像素出发，先到达 $u_R = u_L - d_L(u_L, v)$，

再沿右图记录的对应关系返回，得到 $\hat{u}_L = u_R + d_R(u_R, v)$。若往返一致，则 $\hat{u}_L$ 应接近原来的 u_L，也就是

$$\left| d_L(u_L, v) - d_R(u_L - d_L(u_L, v), v) \right| \le \tau. \tag{8.13}$$

τ 是允许的像素误差。若右图位置不是整数，可在有效邻域内插值；若位置越界或右图视差本身无效，就把这次检查标为无效。

一致性检查可以筛出许多遮挡和错误匹配。两个方向也可能恰好选中同一组错误对应，例如都错配到相邻的重复纹理，因此还要结合匹配代价和邻域关系判断。某些软件把右图视差定义为 $u_R - u_L$；使用它们的输出时，式 (8.13) 中的差相应变为两个视差之和。

唯一性、顺序与平滑性

局部方法每次只处理一个窗口。把一整行或整幅图放在一起考虑时，还可以利用不同像素之间的关系。

对于一个不透明表面点，如果它在两幅图中都可见，就分别投影到一个位置。因此，离散匹配通常采用*唯一性约束*：一个像素至多参与一对匹配。遮挡像素可以不参与匹配。透明物体和像素内混合了多个表面的区域，需要更复杂的成像模型。

*顺序约束*要求：若两点在左图中从左到右排列，它们在右图中的对应点也按这个顺序排列。这一假设适合许多连续可见的表面段，但不同深度的细小物体可以产生顺序反转。例如，取 $f_x = 800$ 像素、$B = 0.12$ 米，主点相同。两个互不遮挡的小物体点在左相机中的坐标分别为 $(0, 0, 4)$ 米和 $(0.05, 0, 1)$ 米。它们相对主点的左图横坐标是 $0, 40$，右图横坐标却是 $-24, -56$，左右顺序发生了变化。

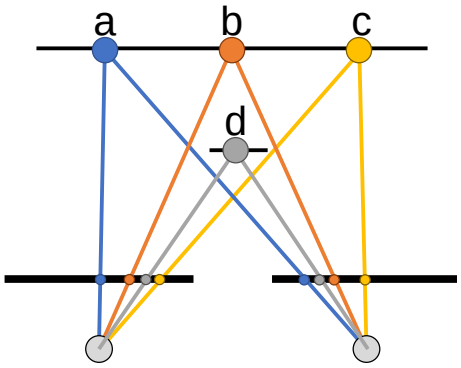


图 8.11 远处的 a, b, c 沿同一表面排列，近处的 d 在两幅图中相对它们的位置会改变。使用顺序约束时，需要考虑这种前后分离的物体布局。

*平滑性*来自表面形状：同一连续表面上，相邻像素的深度通常接近，视差也就变化较缓。斜面可以产生连续变化的视差，物体轮廓处则可能出现明显跳变。后面的全局方法会给相邻视差的差异增加代价，同时允许保留这样的边界。

8.5 扫描线动态规划

一行图像中可能有许多相似窗口。若同时要求匹配保持顺序，并允许部分像素没有对应点，就需要比较一整行的配对方式。动态规划把这个计算拆成较短前缀上的问题，每个前缀的最优结果只计算一次。

代价表与视差空间

设左扫描线有 m 个像素，右扫描线有 n 个像素。用 i, j 分别表示两条扫描线中的位置，$e(i,j)$ 表示将左位置 i 与右位置 j 配成一对的代价。它可以由前面的窗口 SSD 或 ZNCC 计算。所有 $e(i,j)$ 构成一个二维表：一条轴枚举左位置，另一条轴枚举右位置。

若两幅图使用对齐的像素坐标，候选视差为 $d = i - j$。因此 $d_{\min} \le i - j \le d_{\max}$ 对应表中的一条斜带。把这条斜带按 $d = i - j$ 重新排列，就得到以左位置 i 和视差 d 为坐标的表，称为*视差空间图像*（disparity space image，DSI）。它保存某一条扫描线的所有候选匹配代价。

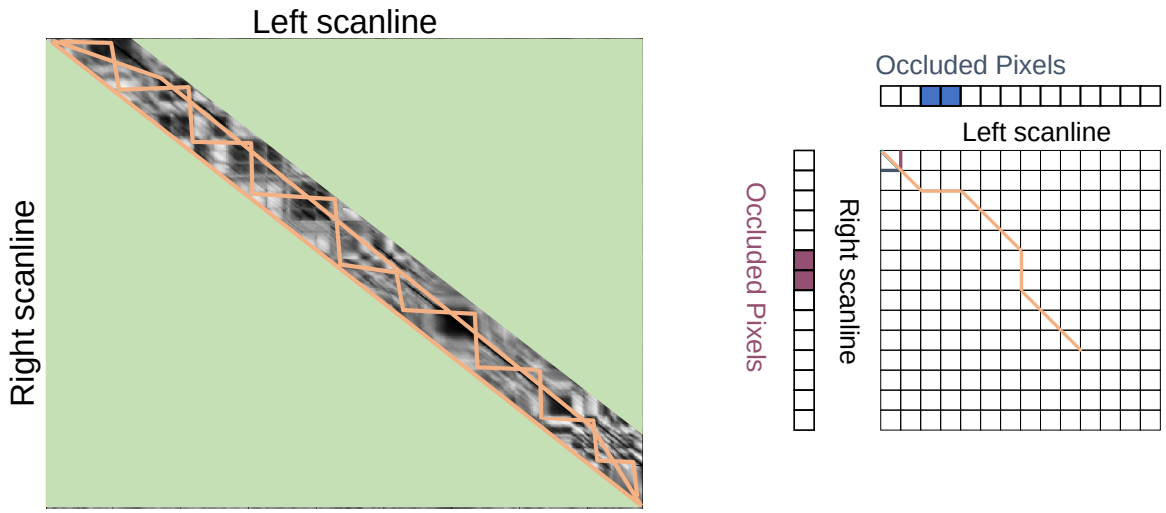


图 8.12 左：代价表的横轴为左图位置，纵轴为右图位置，绿色区域表示视差范围以外的位置，橙色线表示选出的匹配路径。右：路径的三种移动，斜向表示配对，横向或竖向表示跳过一侧的像素。

状态与递推

令 $D(i,j)$ 表示：已经处理左边前 i 个像素和右边前 j 个像素时，所有合法配对方式中最小的总代价。设每留下一个未匹配像素，增加代价 $\gamma > 0$。处理到 (i,j) 的最后一步有三种可能。

如果最后把 i, j 配成一对，前面已经处理到 $(i-1, j-1)$，新增代价为 $e(i,j)$。如果左位置 i 没有对应点，前面处理到 $(i-1, j)$，新增代价为 γ。如果右位置 j 没有对应点，则从 $(i, j-1)$ 到达，并增加同样的代价。所以递推式为

$$D(i,j) = \min \begin{cases} D(i-1, j-1) + e(i,j), & \text{配对} i, j, \\ D(i-1, j) + \gamma, & \text{左像素未匹配}, \\ D(i, j-1) + \gamma, & \text{右像素未匹配}. \end{cases} \tag{8.14}$$

空前缀给出边界条件：

$$D(0,0) = 0, \qquad D(i,0) = i\gamma, \qquad D(0,j) = j\gamma. \tag{8.15}$$

例如，右侧还没有任何像素时，左侧已有的 i 个像素只能全部留下未匹配。视差范围之外的配对可以令 $e(i,j) = +\infty$，从而排除相应的斜向转移。

为什么每个前缀只需要保存最优代价？假设一条最优路径到达 (i,j) 前经过 $(i-1, j-1)$。若这段前缀还可以换成更便宜的路径，接上同一个最后配对，整条路径的代价也会降低。因此，最优路径所使用的前缀必然也是对应状态的最优路径。另外两种最后一步同理，三个候选就覆盖了所有合法情形。

手算与回溯

考虑三个左像素和两个右像素，令未匹配代价为 $\gamma = 2$，匹配代价表为

$$e = \begin{bmatrix} 0 & 9 \\ 9 & 9 \\ 9 & 0 \end{bmatrix}.$$

左侧第一个像素适合与右侧第一个像素匹配，左侧第三个像素适合与右侧第二个像素匹配。左侧第二个像素的两个候选代价都很大。按式 (8.14)逐格计算，得到

$D(i,j)$	$j=0$	$j=1$	$j=2$
$i=0$	0	2	4
$i=1$	2	0	2
$i=2$	4	2	4
$i=3$	6	4	2

例如，$D(2,1) = \min\{2+9, 0+2, 4+2\} = 2$，选择的是“保留第一对匹配，跳过左侧第二个像素”。最后 $D(3,2) = 2$，对应两对零代价匹配加一个未匹配像素。

计算每一格时，同时记录选中的前驱位置。从 (m,n) 反向沿前驱回到 $(0,0)$，便能恢复所有配对和未匹配位置。所有转移都只消耗尚未处理的像素，所以路径自动满足唯一性和顺序约束。若多个候选代价相等，可以按固定次序选择其中一个最优解。

总代价表有 $(m+1)(n+1)$ 个状态，每格比较三个候选，时间复杂度为 $O(mn)$。算法框中的实现保存了完整表和回溯方向，空间复杂度也是 $O(mn)$。若只需要最小代价，可以只保留相邻两行；恢复匹配路径还需要保存或重新计算前驱信息。

遮挡填补

动态规划标记出的未匹配位置形成视差图中的空洞。在典型的不透明前景轮廓附近，这些位置常属于被另一视角挡住的背景。若空洞左右都有可信视差，可以用较小的视差作为一种背景填补办法，因为较小视差对应较远表面。如果已知遮挡发生在哪一侧，也可以沿扫描线从背景一侧向空洞延续视差。

填补采用了周围表面的形状假设。保存结果时，应同时保存原始有效掩码，这样后续测量可以区分实际匹配得到的深度与填补得到的深度。另外，逐行动态规划只联合处理同一行，相邻两行可能选择不同的匹配方式，形成横向条纹。接下来会把竖直方向的邻居也加入计算。

> **备注 | 图像缩放中的接缝搜索**
>
> 课件还用内容感知缩放说明动态规划。以竖直接缝为例，从每一行选一个像素，相邻两行所选列的位置最多相差一格。令 $g(v,u)$ 为像素的重要程度，例如梯度幅值，$S(v,u)$ 为到达该像素的最小累计代价，则
>
> $$S(v,u) = g(v,u) + \min_{\delta \in \{-1,0,1\}} S(v-1, u+\delta).$$

算法实现 ｜ 一条扫描线的匹配

输入 `e` 是 $m \times n$ 的匹配代价数组，`gap` 是未匹配代价。代码返回配对、两侧未匹配位置和最小总代价，索引从零开始。

```
import numpy as np

def scanline_dp(e, gap):
    m, n = e.shape
    D = np.zeros((m + 1, n + 1))
    D[:, 0] = np.arange(m + 1) * gap
    D[0, :] = np.arange(n + 1) * gap
    T = np.zeros_like(D, dtype=np.uint8)
    T[1:, 0], T[0, 1:] = 1, 2
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            costs = (D[i-1, j-1] + e[i-1, j-1],
                     D[i-1, j] + gap, D[i, j-1] + gap)
            T[i, j] = np.argmin(costs)
            D[i, j] = costs[T[i, j]]
    pairs, left_only, right_only = [], [], []
    i, j = m, n
    while i or j:
        step = T[i, j]
        if step == 0:
            pairs.append((i-1, j-1)); i -= 1; j -= 1
        elif step == 1:
            left_only.append(i-1); i -= 1
        else:
            right_only.append(j-1); j -= 1
    return (pairs[::-1], left_only[::-1],
            right_only[::-1], D[m, n])
```

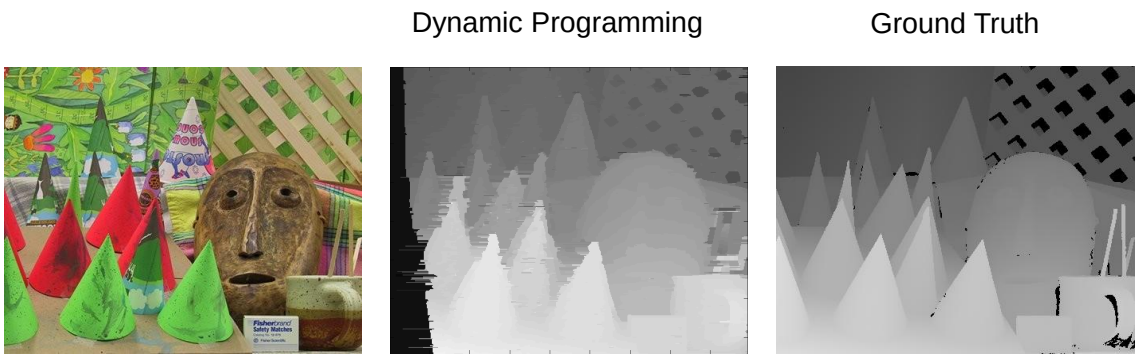


图 8.13 左：输入图像。中：动态规划匹配并填补遮挡后的结果。右：参考视差。填补使表面更连续，物体边界附近仍可能存在偏差。

第一行用 $S(1,u)=g(1,u)$ 初始化，图像外的状态取 $+\infty$。从最后一行的最小值回溯，就得到一条总代价最小的接缝。删去接缝上的像素可以缩窄图像，水平接缝则用于降低图像高度。它与扫描线匹配都利用了最优前缀的递推关系，状态和允许的转移各由具体任务决定。课件中的全景图矩形化还结合了局部调整与整体网格变形，以填满边界并尽量保持直线形状。

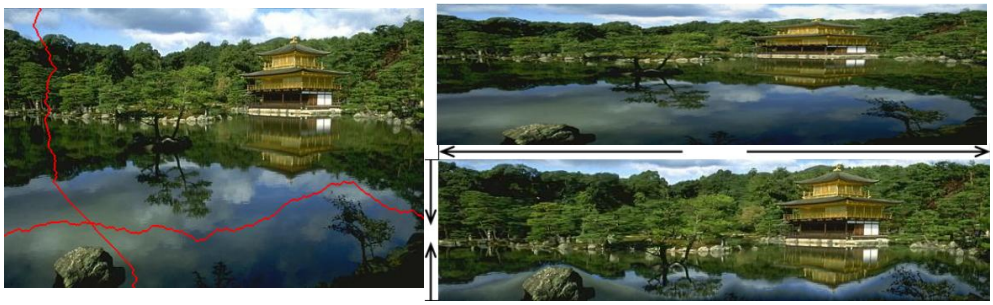


图 8.14 左图标出一条横向接缝，右侧展示改变图像高度后的效果。接缝沿累计代价较小的位置穿过图像，尽量避开需要保留的物体结构。

8.6 整幅图像的视差优化

数据项与平滑项

现在为整幅左图的每个像素 $p=(u,v)$ 指定视差 d_p。记 $\mathcal{N}$ 为相邻像素对的集合，例如每个像素与右侧、下侧像素组成的无向边，这样每一对邻居只计算一次。常见的总能量写成

$$E(d)=\underbrace{\sum_{p} C(p,d_p)}_{\text{数据项}}+\lambda\underbrace{\sum_{(p,q)\in\mathcal{N}} w_{pq}V(d_p,d_q)}_{\text{平滑项}}. \tag{8.16}$$

数据项沿用局部窗口的匹配代价，鼓励每个像素选择外观相似的对应点。平滑项比较相邻像素的视差，$\lambda\ge 0$ 控制它对总分的影响。当 $\lambda=0$ 时，各像素可以独立求最小值；当 $\lambda>0$ 时，一个像素改变视差，还会改变它与邻居之间的代价。

最简单的选择是 $V(d_p,d_q)=|d_p-d_q|$。为了保留物体边界，可以将惩罚截断为

$$V(d_p,d_q)=\min\{|d_p-d_q|,\eta\},\qquad \eta>0.$$

小差异按大小计费，差异超过 η 后费用保持不变，因而允许少数位置出现较大的视差跳变。还可以令

$$w_{pq}=\exp\!\left(-\frac{\|I_L(p)-I_L(q)\|^2}{2\sigma_I^2}\right),$$

使颜色差别较大的相邻像素受到较弱的平滑约束。颜色边缘只是一个线索：同一平面上的花纹也会产生颜色变化，而同色物体之间也可能存在深度边界。

图割与半全局匹配

二维邻接关系会形成环，例如一个 2×2 方块的四条边。这时只保存上一行的单个最优视差，已经不足以描述所有邻居之间的影响。课件介绍的图割方法把像素和视差选择写成图上的优化问题。若每个像素只有 $0,1$ 两个候选标签，且每对邻居的代价满足 $V(0,0)+V(1,1)\le V(0,1)+V(1,0)$，

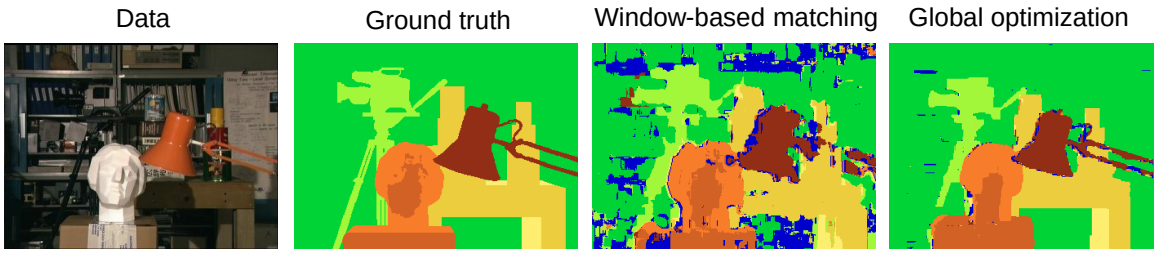


图 8.15 从左到右为输入图像、参考视差、窗口匹配和全局优化结果。加入相邻像素关系后，大块表面内的视差更加连续，部分局部错配得到修正。

这种条件称为*次模性*，最小割可以求出相应成对能量的全局最小值。这个不等式比较了邻居取相同标签与不同标签的两组总代价。多个离散视差通常通过一系列图割步骤逐步降低能量，得到近似解。允许使用哪些图割步骤，取决于所选的平滑代价。

实际双目处理中还常用*半全局匹配*（SGM）。它沿多个方向分别做一维动态规划，再汇总各方向的代价。设 r 是一条扫描方向，$L_r(p,d)$ 表示沿该方向到达像素 p、选择视差 d 的累计代价。一种常用的递推为

$$
\begin{aligned}
L_r(p,d) = C(p,d) + \min\big\{ & L_r(p-r,d), \\
& L_r(p-r,d-1)+P_1,\; L_r(p-r,d+1)+P_1, \\
& \min_{k} L_r(p-r,k) + P_2 \big\} - \min_{k} L_r(p-r,k).
\end{aligned}
\tag{8.17}
$$

同一视差不增加惩罚，改变一个视差单位增加 P_1，较大的改变增加 P_2，通常取 $P_2 > P_1 > 0$。末尾减去的最小值对当前所有候选都相同，用于避免累计数值不断增大。路径起点直接取 $L_r(p,d) = C(p,d)$，视差范围以外的候选取 $+\infty$。最后汇总各方向的 L_r，为每个像素选择总分最小的视差。

多方向汇总把上下、左右以及部分斜向邻居的信息都用起来，能减轻逐行处理的条纹。SGM 的计算由多条一维路径组成；二维能量的精确最小值仍取决于完整的像素邻接关系。

联合双边上采样

在高分辨率图像上计算代价体较耗时。一种做法是先缩小左右图像、估计低分辨率视差，再用原图的边缘帮助上采样。这与第三章的联合双边上采样相同：距离较近、引导图颜色也相近的样本获得较大权重。

若原图的宽、高都缩小为原来的 s 倍，且两幅图使用一致的缩放方式，则 $f_x^{\text{low}} = s f_x$、$d^{\text{low}} = s d$。因此，插值后的视差需要再除以 s，才能恢复成原图的像素单位。例如，四分之一宽度图像上的 6 像素视差，对应原图中的 24 像素。若上采样的是以米为单位的深度，长度单位保持不变。引导图能帮助保持颜色边缘，但低分辨率匹配中已经丢失的细小物体，仍需要额外信息才能恢复。

8.7 双目深度的计算示例

下面把校正、视差估计和深度换算连起来。输入是同一时刻拍摄的两张同尺寸灰度图，以及标定所得的 K_L, K_R、畸变参数 D_L, D_R 和相对位姿 $R, \boldsymbol{t}$。图像使用 `uint8` 类型，内参对应输入图像的分辨率，平移继续满足 $\boldsymbol{X}' = R\boldsymbol{X} + \boldsymbol{t}$。

校正图像

`stereoRectify` 给出两台相机各自的校正旋转、新投影矩阵和重投影矩阵 Q。随后将去畸变与校正合成一次采样映射。这里使用 `CALIB_ZERO_DISPARITY` 使两幅校正图的主点对齐，并令 `alpha=0` 选择尽量去掉无效边界的缩放方式。返回的有效区域仍用于后续匹配。

算法实现 | 双目图像校正

```
import cv2
import numpy as np

def rectify_pair(left, right, KL, DL, KR, DR, R, t):
    h, w = left.shape
    t = np.asarray(t, dtype=np.float64).reshape(3, 1)
    RcL, RcR, PL, PR, Q, roiL, roiR = cv2.stereoRectify(
        KL, DL, KR, DR, (w, h), R, t,
        flags=cv2.CALIB_ZERO_DISPARITY, alpha=0)
    mxL, myL = cv2.initUndistortRectifyMap(
        KL, DL, RcL, PL, (w, h), cv2.CV_32FC1)
    mxR, myR = cv2.initUndistortRectifyMap(
        KR, DR, RcR, PR, (w, h), cv2.CV_32FC1)
    L = cv2.remap(left, mxL, myL, cv2.INTER_LINEAR)
    Rimg = cv2.remap(right, mxR, myR, cv2.INTER_LINEAR)
    return L, Rimg, PL, PR, Q, roiL, roiR
```

校正后可以找几对清楚的角点，查看它们的纵坐标是否接近。若仍存在明显的竖直偏移，就需要重新检查标定、输入尺寸和图像对应关系。以下代码针对基线沿水平方向、右相机位于左相机右侧的配置。此时 P_R 的第四列为 $(-f_x B, 0, 0)^{\mathsf{T}}$，可由它读出基线长度。

视差与三维坐标

OpenCV 的 `StereoSGBM` 按半全局方法计算视差。`numDisparities` 表示候选数，取 16 的倍数；示例选择从 0 到 127 的整数候选，并在内部作亚像素估计。实际候选范围可按目标深度调整。P_1, P_2 在示例中随窗口面积设置，图像噪声和物体细节不同，合适的取值也会变化。

`compute` 返回的 16 位整数把视差放大了 16 倍，因此先除以 16 恢复像素单位。无效标记随 `minDisparity` 改变，代码按相同约定过滤。Q 将 $(u, v, d, 1)^{\mathsf{T}}$ 映射成齐次三维坐标，再除以最后一个分量，得到校正后左相机坐标系中的点。其第三个坐标应与上面的 Z 一致。

有效掩码排除了匹配范围外和程序标记无效的位置。观察结果时，还可以结合式 (8.13)的往返误差、代价曲线和原图边界判断质量。如果需要原左相机坐标系下的点，对校正后的三维坐标乘以 R_{cL}^{T} 即可。

算法实现 | 视差、深度与点云

输入为上一个算法返回的变量。输出 Z 和 xyz 的长度单位与标定平移相同。

```
min_d, count, block = 0, 128, 5
matcher = cv2.StereoSGBM_create(
    minDisparity=min_d, numDisparities=count, blockSize=block,
    P1=8*block**2, P2=32*block**2,
    uniquenessRatio=10, disp12MaxDiff=1,
    speckleWindowSize=50, speckleRange=2,
    mode=cv2.STEREO_SGBM_MODE_SGBM_3WAY)
raw = matcher.compute(L, Rimg)
d = raw.astype(np.float32) / 16.0
x, y, w, h = cv2.getValidDisparityROI(
    roiL, roiR, min_d, count, block)
valid = np.zeros(d.shape, dtype=bool)
valid[y:y+h, x:x+w] = True
fx = PL[0, 0]
B = -PR[0, 3] / fx
delta = d - (PL[0, 2] - PR[0, 2])
valid &= (raw > (min_d-1)*16) & (delta > 0)
Z = np.full(d.shape, np.nan, dtype=np.float32)
Z[valid] = fx * B / delta[valid]
xyz = cv2.reprojectImageTo3D(d, Q)
xyz[~valid] = np.nan
```

8.8 学习匹配代价与深度

学习窗口特征

SSD 直接比较像素灰度，ZNCC 先消去局部亮度的平移和缩放。另一种办法是从数据中学习怎样表示一个图像窗口，使同一物体点周围的窗口得到相近的特征。课件介绍的卷积网络分别接收左右窗口，并在两条分支中使用相同的权重。

记网络为 ϕ_θ，对非零输出作长度归一化，得到

$$\boldsymbol{h}_L = \frac{\phi_\theta(\boldsymbol{w}_L)}{\|\phi_\theta(\boldsymbol{w}_L)\|}, \qquad \boldsymbol{h}_R = \frac{\phi_\theta(\boldsymbol{w}_R)}{\|\phi_\theta(\boldsymbol{w}_R)\|}.$$

可以用内积 $s = \boldsymbol{h}_L^{\top}\boldsymbol{h}_R$ 衡量相似程度，再用 $C = 1 - s$ 作为匹配代价。实现中会给归一化分母加一个很小的正数，以处理接近零的特征。

训练时，可以为一个左窗口提供真实的右窗口和一个错误候选，分别得到分数 s_+、s_-。例如，间隔损失

$$\mathcal{L} = \max\{0, m - s_+ + s_-\}, \qquad m > 0,$$

要求真实匹配的分数比错误候选至少高出 m。当差距已经足够大时，这一组样本的损失为零；否则继续调整网络参数。这种损失训练的是窗口之间的匹配关系。训练好的网络可以逐个候选计算代价，再交给前面的代价汇总与视差选择步骤。

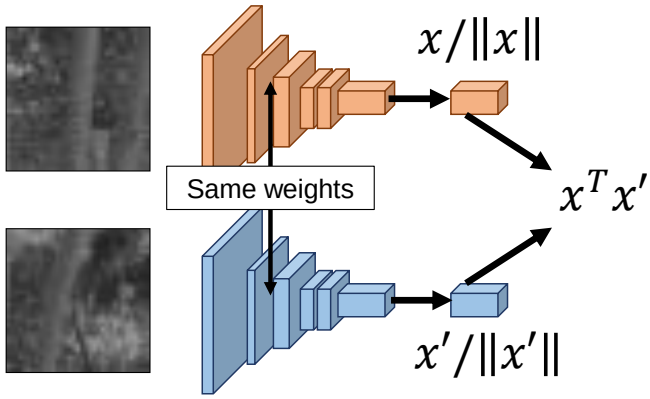


图 8.16 左右窗口经过使用相同权重的卷积网络，得到归一化特征，再比较特征的内积。网络学习哪些局部图像变化可以容许，哪些差别有助于区分位置。

利用图像重建训练深度

如果训练数据有已标定的左右图像，还可以通过图像重建训练深度网络。课件中的方法在训练时使用双目图像，让网络从左图预测深度 $Z_\theta(u,v)$。在这里采用的平行双目设置下，先将预测深度换成视差 $d_\theta(u,v) = f_x B / Z_\theta(u,v)$，再到右图的对应位置采样：

$$\hat{I}_L(u,v) = I_R\big(u - d_\theta(u,v), v\big). \tag{8.18}$$

若预测深度正确、该点在两图中都可见，且亮度随视角变化较小，重建的 $\hat{I}_L$ 就应接近原左图 I_L。非整数位置用双线性插值，因此采样结果可随预测视差连续变化。

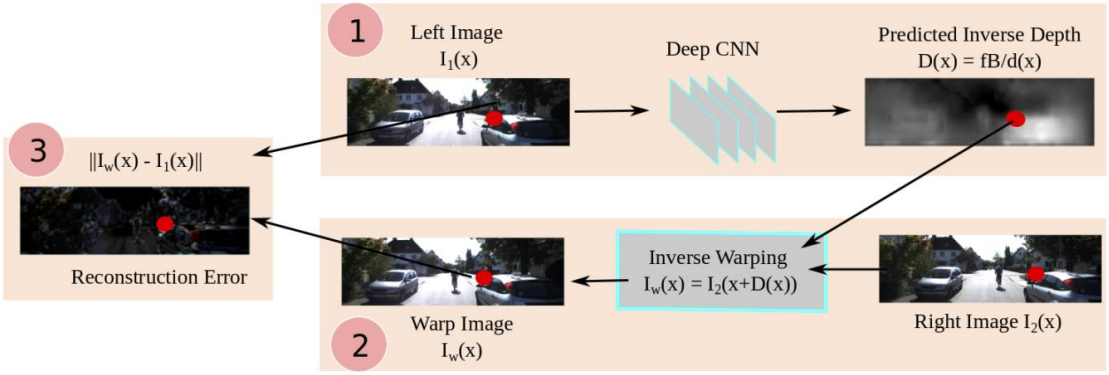


图 8.17 网络利用已知双目几何把右图采样成左视角，再比较重建图与原左图。图中的 $D(x)$ 表示预测的水平位移，采用右图采样位置为 $x + D(x)$ 的符号约定；正文使用 $d = u_L - u_R$，采样位置相应写为 $u - d$。

令 Ω_{vis} 表示对应位置在图内、且通过可见性判断的像素集合，一种基本的重建损失是

$$\mathcal{L}_{\text{photo}} = \frac{1}{|\Omega_{\text{vis}}|} \sum_{(u,v)\in\Omega_{\text{vis}}} \left| I_L(u,v) - \hat{I}_L(u,v) \right|.$$

式中假设有效集合非空。遮挡区域需要可见性掩码或稳健的损失处理，无纹理区域则常结合深度平滑项。高光、透明表面、不同曝光和拍摄时的运动也会影响重建误差。

这种训练方法使用双目图像提供几何约束，训练好以后可以只输入一张图预测深度。双图匹配在推理时还会直接比较另一幅图；单图网络则利用训练中学到的外观与深度关系。两者对输入信息的使用方式由此不同。

数据与评估

课件列出的数据包括 Middlebury、KITTI 和合成场景。Middlebury 常用于室内场景和细节评估，KITTI 提供车载场景，合成数据则便于同时生成图像与已知的深度、视差。比较方法时，应在相同的有效像素集合上计算误差，并单独说明遮挡和边界的处理。例如，平均绝对视差误差和超过阈值的比例分别为

$$\mathrm{MAE} = \frac{1}{|\Omega|}\sum_{p\in\Omega}|d_p - d_p^*|, \qquad \mathrm{Bad}_\tau = \frac{1}{|\Omega|}\sum_{p\in\Omega}\mathbf{1}\{|d_p - d_p^*| > \tau\}.$$

这里 d_p^* 是参考视差，Ω 是用于评估的有效位置集合。阈值 τ 的单位是像素，不同数据集的正式评测还可能使用相对误差等条件。根据式 (8.5)，同样的视差误差在不同距离上对应不同的深度误差，因此测距任务还应结合目标距离检查结果。

8.9 主动立体视觉

前面的方法依靠物体本身的纹理寻找对应点。对于一面颜色均匀的墙，可以主动投射一组图案，让相机看到更多可区分的位置。主动立体视觉中的几何计算仍然利用视线与基线，主要变化在于对应信息的来源。

结构光

结构光系统把条纹、彩色编码或点阵投到物体上。若图案编码能识别某个相机像素来自投影器的哪个位置，就得到相机与投影器之间的一组对应关系。投影器的每个像素发出一个方向已知的光束，因此在标定后，可以把它看成沿相反方向工作的相机：相机像素确定接收视线，投影器像素确定投射视线。两条视线给出物体点的位置。

图 8.18 左：普通外观。中：投射彩色条纹后的表面。右：由条纹对应关系重建的形状。不同颜色及其排列帮助确定投影图案中的位置。

单张图案可以依靠空间上的编码识别位置；多帧图案则可以让同一个位置接收不同的亮暗序列。使用多帧时，需要物体在采集期间保持足够稳定，或在计算中补偿运动。另一些双目系统使用投射点阵为场景增加纹理，仍在两台相机之间寻找对应点。这时投影器主要负责照明，三角测量使用的是两相机之间的几何关系。

课件中的第一代 Kinect 展示了红外结构光的应用，TrueDepth 也使用红外投射图案获取面部深度。红外图案便于在不明显改变可见光外观的情况下提供纹理。测量质量还取决于投射光能否被相机清楚接收：强环境光会降低图案对比度，吸收较强、透明或镜面反射明显的表面也会改变接收到的光。

激光扫描

若投射的是一条激光线，它在三维空间中对应一个已标定的光平面。相机中检测到的亮线像素确定一条视线，求这条视线与光平面的交点，就能恢复空间位置。

将所有量写在相机坐标系中，令光平面为 $\boldsymbol{n}^{\mathsf{T}}\boldsymbol{X} + h = 0$，其中 $\boldsymbol{n}$ 是法向量，h 是平面常数。去畸变像素 $\tilde{\boldsymbol{x}}$ 对应方向 $\boldsymbol{q} = K^{-1}\tilde{\boldsymbol{x}}$，空间点可以写成 $\boldsymbol{X} = \lambda \boldsymbol{q}$。代入平面方程并解出 λ，得到

$$\lambda = -\frac{h}{\boldsymbol{n}^{\mathsf{T}}\boldsymbol{q}}, \qquad \boldsymbol{X} = -\frac{h}{\boldsymbol{n}^{\mathsf{T}}\boldsymbol{q}}\boldsymbol{q}. \tag{8.19}$$

当 $\boldsymbol{n}^{\mathsf{T}}\boldsymbol{q} \neq 0$ 时交点唯一；取 $q_3 = 1$ 后，还要求 $\lambda > 0$，使交点位于相机前方。若分母接近零，视线接近平行于光平面，定位误差对交点的影响会增大。分母为零时，视线与平面可能平行分离，或整条视线位于平面内。

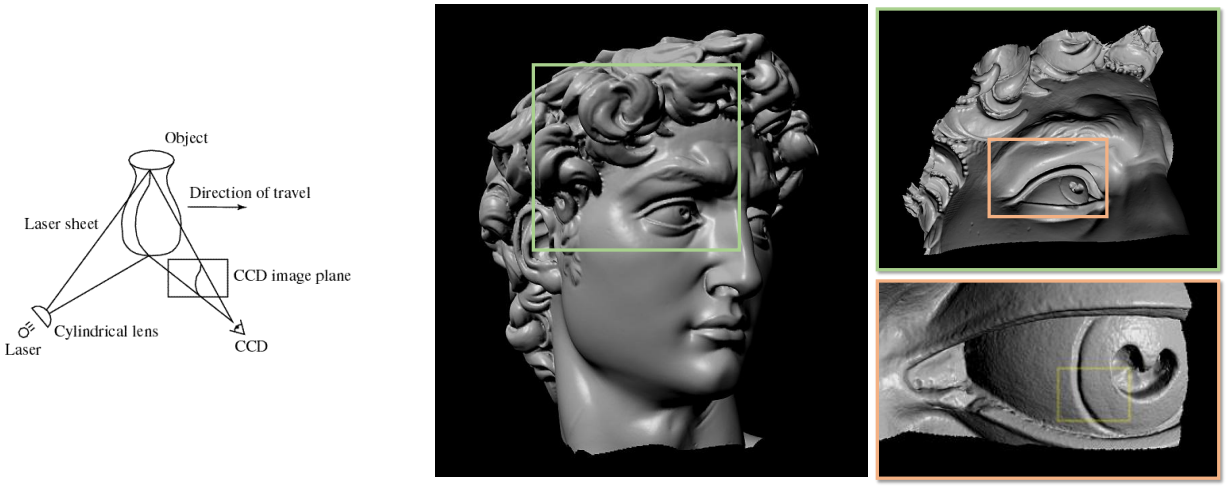


图 8.19 左：激光平面与表面相交形成亮线，相机观测亮线的位置。右：逐步扫描得到的雕塑表面与局部细节。

移动激光平面或物体，每次测得表面上的一条曲线，再把这些曲线放到同一坐标系中，便能逐步得到较完整的表面。扫描期间的相对运动也需要已知或经过估计。

多帧深度融合

一张深度图只包含一个视角可见的表面。移动深度相机后，可以先估计每一帧的相机旋转和平移，将各帧三维点转到同一坐标系，再融合重复观测。这样既能覆盖新的表面，也能减小部分随机测量误差。

课件中的 KinectFusion 用连续深度图跟踪相机，并融合静止场景的表面。DynamicFusion 进一步估计物体随时间的形变，用于处理运动和变形的场景。这些方法接收的深度图，可以来自本章的双目匹配或主动测深；后续计算还要处理不同帧之间的对齐和物体运动。